

Java 27 for Programmers New to Java

- [Java 27, for Programmers New to Java](#)
 - [What "Java 27" means here](#)
 - [Getting started](#)
 - [The curriculum](#)
 - [How to use this](#)
 - [Running examples](#)
 - [Verifying everything](#)
 - [PDFs](#)
 - [Project documents](#)
 - [A note on how this was built](#)
- [Spec: Java 27 Learning Curriculum](#)
 - [Assumptions I'm Making](#)
 - [Objective](#)
 - [Tech Stack](#)
 - [Commands](#)
 - [Project Structure](#)
 - [Curriculum Outline](#)
 - [Code Style](#)
 - [Testing Strategy](#)
 - [Boundaries](#)
 - [Success Criteria](#)
 - [Open Questions, as resolved](#)
 - [Corrections Made During Implementation](#)
 - [Amendment 1 — PDF output and a Turkish translation](#)
- [Module 01: Getting Started](#)
 - [What You'll Learn](#)
 - [Coming From Another Language](#)
 - [The Lesson](#)
 - [Run It](#)
 - [Common Mistakes](#)
 - [Key Takeaways](#)
 - [Homework](#)
- [Homework 01: About Me](#)
 - [The Task](#)
 - [Then Break It](#)
 - [Acceptance Criteria](#)

- Stretch
 - Hint, if you are stuck on the age
- Module 02: Variables and Types
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 02: The Receipt That Adds Up
 - The Task
 - The Money Rule
 - The Rounding Decision
 - Then Reproduce Two Traps
 - Acceptance Criteria
 - Stretch
 - Hint, if the formatting is fighting you
- Module 03: Control Flow
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 03: Grades and Grids
 - Part One: Grades
 - Part Two: Find In Grid
 - Acceptance Criteria
 - Stretch
 - Hint, if the switch will not compile
- Module 04: Methods
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways

- Homework
- Homework 04: What A Method Can And Cannot Change
 - Part One: Inventory
 - Part Two: The Remove Trap
 - Acceptance Criteria
 - Stretch
 - Hint, if part one looks like it already works
- Module 05: Classes and Objects
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 05: An Object That Cannot Be Broken
 - The Task
 - Prove It
 - Also Include
 - Acceptance Criteria
 - Stretch
 - Hint, if rule 4 is slipping through
- Module 06: Inheritance and Polymorphism
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 06: The Set That Loses Things
 - Part One: Break A HashSet
 - Part Two: A Mixed Shelf
 - Acceptance Criteria
 - Stretch
 - Hint, if part one refuses to misbehave
- Module 07: Interfaces
 - What You'll Learn
 - Coming From Another Language

- The Lesson
- Run It
- Common Mistakes
- Key Takeaways
- Homework
- Homework 07: Capabilities, Conflicts and a Fragile Base
 - Part One: Smart Devices
 - Part Two: A Conflict You Must Resolve
 - Part Three: Break A Subclass
 - Acceptance Criteria
 - Stretch
 - Hint, if part three gives you 3 straight away
- Module 08: Records and Enums
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 08: An Order System
 - Part One: Money
 - Part Two: Two Leaks
 - Part Three: Shipping and Status
 - Acceptance Criteria
 - Stretch
 - Hint, if part two's second attack does not work
- Module 09: Sealed Types and Pattern Matching
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 09: An Expression Evaluator
 - Part One: The Model
 - Part Two: Three Operations
 - Part Three: Prove The Compiler Is Watching

- Part Four: Simplify
- Acceptance Criteria
- Stretch
- Hint, if the nested pattern will not compile
- Module 10: Exceptions
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 10: A Config Loader That Tells The Truth
 - Part One: The Loader
 - Part Two: Three Anti-Patterns
 - Part Three: Resources
 - Acceptance Criteria
 - Stretch
 - Hint, if the suppressed list comes back empty
- Module 11: Generics
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 11: A Typed Cache
 - Part One: The Cache
 - Part Two: Bounds and PECS
 - Part Three: Pollute The Heap
 - Acceptance Criteria
 - Stretch
 - Hint, if part three does not throw
- Module 12: Collections
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It

- Common Mistakes
- Key Takeaways
- Homework
- Homework 12: A Word Index
 - The Text
 - Part One: Counting
 - Part Two: Indexing
 - Part Three: Lose An Entry
 - Acceptance Criteria
 - Stretch
 - Hint, if the top-three keeps changing
- Module 13: Lambdas
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 13: A Rules Engine
 - The Data
 - Part One: Composed Predicates
 - Part Two: All Four Method Reference Forms
 - Part Three: The Capture Rule
 - Acceptance Criteria
 - Stretch
 - Hint, if `compose` will not type-check
- Module 14: Streams
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 14: Log Analysis
 - The Data
 - Part One: Grouping
 - Part Two: Gatherers

- Part Three: Race It
- Part Four: Laziness and Single Use
- Acceptance Criteria
- Stretch
- Hint, if your deltas come out one element too long
- Module 15: Optional
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 15: A Contact Book
 - The Data
 - Part One: Lookups Without Unwrapping
 - Part Two: Prove The Trap
 - Part Three: Fix Four Anti-Patterns
 - Part Four: What It Does Not Protect You From
 - Acceptance Criteria
 - Stretch
 - Hint, if part one's `flatMap` point is not landing
- Module 16: Files and IO
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 16: A Log Tool
 - Part One: Stream, Do Not Slurp
 - Part Two: Write A Filtered Copy
 - Part Three: The Path Validator
 - Part Four: Exceptions That Say Something
 - Acceptance Criteria
 - Stretch
 - Hint, if `/etc/passwd` slips through
- Module 17: Standard Library Tour

- What You'll Learn
- Coming From Another Language
- The Lesson
- Run It
- Common Mistakes
- Key Takeaways
- Homework
- Homework 17: A Scheduler That Survives March
 - Part One: Monthly Billing
 - Part Two: A Meeting Across Daylight Saving
 - Part Three: Four Math Traps
 - Part Four: Reproducible Sampling and Formatting
 - Acceptance Criteria
 - Stretch
 - Hint, if your output changes between runs
- Module 18: Concurrency Basics
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 18: A Counter That Cannot Count
 - Part One: Four Counters
 - Part Two: A Loop That Never Ends
 - Part Three: Check Then Act
 - Part Four: The Failure Nobody Hears
 - Acceptance Criteria
 - Stretch
 - Hint, if the unsafe counter comes out correct
- Module 19: Virtual Threads
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework

- Homework 19: A Download Service
 - Part One: Measure It
 - Part Two: The Limit You Removed
 - Part Three: Scoped Values
 - Part Four: What Did Not Change
 - Acceptance Criteria
 - Stretch
 - Hint, if your tracker's peak looks too low
- Module 20: Packages and Modules
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 20: Build It Yourself
 - Part One: Two Packages
 - Part Two: Package It
 - Part Three: Three Startup Failures
 - Part Four: Modules In Practice
 - Acceptance Criteria
 - Stretch
 - Hint, if failure 2 refuses to fail
- Module 21: Testing Your Code
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 21: Build The Framework You Would Otherwise Import
 - Part One: A Better Harness
 - Part Two: Isolation
 - Part Three: The Bug `assert` Hides
 - Part Four: Which Check, Where
 - Acceptance Criteria
 - Stretch

- Hint, if part three shows the removal happening
- Module 22: What's New, Honestly
 - What You'll Learn
 - Coming From Another Language
 - The Lesson
 - Run It
 - Common Mistakes
 - Key Takeaways
 - Homework
- Homework 22: Research It Yourself
 - Part One: Three Features, Researched
 - Part Two: The Three Questions
 - Part Three: Read A History Section
 - Part Four: Check The Version Properly
 - Acceptance Criteria
 - Stretch
 - Hint, if you cannot find the History section
- Implementation Plan: Java 27 Learning Curriculum
 - Verified Preconditions
 - Decisions Taken on Open Questions
 - Components
 - Why This Order
 - Risks and Mitigations
 - Parallel vs. Sequential
 - Verification Checkpoints
- Task List: Java 27 Learning Curriculum
 - scaffold
 - template
 - core-language
 - type-system
 - functional
 - platform
 - release-literacy
 - integration
 - amendment-1 — post-delivery scope

Java 27, for Programmers New to Java

A self-paced curriculum for developers who already write code in another language and now need Java. It does not re-teach what a variable or a loop is. It teaches how Java does things, and where Java will surprise you.

Twenty-two modules. Each has a lesson, runnable examples whose comments explain the *why*, key takeaways, and homework with a reference solution.

Every example in this repository has been run on JDK 27. Every compiler error quoted in a lesson was produced by actually compiling the broken code, and every number was measured rather than recalled.

Türkçe: the whole curriculum is also available in Turkish under [tr/](#). Code, commands and compiler output stay in English there; [tr/CEVIRI-NOTLARI.md](#) explains why.

What "Java 27" means here

[JDK 27](#) reached General Availability on 15 September 2026. It is **not** an LTS release, and its finalised features are almost entirely runtime-level: G1 as the default collector, compact object headers, post-quantum TLS key exchange, flight recorder redaction. Nothing in that list changes how you write Java.

So this curriculum is honest about its own title:

JDK 27 is the runtime and toolchain. The language you learn is the stable Java accumulated through [JDK 25 LTS](#).

Module 22 covers what JDK 26 and 27 actually changed, why non-LTS releases matter less than the version number suggests, and how to read a JEP for yourself.

Getting started

The only requirement is JDK 27. No Maven, no Gradle, no third-party libraries anywhere in this repository.

```
java -version      # you want: java version "27"
```

Installing it

Platform	Command
macOS (Homebrew)	<code>brew install openjdk@27</code>
Linux (SDKMAN)	<code>sdk install java 27-open</code>
Windows (winget)	<code>winget install Microsoft.OpenJDK.27</code>
Any	download from Adoptium or Oracle

Pointing at it

```
# macOS
export JAVA_HOME=$(/usr/libexec/java_home -v 27)

# Linux
export JAVA_HOME=/usr/lib/jvm/jdk-27
export PATH="$JAVA_HOME/bin:$PATH"

# Windows PowerShell
$env:JAVA_HOME = "C:\Program Files\Microsoft\jdk-27"
$env:PATH = "$env:JAVA_HOME\bin;$env:PATH"
```

Your first run

```
java modules/01-getting-started/examples/HelloWorld.java
```

No compile step and no build file. Module 01 explains what is actually happening.

The curriculum

Work through them in order. Each module's prose assumes you have read the ones before it.

#	Module	What it covers
01	getting-started	JVM, JRE and JDK; running a file directly; reading a compiler error
02	variables-and-types	Primitives against references, autoboxing traps, <code>var</code> , text blocks
03	control-flow	No truthiness, switch expressions, exhaustiveness, labelled break
04	methods	Pass-by-value including for objects, overload resolution, varargs
05	classes-and-objects	The classic <code>main</code> , encapsulation, static factories, flexible constructors
06	inheritance-and-polymorphism	Dynamic dispatch, <code>abstract</code> against <code>final</code> , the <code>equals</code> / <code>hashCode</code> contract
07	interfaces	Capabilities, default methods, the diamond conflict, fragile base classes
08	records-and-enums	Records, compact constructors, shallow immutability, enums with behaviour
09	sealed-and-pattern-matching	Sealed types, record patterns, guards, why exhaustiveness is the payoff
10	exceptions	Checked against unchecked, try-with-resources, the three ways failures vanish
11	generics	Erasure and what it forbids, heap pollution, PECS, invariance
12	collections	Choosing an implementation, sequenced collections, why keys must not change
13	lambdas	Method reference forms, capture and effective finality, the standard interfaces
14	streams	Laziness, collectors, gatherers, and the parallel data race
15	optional	What it is for, <code>orElse</code> against <code>orElseGet</code> , five anti-patterns
16	files-and-io	<code>Path</code> against <code>Files</code> , path traversal, streaming, why not <code>java.io.File</code>
17	standard-library-tour	Strings, <code>java.time</code> , daylight saving, <code>Math</code> traps, reproducible randomness
18	concurrency-basics	Races and visibility as separate hazards, <code>synchronized</code> , atomics, executors
19	virtual-threads	Carriers and unmounting, the benchmark, pinning today, scoped values
20	packages-and-modules	The directory rule, the classpath, jars, and why <code>--add-opens</code> exists
21	testing-your-code	Why <code>assert</code> is not a testing tool; a test harness in 39 lines
22	whats-new	What JDK 26 and 27 really changed, LTS, and how to read a JEP

How to use this

Read the module README, run the examples, then do the homework **before** opening the solution.

```
modules/<nn>--<name>/
├─ README.md      the lesson, ending in Key Takeaways
├─ examples/      runnable files, heavily commented
├─ homework/      the assignment and its acceptance criteria

solutions/<nn>--<name>/
├─ reference solutions, kept out of the lesson tree on purpose
```

The solutions are for checking yourself against, not for reading first. Each one is commented to explain *why* it made the choices it did, so it is worth reading after you have your own answer, and close to worthless before.

Every homework has explicit acceptance criteria. If your version meets them, it is correct even where it differs from the reference.

Running examples

```
# an ordinary example
java modules/02-variables-and-types/examples/TheAutoboxingTrap.java

# one that needs assertions
java -ea modules/21-testing-your-code/examples/UsingAssertions.java

# one that needs a preview feature
java --enable-preview --source 27 modules/22-whats-
    new/examples/PreviewStructuredConcurrency.java
```

Some examples fail **on purpose**, because a compiler error you produced yourself teaches more than one quoted at you. Each declares what it expects in its first ten lines:

Marker	Meaning
(none)	Compiles and runs, exits 0
// EXPECT: compile-error	Must fail to compile; it is teaching a compile error
// EXPECT: runtime-error	Must compile, then crash at runtime
// EXPECT: compile-only	Must compile; not run on its own
// EXPECT: assertions	Run with <code>-ea</code> , so <code>assert</code> statements are live
// EXPECT: preview	Run with <code>--enable-preview --source 27</code>

Verifying everything

```
./scripts/verify-examples.sh # the whole repository
./scripts/verify-examples.sh modules/09-sealed-and-pattern-matching # one module
```

The script runs every example and every reference solution, checks each behaves as its marker declares, and refuses to run at all on a JDK other than 27, because an example failing for toolchain reasons teaches you nothing.

Current state of this repository:

Modules	22
Example files	105
Reference solutions	25
Files verified	130
Full verification time	about 70 seconds

PDFs

Every lesson and homework is also available as a PDF, plus the whole curriculum as a single 209-page book.

```
./scripts/build-pdfs.sh # both languages, 99 PDFs
./scripts/build-pdfs.sh --en # English only -> pdf/
./scripts/build-pdfs.sh --tr # Turkish only -> pdf-tr/
./scripts/build-pdfs.sh --book # just the combined books
```

Output	What
pdf/Java-27-Curriculum.pdf	the complete book, with a table of contents
pdf-tr/Java-27-Mufredat.pdf	the same book in Turkish
pdf/NN-<module>.pdf	one lesson
pdf/NN-<module>-homework.pdf	one homework
pdf/00-overview.pdf, 00-spec.pdf, 00-plan.pdf, 00-tasks.pdf	this file and the project documents

The script needs [pandoc](#) and a Chromium-based browser. Chrome is used as the PDF engine because pandoc's default requires a LaTeX installation, and Chrome renders these tables and code blocks correctly without one. Rebuild after editing any Markdown; the whole set takes about a minute.

Project documents

- [SPEC.md](#) — what this project set out to build, and its success criteria
 - [tasks/plan.md](#) — the implementation plan
 - [tasks/todo.md](#) — the task breakdown, linked to issues
-

A note on how this was built

Every factual claim in these lessons was produced rather than recalled. Compiler errors were generated by compiling the broken code. Benchmarks were measured. JEP numbers, release dates and LTS designations were read from openjdk.org.

That discipline caught real mistakes during writing, including a composition example that disproved its own claim, a bucket-index demonstration where the chosen numbers hid the bug entirely, and a "what's new" history that turned out to have two more rounds than assumed. Each correction is recorded in the git history.

If you find something here that is wrong, the JEP index outlives this document. Check the source.

Spec: Java 27 Learning Curriculum

Status: Delivered, then amended — see *Amendment 1* below **Original scope:** all 24 tasks complete, verified 2026-09-23 **Date:** 2026-09-23 **Phase:** 1 (Specify) of Specify → Plan → Tasks → Implement

Assumptions I'm Making

1. The learner already programs in another language (Python, JavaScript, C#, ...) and is new to **Java specifically**. Modules do not re-teach what a variable or a loop is; they teach how Java does it and where Java differs.
2. "Takeout" in the original request means **Key Takeaways** — a short recap block closing each module.
3. Content and all code are in **English**. This spec is a repository document, also in English.
4. The curriculum covers the **core language and the JDK standard library only**. No Spring, no JDBC, no build frameworks.
5. Lessons and their example code live **together inside each module folder**. Homework solutions live in a **separate top-level solutions/ tree** so the learner is not spoiled by accident.
6. No build tool. Examples run through single-file source launch: `java Example.java`.

→ Correct any of these now, or I proceed with them.

Objective

Build a self-paced Java curriculum that takes a programmer who knows another language and makes them fluent in modern Java, running on JDK 27.

User: A developer with general programming literacy and zero Java experience.

What success looks like: The learner works through numbered modules in order. Each module explains one area in a light, plain-spoken tone, backed by runnable example files whose comments teach the *why* rather than restating the syntax. Each module closes with Key Takeaways and a homework assignment; reference solutions exist but sit apart from the lesson.

A finding that shapes this curriculum

JDK 27 reached General Availability on **15 September 2026**. It is **not an LTS release** (JDK 25 is the current LTS). Its delivered JEPs are almost entirely runtime-level:

JEP	Title	Status
523	Make G1 the Default Garbage Collector in All Environments	Final
527	Post-Quantum Hybrid Key Exchange for TLS 1.3	Final
534	Compact Object Headers by Default	Final
536	JFR In-Process Data Redaction	Final
531	Lazy Constants	Third Preview
532	Primitive Types in Patterns, instanceof, and switch	Fifth Preview
533	Structured Concurrency	Seventh Preview
537	Vector API	Twelfth Incubator
538	PEM Encodings of Cryptographic Objects	Third Preview

Consequence: "Learning Java 27" does not mean learning nine new language features. Nothing in that list changes how a beginner writes code. The honest framing, and the one this curriculum adopts: *JDK 27 is the runtime and toolchain; the language you learn is the stable Java accumulated through JDK 25 LTS*. One closing module covers what 26 and 27 actually changed, why non-LTS releases matter less than release notes suggest, and how to read a JEP.

The on-ramp decision

JEP 512 (Compact Source Files and Instance Main Methods) was **finalized in JDK 25**, so it is stable on 27. Module 01 therefore opens with:

```
void main() {
    IO.println("Hello, World!");
}
```

No `public class`, no `static`, no `String[] args`, no import. Module 05 then deliberately "graduates" the learner to the classic form, because every real codebase, tutorial, and Stack Overflow answer they will ever meet uses it. Teaching the easy form and never explaining the classic one would leave them unable to read real Java.

Tech Stack

Item	Choice
JDK	OpenJDK 27 (GA 2026-09-15)
Language level	Stable features through JDK 25; preview features only in the final module, clearly labeled
Build tool	None
Dependencies	None
Lesson format	Markdown
Example format	Single-file <code>.java</code> , runnable directly

Commands

```
# Run a single example
java modules/01-getting-started/examples/HelloWorld.java

# Run an example that needs preview features (final module only)
java --enable-preview --source 27 modules/22-whats-new/examples/StructuredDemo.java

# Verify every example in the repo compiles and runs
./scripts/verify-examples.sh

# Verify a single module
./scripts/verify-examples.sh modules/09-sealed-and-pattern-matching

# Check the active JDK is 27
java -version
```

Project Structure

```
Java-27-edu/
├── README.md           → Table of contents, how to start, JDK setup
├── SPEC.md             → This document
├── scripts/
│   └── verify-examples.sh → Compiles and runs every example, reports failures
├── modules/
│   ├── 01-getting-started/
│   │   ├── README.md   → Lesson prose + Key Takeaways
│   │   ├── examples/
│   │   │   ├── HelloWorld.java
│   │   │   └── CompileVsRun.java
│   │   └── homework/
│   │       └── README.md → Assignment brief and acceptance criteria
│   ├── 02-variables-and-types/
│   └── ...
├── solutions/
│   ├── 01-getting-started/
│   │   └── Solution.java → Reference solution, kept out of the lesson tree
│   └── ...
```

Module ids are kebab-case, zero-padded, and fixed once assigned. They are never renamed mid-project.

Module README template

```
# Module NN: Title

## What You'll Learn
[3-5 bullets]

## Coming From Another Language
[What this looks like in Python/JS/C# vs. how Java does it]

## The Lesson
[Prose, light in tone, broken by runnable examples]

## Run It
[Exact commands for this module's examples]

## Common Mistakes
[Real traps, with the error message the learner will actually see]

## Key Takeaways
[4-6 bullets – the "takeout"]

## Homework
[Link to homework/README.md]
```

Curriculum Outline

#	Module id	Covers
01	getting-started	JDK 27 setup, JVM/JRE/JDK, compact source files, <code>IO.println</code> , compile vs. run
02	variables-and-types	Primitives vs. references, <code>var</code> , autoboxing traps, String immutability, text blocks
03	control-flow	Operators, <code>if / else</code> , switch expressions, loops, enhanced <code>for</code>
04	methods	Overloading, varargs, pass-by-value-of-references, static vs. instance
05	classes-and-objects	Classes, constructors, encapsulation, the classic <code>main</code> , flexible constructor bodies
06	inheritance-and-polymorphism	<code>extends</code> , <code>super</code> , <code>@Override</code> , <code>abstract</code> , <code>equals</code> / <code>hashCode</code> / <code>toString</code>
07	interfaces	Interfaces, default/static/private methods, functional interfaces
08	records-and-enums	Records, compact constructors, enums with behavior, choosing between them
09	sealed-and-pattern-matching	Sealed types, <code>instanceof</code> patterns, switch patterns, record patterns, exhaustiveness
10	exceptions	Checked vs. unchecked, try-with-resources, custom exceptions, what not to do
11	generics	Generic classes and methods, bounds, wildcards, type erasure, PECS
12	collections	List/Set/Map/Deque, sequenced collections, the equals/hashCode contract, immutable factories
13	lambdas	Lambda syntax, method references, the standard functional interfaces, capture rules
14	streams	Pipelines, intermediate vs. terminal ops, collectors, gatherers, parallel-stream caveats
15	optional	<code>Optional</code> done right, and the anti-patterns that make it worse than null
16	files-and-io	<code>Path</code> , <code>Files</code> , reading and writing text, try-with-resources applied
17	standard-library-tour	Strings, formatting, <code>java.time</code> , <code>Math</code> , random, comparators
18	concurrency-basics	Threads, races, <code>synchronized</code> , atomics, <code>ExecutorService</code>
19	virtual-threads	Virtual threads, scoped values, why blocking is cheap again
20	packages-and-modules	Packages, classpath, module system basics, module import declarations, <code>jar</code> , <code>javadoc</code>
21	testing-your-code	Assertions with <code>-ea</code> , a dependency-free micro test harness, what JUnit adds
22	whats-new	What JDK 26 and 27 actually changed, LTS vs. non-LTS, reading JEPs, preview features

Code Style

One concept per file. Comments teach the *why*; they never narrate the syntax.

```
// Module 02 – examples/AutoboxingTrap.java
//
// Java has two parallel worlds for numbers: primitives (int) and objects
// (Integer). Most of the time Java quietly converts between them, which is
// convenient right up until the moment it bites you. Here is the bite.

void main() {
    Integer a = 127;
    Integer b = 127;
    IO.println(a == b);    // true – and this is the misleading one

    Integer c = 128;
    Integer d = 128;
    IO.println(c == d);    // false! Same value, different objects.

    // Why: Java caches Integer objects for -128..127, so `a` and `b` are
    // literally the same object. Past 127 you get fresh objects, and `==`
    // compares identity, not value.
    //
    // The rule that saves you: use == for primitives, .equals() for objects.
    IO.println(c.equals(d)); // true – what you actually meant
}
```

Conventions: standard Java naming (`PascalCase` types, `camelCase` members, `UPPER_SNAKE` constants), 4-space indent, no line over 100 characters, every example runnable on its own with no arguments.

Testing Strategy

There is no application to unit-test here; the deliverable is teaching material. Correctness means **every example actually runs**.

- `scripts/verify-examples.sh` walks `modules/**/examples/*.java` and `solutions/**/*.*.java`, runs each one, and fails on any non-zero exit or compile error.
- Examples that demonstrate a compile error on purpose carry a `// EXPECT: compile-error` header and are asserted to fail, not skipped.
- Examples needing preview flags carry `// EXPECT: preview` and are run with `--enable-preview -source 27`.
- A module is not "done" until its examples, its homework brief, and its reference solution all pass the script.

- Module 21 teaches testing using `assert` and a hand-rolled harness, keeping the zero-dependency rule intact, and explains what a real JUnit setup would add.
-

Boundaries

Always

- Write all lesson prose, code, comments, and identifiers in English.
- Give every module a Key Takeaways section and a homework assignment.
- Run every example before marking a module complete.
- Show the actual error message a learner will hit when covering a common mistake.
- State plainly when a feature is preview, incubating, or JDK-version-gated.

Ask first

- Adding any third-party dependency, including JUnit.
- Introducing Maven or Gradle.
- Changing approved module ids, their order, or the curriculum outline.
- Expanding scope to frameworks, Android, or web development.

Never

- Ship an example that does not compile without marking it as intentional.
 - Invent JEP numbers, release dates, or feature claims. Verify against openjdk.org.
 - Use jargon before defining it.
 - Present a deprecated or discouraged practice without labeling it as such.
 - Put homework solutions inside the module folder.
-

Success Criteria

All eight checked on 2026-09-23 against the delivered repository. Each line records how it was verified, not merely that it was.

- ✓ 1. `./scripts/verify-examples.sh` exits zero across the whole repository. 130 files, all behaving as declared, in about 70 seconds.
- ✓ 2. All 22 module folders exist, each containing `README.md`, `examples/` with at least two runnable files, and `homework/README.md`. Checked by script across every module directory. 105 example files total, minimum 4 per module.

- ✓ **3. Every module README contains all seven template sections, including Key Takeaways.**
Checked by grepping each heading in all 22 files. One gap was found during the final pass, in `22-whats-new`, and fixed.
- ✓ **4. Every homework has a matching reference solution under `solutions/<module-id>/` that runs clean.** 22 solution directories for 22 modules, 25 solution files, all included in the verification run above.
- ✓ **5. Root `README.md` links every module in order and documents JDK 27 setup.** All 22 links confirmed present by script. Setup documented for macOS, Linux and Windows, with `JAVA_HOME` instructions for each.
- [~] **6. No file contains non-English prose, identifiers, or comments. Superseded by Amendment 1.**
1. True for the English curriculum and for all code, which is what the criterion was protecting. No longer true of the repository as a whole, because `tr/` now holds a Turkish translation of every lesson. Replaced by criteria 9 through 12.
- ✓ **7. No third-party dependency appears anywhere in the repository.** No `pom.xml`, `build.gradle` or `.jar` exists. Every import across all 130 files resolves to `java.*`, `javax.*` or `jdk.*`, plus the JEP 511 `import module java.base` declaration, which is a language feature rather than a dependency.

****Amendment 1 note.**** `scripts/build-pdfs.sh` requires pandoc and a Chromium-based browser. These are external *tools* invoked to render documentation, not libraries the curriculum's code links against. Nothing a learner compiles or runs depends on them, and the PDFs are optional output. The criterion stands as written for the Java code, and this note exists so the distinction is stated rather than assumed.
- ✓ **8. Every factual claim about a JDK version traces to openjdk.org release data.** 32 distinct openjdk.org links across the module and root READMEs. The JDK 26 and 27 JEP tables, the JDK 25 finalisation claims, and JEP 491's removal of virtual thread pinning were each fetched from the source during writing rather than recalled.

Open Questions, as resolved

1. **JDK 27 installation.** Resolved. JDK 27 (build 27+35-2325) was installed before implementation began, and every example has been verified against it.
2. **Module 21 and JUnit.** Resolved in favour of the zero-dependency rule. Testing is taught with bare `assert` plus a 39-line hand-rolled harness, and the module makes the case for adopting JUnit on any real project. The constraint turned out to improve the lesson: seeing a whole working harness explains what a framework does better than using one.
3. **Preview features.** Resolved as proposed. Structured concurrency (JEP 533) and primitive type patterns (JEP 532) appear only in Module 22, labelled with their JEP number and preview round, and

run behind `--enable-preview`. The module argues explicitly against shipping preview features, since a preview class file is rejected by a different release even with the flag.

Corrections Made During Implementation

Recorded because the spec's boundary forbade inventing claims, and holding to that surfaced several that were wrong:

- A composition example asserted that subclassing `ArrayList` would double-count additions. Running it disproved that: modern `ArrayList.addAll` copies in bulk. `HashSet` is the collection that exhibits the bug, and the example now shows both, which makes the fragile-base-class point better than the original would have.
- Three quoted compiler-error line numbers were wrong. All were corrected by producing the errors.
- A bucket-index demonstration used 8 buckets, and `Integer.MIN_VALUE` is divisible by 8, so both the broken and correct forms returned `0` and the bug was invisible. Changed to 7.
- A homework's stated output disagreed with what its reference solution printed. Homework outputs are now diffed against solution output rather than eyeballed.
- The structured concurrency preview history was written from recollection as seven rounds. Reading JEP 533's History section showed two earlier incubator rounds, making it nine releases.

Amendment 1 — PDF output and a Turkish translation

Date: 2026-09-23, after the original 24 tasks were delivered.

Two capabilities were added after the spec was signed off, and neither appeared in any spec or task list at the time it was built. The `spec-driven-development` skill names that exact situation as a red flag: *"Implementing features not mentioned in any spec or task list."* This amendment brings the document back in line with what was actually delivered.

What was added

Capability	What it is
PDF output	<code>scripts/build-pdfs.sh</code> renders every Markdown file to PDF, plus one combined book per language. <code>scripts/pdf.css</code> styles them for print.
Bilingual content	<code>tr/</code> holds a Turkish translation of all 48 Markdown files, plus <code>tr/CEVIRI-NOTLARI.md</code> recording the translation rules.

What the translation deliberately does not translate

Recorded here because it is a constraint, not a preference, and the original spec's Boundaries section had no rule covering a second language:

- **Code blocks stay verbatim English.** They quote the real files under `examples/`, so translating their comments would break the correspondence with what the learner actually runs.
- **Compiler output stays verbatim.** A translated error message is unsearchable and does not match what the terminal prints.
- **Commands, file paths, Java API names and JEP numbers stay as-is.**
- **No Java source lives under `tr/`.** Both languages share the same `modules/*/examples/` and `solutions/` trees.

A violation this amendment caught

Stating the rule above forced a check against it, and the check failed. **Thirty-five Java code blocks across eighteen Turkish files had their comments translated**, in direct violation of the convention the translation itself documents. They were restored to their English originals, and criterion 10 now guards against a recurrence.

This is the value of writing the constraint down rather than holding it in mind: the rule was clear, it was written, and it was still broken forty-five times before anything checked.

Amended success criteria

Criteria 1 through 5, 7 and 8 stand unchanged. Criterion 6 is superseded.

- ✓ **9. Every English Markdown file has a Turkish counterpart at the same path under `tr/`.** All 48 confirmed by script.
- ✓ **10. No `java` code block in any Turkish file contains non-English text.** Zero violations, after 35 were found and restored.
- ✓ **11. Java code blocks align one-to-one between the two languages.** Every one of the 44 paired files has an identical count, so a reader can follow either version against the same code.
- ✓ **12. No Java source file exists under `tr/`.** Zero. Both languages share one set of examples and solutions, so the verification script covers both.
- ✓ **13. `scripts/build-pdfs.sh` produces both languages.** 49 English PDFs in `pdf/`, 50 Turkish in `pdf-tr/`, each with a combined book, in about 100 seconds.
- ✓ **14. The translation does not affect example verification.** `./scripts/verify-examples.sh` still reports all 130 files behaving as declared.

Process note

The correct order would have been to amend this spec **before** building either capability, then re-derive the plan and tasks from it. That did not happen: both were built directly on request and the spec was left stale, with criterion 6 reading ☐ while it was false.

Writing the amendment is what surfaced the code-block violation, which had otherwise shipped. The lesson is the one this curriculum's Module 21 makes about tests: a rule nobody checks is a rule that is already broken.

Module 01: Getting Started

You already know how to program. This module gets Java running on your machine, shows you the smallest program that does something, and explains what actually happens when you run it. Half an hour, and you will never again be confused about the difference between the JDK, the JRE, and the JVM.

What You'll Learn

- How to get JDK 27 running and confirm you have the right one
- What the JVM, the JRE, and the JDK each are, and why people mix them up
- How to write and run a Java program without a class declaration or a build tool
- What the compiler does with your file, shown rather than asserted
- How to read a compiler error and find the exact spot it is complaining about

Coming From Another Language

In Python or JavaScript you write a file and run it. One step. Java traditionally made you do two: compile the source into bytecode, then run the bytecode. That extra step is where the "Java is heavyweight" reputation started.

Both of those things have changed, and this module is where you meet the change.

	Python / JavaScript	Java, traditionally	Java 27
Run a script	<code>python app.py</code>	<code>javac App.java</code> then <code>java App</code>	<code>java App.java</code>
Minimum program	<code>print("hi")</code>	<code>class</code> + <code>static main</code> + <code>String[] args</code>	<code>void main()</code> + <code>IO.println</code>
Errors found	at runtime	at compile time	at compile time

That last row is the one that will actually change how you work. Python tells you about a typo in a rarely-taken branch when a user hits it in production. Java tells you before the program starts. That is the trade you are making: more ceremony up front, fewer surprises later.

The Lesson

The three-letter words

People use JVM, JRE, and JDK interchangeably and then confuse each other. They are three different things stacked on top of each other:

- **JVM**, Java Virtual Machine. The thing that executes bytecode. It is why compiled Java runs unchanged on macOS, Linux, and Windows. Your code does not target a processor, it targets this.
- **JRE**, Java Runtime Environment. The JVM plus the standard library. Enough to *run* Java, not enough to compile it. You rarely install one separately now.
- **JDK**, Java Development Kit. The JRE plus the tools: `javac` the compiler, `javap` the disassembler, `jar`, `javadoc`. This is what you install.

You installed a JDK. Confirm it:

```
java -version
```

You want `java version "27"`. Anything else and the examples here may fail for reasons that have nothing to do with the lesson.

The smallest program

Open `examples/HelloWorld.java`. The entire file:

```
void main() {  
    IO.println("Hello, World!");  
}
```

If you have seen Java before, notice what is not there. No `public class`, no `static`, no `String[] args`, no import for the printing. That boilerplate was Java's most-mocked feature for twenty-five years, and JEP 512 removed the need to type it in JDK 25. It is stable, not preview.

`IO` is available without an import because it lives in `java.lang`, which every Java file imports automatically.

The boilerplate did not disappear

This is the part worth slowing down for, because it prevents a misconception that would bite you in Module 05.

Java did not grow a second, simpler language for beginners. The compiler writes the ceremony for you. Prove it:

```
javac -d /tmp/java27 modules/01-getting-started/examples/WhatTheCompilerWrites.java  
javap -cp /tmp/java27 WhatTheCompilerWrites
```

What comes back:

```
final class WhatTheCompilerWrites {  
    WhatTheCompilerWrites();  
    void main();  
}
```

There is your class. The compiler named it after the file, marked it `final`, gave it a no-argument constructor, and left `main` as an **instance** method, which is why the feature is called "instance main methods". The runtime builds one object of that class and calls `main` on it.

So you are writing ordinary Java the entire time, just typing less of it. Everything here transfers to the classic form in Module 05, because it *is* the classic form.

What running actually does

When you run `java HelloWorld.java`, three things happen in order:

1. `javac` compiles your source into **bytecode**, an instruction set the JVM understands. Since JDK 11 this can happen in memory, which is why you see no `.class` file appear.
2. The JVM loads that bytecode and verifies it.
3. The JVM executes it, compiling hot paths to native machine code as it goes.

Step 1 is where your typos are caught. This is the whole bargain of a compiled, statically typed language: the compiler reads every line before any of it runs.

You can still do it the long way, and sometimes you will want to:

```
javac Greet.java    # produces Greet.class
java Greet          # note: no .java, no .class, just the class name
```

Reading what the compiler tells you

Your first real skill in a new language is not writing code. It is reading the error and knowing where to look.

`examples/ReadTheError.java` is broken on purpose. Run it:

```
ReadTheError.java:29: error: ';' expected
    IO.println("I am missing something")
                        ^
1 error
```

Read it backwards from the caret:

- `^` points at the exact column
- `:29` is the line
- `';' expected` is what the compiler wanted to find there

Java is not guessing at your intent. It parsed your code up to that caret, and at that position the only legal token was a semicolon. Fix where the caret points, not the whole line.

Run It

```
# The minimal program
java modules/01-getting-started/examples/HelloWorld.java

# See what the compiler generated for you
java modules/01-getting-started/examples/WhatTheCompilerWrites.java

# Interactive, so run this one yourself
java modules/01-getting-started/examples/AskingForInput.java

# Broken on purpose. Read the error.
java modules/01-getting-started/examples/ReadTheError.java

# Check every example in this module still behaves
./scripts/verify-examples.sh modules/01-getting-started
```

Common Mistakes

Running `java Greet` when you only have `Greet.java`.

```
Error: Could not find or load main class Greet
Caused by: java.lang.ClassNotFoundException: Greet
```

`java Greet` looks for compiled bytecode named `Greet`. `java Greet.java` runs the source. The `.java` is the difference, and the error message does not spell that out for you.

Typing `io.println` instead of `IO.println`.

```
error: cannot find symbol
  io.println("hi");
  ^
symbol:   variable io
location: class E3
```

Java is case sensitive, and `IO` is a class name. "Cannot find symbol" is the error you will see most often in your first month. It nearly always means a typo, a missing import, or a name that does not exist yet.

Expecting `+` to add when the left side is text.

```
"Next year is " + year + 1    // "Next year is 20261"
"Next year is " + (year + 1)  // "Next year is 2027"
```

`+` evaluates left to right. Once one side is a `String`, every `+` after it means "append", not "add". Parenthesise the arithmetic.

Assuming `System.out.println` is obsolete. It is not. `IO.println` is shorthand that exists in compact source files. Every real codebase you will read uses `System.out.println` or a logging

framework, and both still work here.

Key Takeaways

- **The JDK is what you install.** It contains the JVM that runs bytecode and the tools that produce it. `java -version` must say 27 for this curriculum.
- **`java File.java` runs a source file directly.** No build tool, no separate compile step, no `.class` file left behind.
- **Compact source files are real Java.** The compiler writes the class declaration, the constructor, and the `final` modifier for you. `javap` shows you exactly what it wrote.
- **`main` here is an instance method**, so `this` exists. The classic `public static void main` does not have that, and Module 05 explains why.
- **Compile errors are located, not vague.** The caret points at the column, the number gives the line, and the message states what the compiler expected.
- **Nothing was replaced.** `System.out.println`, classes, and `static main` all still work. You are learning less typing, not a different language.

Homework

[homework/README.md](#)

Build a small interactive program from what you have just seen, then break it on purpose and read what the compiler says. Reference solution lives in [solutions/01-getting-started/](#), but try it first. Reading the answer teaches you much less than getting the error and fixing it.

Homework 01: About Me

Build a small interactive program using only what Module 01 covered, then break it on purpose and read what the compiler tells you.

The Task

Create a file called `AboutMe.java` anywhere you like outside this repository, for example in a scratch folder. Write it as a **compact source file**, meaning no `class` declaration and no `static`.

It should:

1. Ask for the user's name.
2. Ask for the year they were born.
3. Ask for the current year.
4. Print a small profile that includes their name and their approximate age, where age is the current year minus the birth year.

Sample run, with typed input shown after the prompts:

```
What is your name? Ada
What year were you born? 1990
What year is it now? 2026

--- About Ada ---
Name: Ada
Age this year: 36
```

Then Break It

Once it works:

1. Delete a semicolon somewhere in the middle of the file.
2. Run it again.
3. Add a comment at the bottom of the file recording the exact error message, including the line number and the caret line.
4. Put the semicolon back.

This step is the actual point of the assignment. You will read thousands of compiler errors in your career and the first one should not happen under pressure.

Acceptance Criteria

- ☐ The file runs with `java AboutMe.java`, with no separate `javac` step.
- ☐ It has no `class` declaration and no `static` keyword.
- ☐ It uses `IO.readLine` to ask questions and `IO.println` to print.
- ☐ The age is computed with arithmetic, not typed in by the user.
- ☐ The age prints as a number, not concatenated text. `"Age: " + year - birth` will not compile, and working out why is part of the exercise.
- ☐ A comment at the bottom records a real compiler error you produced and read.

Stretch

Run `javap` against your own file and find the class name the compiler invented:

```
javac -d /tmp/hw01 AboutMe.java
javap -cp /tmp/hw01 AboutMe
```

Does the generated class match what Module 01 said it would? What is the method signature of `main`, and is it static?

Hint, if you are stuck on the age

`IO.readLine` always hands you a `String`, even when the user types digits. Java will not quietly treat `"1990"` as a number. Look again at how `AskingForInput.java` turns text into an `int`.

Module 02: Variables and Types

You know what a variable is. What you need from Java is narrower: which types exist, which of them are objects and which are not, and where the boundary between those two worlds will trip you up. That boundary is the whole module.

What You'll Learn

- The split between primitives and references, and why Java still has it
- Five numeric behaviours that surprise people, none of which are bugs
- Autoboxing, and the two ways it bites
- Why `==` lies to you about objects, and what to use instead
- Strings as immutable objects, plus text blocks
- `var`, what it does, and where it stops helping

Coming From Another Language

In Python and JavaScript everything is an object, including numbers. Java keeps two separate worlds, and you will feel the seam.

	Python / JS	Java
<code>3</code>	an object	a primitive, 32 bits, no allocation
Declaring	<code>x = 3</code>	<code>int x = 3;</code> or <code>var x = 3;</code>
Changing type later	fine	impossible, the type is fixed at compile time
Integer overflow	grows the number	wraps around silently
<code>"a" == "a"</code>	compares content	compares <i>identity</i> , use <code>.equals</code>
Truthiness	<code>if (items)</code> works	only <code>boolean</code> is allowed in <code>if</code>

That last one deserves a note. Java has no truthiness. `if (someString)` does not compile. You write `if (!someString.isEmpty())`, and the verbosity is the point: there is no rule to memorise about which values count as false.

The Lesson

Two kinds of value

Eight primitive types exist, and the list is closed forever:

boolean	int	long	double
char	byte	short	float

A primitive variable holds the value. Everything else in Java is a **reference**: the variable holds the address of an object living somewhere else.

The difference is invisible until you copy something. See [TwoKindsOfValue.java](#):

```
int[] original = {1, 2, 3};
int[] alias = original; // copies the reference, not the array
alias[0] = 999;
// original[0] is now 999
```

Copying a primitive copies the value. Copying a reference copies the label, and both labels point at the same object. Module 04 returns to this when we look at what happens to method arguments.

In practice you will use `int`, `long`, `double`, `boolean` and `char`. The other three exist mostly for memory-constrained situations you are unlikely to meet.

Numbers will surprise you

Run [NumbersWillSurpriseYou.java](#). Real output:

```
7 / 2 = 3
7 / 2.0 = 3.5
Integer.MAX_VALUE + 1 = -2147483648
Math.addExact says: integer overflow
0.1 + 0.2 = 0.30000000000000004
7 % -2 = 1
-7 % 2 = -1
Math.floorMod(-7, 2) = 1
'A' + 1 = 66
```

Five things are happening there:

1. **Integer division discards the remainder.** `7 / 2` is `3`. Make one operand a `double` and you get `3.5`. When both operands are variables you cannot see this coming, which is what makes it dangerous.
2. **Overflow wraps silently.** An `int` is 32 bits, and adding one to the maximum rolls to the minimum with no exception. `Math.addExact` complains instead, when you would rather know.
3. **`double` cannot hold most decimals exactly.** `0.1` is a repeating fraction in binary. Never store money in a `double`. Use a `long` of cents, or `BigDecimal`.
4. **`%` keeps the sign of the left operand.** `-7 % 2` is `-1`, not `1`. If you wanted the mathematical modulo, that is `Math.floorMod`.
5. **`char` is a number.** `'A' + 1` is `66`, because `+` promoted the char to an int. Cast back with `(char)` to get `B`.

Autoboxing, and its two bites

Every primitive has an object twin: `int` / `Integer`, `double` / `Double`, and so on. Java converts between them automatically. That is autoboxing, and it is genuinely convenient until these two moments.

Bite one, `==` on boxed values. From `TheAutoboxingTrap.java`:

```
127 == 127 : true
128 == 128 : false
```

Same code, same kind of value, opposite answer. Java caches `Integer` objects for `-128..127`, so inside that range `==` compares an object to itself. Outside it, you get two objects and `==` correctly reports that they are different objects.

Nothing is broken. `==` on objects has always meant "the same object", and the cache was hiding that from you.

The rule: `==` for primitives, `.equals()` for objects.

Bite two, unboxing null. `NullUnboxing.java` crashes on purpose:

```
Exception in thread "main" java.lang.NullPointerException:
  Cannot invoke "java.lang.Integer.intValue()" because the return value of
  "java.util.Map.get(Object)" is null
```

`Map.get` returns `null` for a missing key, and assigning that to an `int` forces Java to call `intValue()` on nothing. Notice how specific that message is: it names the call that returned null *and* the call that failed on it. Modern Java NPEs do a lot of the debugging for you.

Strings

Strings are objects, and they are **immutable**. No method ever changes the string you called it on. From `StringsAndTextBlocks.java`:

```
String name = "ada";
name.toUpperCase();           // computes "ADA" and throws it away
name = name.toUpperCase();    // you have to keep the result
```

The first line is a real mistake people make, and nothing warns you.

The `==` trap applies here too, for the same reason as `Integer`:

```
literal == literal : true      // the compiler pools identical literals
literal == new      : false    // new String() forces a separate object
equals           : true
```

One habit worth forming immediately: put the literal on the left.

```
"java".equals(maybeNull)    // false
maybeNull.equals("java")  // NullPointerException
```

Text blocks give you multi-line strings without escaping:

```
String json = """
    {
        "name": "Ada"
    }""";
```

Java strips the common leading indentation, measured from the least-indented line, so the block lines up with your code without that whitespace ending up in the value.

var

`var` asks the compiler to work out the type from the right-hand side. Read this part carefully, because it is the thing people get wrong:

`var` is not dynamic typing. The variable still has exactly one type, fixed forever, checked at compile time. You just did not type it out.

```
var x = "hello";
x = 42;    // error: incompatible types: int cannot be converted to String
```

It earns its keep when the type is long and already visible:

```
var scores = new HashMap<String, List<Integer>>();
```

It hurts when the right-hand side does not tell you what you got:

```
var result = service.process(input);    // now go look up process()
```

Three things it cannot do, all compile errors:

```
var a;           // nothing to infer from
var b = null;    // null belongs to every reference type
var c = () -> 1; // a lambda needs a target type
```

`VarCannotInferNull.java` demonstrates the second one. It is also local variables only. Fields, parameters and return types must still be spelled out, which keeps APIs readable.

Run It

```
java modules/02-variables-and-types/examples/TwoKindsOfValue.java
java modules/02-variables-and-types/examples/NumbersWillSurpriseYou.java
java modules/02-variables-and-types/examples/TheAutoboxingTrap.java
java modules/02-variables-and-types/examples/StringsAndTextBlocks.java
java modules/02-variables-and-types/examples/VarAndInference.java

# These two fail on purpose. Read what they say.
java modules/02-variables-and-types/examples/NullUnboxing.java
java modules/02-variables-and-types/examples/VarCannotInferNull.java

./scripts/verify-examples.sh modules/02-variables-and-types
```

Common Mistakes

Using `==` to compare Strings. It works in your test with short literals and fails in production with strings built at runtime. This is the single most common Java bug written by newcomers. Use `.equals`.

Storing money in a `double`. `0.1 + 0.2` is not `0.3`. Use a `long` of cents or `BigDecimal`. Accounting systems have been written the wrong way and the discrepancy shows up months later.

Forgetting that String methods return a new String. `name.strip();` on its own line does nothing at all.

Letting `Map.get` unbox into an `int`. Any missing key becomes a `NullPointerException`. Use `getOrDefault`, or keep it as an `Integer` and check.

Boxing inside a hot loop.

```
Long sum = 0L; // a million allocations
for (long i = 0; i < 1_000_000; i++) sum += i;
```

Changing `Long` to `long` removes every allocation. The two versions look almost identical, which is exactly why this is worth knowing.

Writing `long big = 9000000000;` without the `L`. The literal is parsed as an `int` first, and it does not fit, so it fails to compile.

Key Takeaways

- **Two worlds.** Primitives hold values, references hold addresses. Copying a reference copies the label, not the object.
- `==` asks "same object", `.equals` asks "same content". On anything that is not a primitive, you want `.equals`.

- **The Integer cache makes `==` look correct below 128.** That is what makes it dangerous, not the boundary itself.
- **Numbers are fixed-width.** Integer division truncates, overflow wraps, and `double` cannot hold most decimals exactly.
- **Strings never change.** Every method returns a new one; keep the result.
- **`var` is inference, not dynamic typing.** The type is still fixed and still checked. Use it when the type is obvious on the same line.
- **Java has no truthiness.** Only a `boolean` goes in an `if`, and that is a feature.

Homework

[homework/README.md](#)

Build a small receipt calculator that gets the money handling right, then reproduce two of this module's traps on purpose. Reference solution is in [solutions/02-variables-and-types/](#).

Homework 02: The Receipt That Adds Up

A shop's till has been rounding wrong and nobody can work out why. You are going to write one that does not, which means getting the money type right.

The Task

Create `Receipt.java` as a compact source file. It should:

1. Hold three items with a name and a price: `19.99`, `4.05` and `0.99`.
2. Print each item on its own line.
3. Print the subtotal, a 20% tax amount, and the total.
4. Get the arithmetic exactly right, and get the *formatting* exactly right too.

Target output:

```
Bread      19.99
Milk       4.05
Gum        0.99
-----
Subtotal   25.03
Tax (20%)  5.01
Total      30.04
```

The Money Rule

Do **not** use `double` for the prices. Module 02 showed why: `0.1 + 0.2` is not `0.3`, and a till that is a hundredth of a penny out on every line eventually fails an audit.

Store each price as a `long` counting **cents**, so `19.99` becomes `1999`. Do all the arithmetic in whole cents and convert to text only at the moment you print.

To format cents as currency, integer division and remainder give you both halves:

```
long cents = 405;
String shown = (cents / 100) + "." + (cents % 100);
```

Run that on the milk and see what you get. It is wrong, and working out *why* it is wrong is the most valuable part of this exercise. Four of the six lines on this receipt hit the same bug, so it is not an edge case you can wave away.

The Rounding Decision

20% of 2503 cents is 500.6 exactly. You cannot charge six tenths of a cent, so you have to decide what happens to it, and your code should make that decision visible rather than letting it happen by accident.

Whatever you choose, leave a comment saying what you chose and why.

Then Reproduce Two Traps

Add a second file, `Traps.java`, that demonstrates and explains:

1. **The `==` trap on boxed integers.** Print a comparison that is `true` and one that is `false` using the same kind of value, then print the `.equals` result showing what you actually meant.
2. **String immutability.** Call a String method without keeping the result, print the unchanged value, then do it properly.

Each needs a comment explaining *why* in your own words. Copying the wording from the examples does not count. Explaining it back is the point.

Acceptance Criteria

- ☐ Both files run with `java Receipt.java` and `java Traps.java`
- ☐ No `double` or `float` appears anywhere in `Receipt.java`
- ☐ Tax is computed, not typed in as a literal
- ☐ `4.05` prints as `4.05`, not `4.5`
- ☐ The subtotal, tax and total all print with exactly two decimal places
- ☐ The columns line up
- ☐ A comment states what happens to the fraction of a cent, and why
- ☐ `Traps.java` shows a boxed `==` printing `true` and another printing `false`
- ☐ Every trap carries an explanation in your own words

Stretch

The till now needs a 7.5% rate. Tax on 2503 cents at 7.5% is 187.725 cents.

Implement it, decide what happens to the fraction, and defend the choice in a comment. There is no single right answer, which is exactly why it is worth thinking about. Real payment systems have published rules for this, some of them requiring rounding per line item rather than on the total, which gives a different total again.

Hint, if the formatting is fighting you

`String.format` understands padding and zero-filling:

```
String.format("%-12s %8s", name, amount)    // left-align 12, right-align 8
String.format("%d.%02d", whole, remainder) // %02d forces two digits
```

The second one is the fix for the bug you found earlier.

Module 03: Control Flow

Loops and conditions in Java hold few surprises for anyone who has written code before. Three things here are worth your attention: Java has no truthiness, the modern `switch` is genuinely better than the one you may have met, and labelled break is a piece of syntax most languages do not have.

What You'll Learn

- Why `if (name)` does not compile, and why that is a feature
- Short-circuit evaluation as a correctness tool, not an optimisation
- Switch expressions, multi-label cases, and `yield`
- Exhaustiveness checking, and why you often want no `default` branch
- Every loop form, and which one to reach for
- Labelled break, for the nested-loop case

Coming From Another Language

	Python / JS	Java
Condition	any value	<code>boolean</code> only
Empty string is falsy	yes	there is no falsy
<code>switch</code> on strings	JS yes, Python 3.10+ match	yes
Fall-through	JS yes	only in the old statement form
Break out of nested loops	flag variable	<code>break label;</code>
<code>for x in xs</code>	native	<code>for (var x : xs)</code>

The single biggest adjustment is the first row. Java will not let you write `if (items)` and decide for you what an empty list means.

The Lesson

No truthiness

```
String name = "";
if (name) { }    // does not compile
if (0) { }       // does not compile
```

You write the comparison out:

```
if (!name.isEmpty()) { ... }
```

More typing, and in exchange there is no table of falsy values to memorise and no debate about whether `"0"` or `[]` counts as false. Every conditional says exactly what it tests.

Short-circuit as a guard

`&&` skips its right operand when the left one already decided the answer. That is not only a performance detail, it is how you guard against null:

```
if (maybe != null && maybe.length() > 3) { ... }
```

Reverse the two and it throws. The order of `&&` operands is a correctness decision.

`&` and `|` also work on booleans and do **not** short-circuit. You will almost never want them there. Their real job is bitwise work:

```
6 & 3 = 2      6 | 3 = 7      6 ^ 3 = 5      6 << 1 = 12
```

Switch, the good version

A switch **expression** produces a value. See [SwitchExpressions.java](#):

```
String kind = switch (day) {  
    case 1, 2, 3, 4, 5 -> "weekday";  
    case 6, 7 -> "weekend";  
    default -> "not a day";  
};
```

No `break`, several labels per branch, and the whole thing is an expression you can assign. When a branch needs more than one statement, use a block and `yield`:

```
int workload = switch (day) {  
    case 6, 7 -> {  
        int base = 10;  
        yield base * 2;  
    }  
    default -> 8;  
};
```

`yield` exists because `return` would try to leave the entire method.

Exhaustiveness, the part that actually matters

This method has no `default`, and the compiler accepts it:

```
String describe(Light light) {
    return switch (light) {
        case RED -> "stop";
        case AMBER -> "get ready";
        case GREEN -> "go";
    };
}
```

Three constants, three branches, complete. Now delete the `GREEN` branch, as `SwitchMustBeExhaustive.java` does:

```
error: the switch expression does not cover all possible input values
    return switch (light) {
        ^
missing patterns:
    Light.GREEN
```

The compiler names the case you forgot.

This leads to advice that sounds backwards at first:

Leave out the `default` branch when the input is a closed set.

With no `default`, adding a fourth constant to `Light` breaks compilation at every switch that needs updating, and the error points at each one. With a `default`, all of them keep compiling and quietly route the new constant to the fallback. A compile error you get today beats a bug you find in production.

Fall-through, and why it went away

The old statement form runs into the following cases until something stops it.

`WhyFallThroughWentAway.java` shows both, with `n = 2`:

```
n = 2, old statement form:
    two
    three
n = 2, arrow form:
    two
```

The old form matched case 2, printed, and then fell into case 3 because there was no `break`. One missing keyword, and the wrong behaviour reads as correct.

Fall-through was occasionally used to group cases. The arrow form does that with a comma and no risk:

```
case 1, 2, 3 -> handleSmall();
```

Loops

Four forms, and the choice is usually obvious. From `Loops.java`:

Form	Use it when
<code>for (var x : xs)</code>	you do not need the index. Most of the time.
<code>for (int i = 0; ...)</code>	you need the index, a step other than 1, or to go backwards
<code>while</code>	the iteration count is not known up front
<code>do-while</code>	the body must run at least once. Rare.

Labelled break is the one piece of loop syntax with no equivalent in most languages. It exits an outer loop from inside an inner one:

```
search:
for (int row = 0; row < grid.length; row++) {
    for (int col = 0; col < grid[row].length; col++) {
        if (grid[row][col] == target) {
            break search;    // leaves both loops
        }
    }
}
```

Without it you need a flag variable and an extra condition on the outer loop, which is the kind of code that rots. Use it sparingly: more than one label in a method usually means the method wants splitting up.

Run It

```
java modules/03-control-flow/examples/OperatorsAndConditions.java
java modules/03-control-flow/examples/SwitchExpressions.java
java modules/03-control-flow/examples/WhyFallThroughWentAway.java
java modules/03-control-flow/examples/Loops.java

# Fails on purpose. Read which case it names.
java modules/03-control-flow/examples/SwitchMustBeExhaustive.java

./scripts/verify-examples.sh modules/03-control-flow
```

Common Mistakes

Getting the `&&` operands the wrong way round. `maybe.length() > 3 && maybe != null` throws before the null check ever runs.

Adding a `default` to a switch over an enum out of habit. It compiles today and hides every case you forget to add tomorrow.

Forgetting `break` in the old statement form. If you are writing new code, use the arrow form and the problem cannot occur.

Assuming compound assignment is just shorthand.

```
byte b = 10;
b += 300;      // compiles, prints 54
b = b + 300;   // does not compile
```

`+=` quietly inserts a cast. `10 + 300` is `310`, and truncating that to a byte gives `54`. The explicit form refuses instead, which is the safer of the two.

Using `==` on Strings built at runtime.

```
a1 == a2 : true      // both compile-time literals, pooled
a1 == a3 : false     // a3 was built by a StringBuilder
```

Same text, different answers, depending on when the string came into existence.

Writing a loop condition that relies on overflow.

```
for (int i = 1; i > 0; i *= 2) { }
```

This does terminate, but only because `i` wraps negative past `Integer.MAX_VALUE`, which is not the reason you wrote.

Key Takeaways

- Only a `boolean` goes in an `if`. There is no truthiness and no falsy table.
- `&&` and `||` short-circuit, which makes operand order a correctness concern.
- Switch expressions produce values, take several labels per branch, and never fall through.
- `yield` returns a value from a switch block; `return` would leave the method.
- Omit `default` on a closed set so that adding a case breaks the build instead of silently falling through.
- Use the enhanced `for` unless you need the index.
- `break label;` exits nested loops without a flag variable.

Homework

[homework/README.md](#)

Write a small grading tool with a switch expression, then convert a fall-through bug into a compile error. Reference solution in [solutions/03-control-flow/](#).

Homework 03: Grades and Grids

Two small programs. The first asks you to pick the right conditional for the job. The second shows you why labelled break exists by making you live without it first.

Part One: Grades

Create `Grades.java`. Given this list of scores:

```
int[] scores = {95, 83, 71, 64, 42, 100, 0};
```

Print each score with its letter grade and a short comment, then the class average with its grade.

Grade boundaries: 90+ is A, 80+ is B, 70+ is C, 60+ is D, anything less is F.

Target output:

```
95 A    excellent
83 B    solid
71 C    passing
64 D    scraped through
42 F    see me
100 A   excellent
0  F    see me
```

```
Class average: 65 (D)
```

The Decision You Have To Make

One of these two jobs suits a `switch` expression and the other does not.

- Turning a **score** into a letter
- Turning a **letter** into a comment

Work out which is which, use `switch` only where it fits, and leave a comment explaining why the other one is an if-chain. Forcing a switch where it does not belong is a worse answer than not using one at all.

Part Two: Find In Grid

Create `FindInGrid.java`. Search this grid for a target value and report its row and column, or say it is absent:

```
int[][] grid = {
    { 7, 12, 3},
    { 9, 5, 21},
    {14, 2, 18}
};
```

Write the search **twice**:

1. `withFlag` — using a `boolean found` variable and a plain `break`
2. `withLabel` — using a labelled break

Run both against `21`, which is present, and `99`, which is not.

Then write a comment comparing them. Specifically: list what the flag version needs that has nothing to do with searching a grid.

Acceptance Criteria

- ☐ Both files run with `java Grades.java` and `java FindInGrid.java`
- ☐ `Grades.java` uses a `switch` expression for exactly one of the two jobs
- ☐ A comment explains why the other job is not a switch
- ☐ Columns line up; `100` and `0` both render correctly
- ☐ `FindInGrid.java` contains both `withFlag` and `withLabel`
- ☐ Both versions produce identical output for `21` and for `99`
- ☐ A comment lists the bookkeeping the flag version needs

Stretch

There is a third way to escape both loops that needs neither a flag nor a label. Find it, implement it as `withExtractedMethod`, and say when you would prefer it over the labelled break.

Hint, if the switch will not compile

A switch expression has to produce a value for every possible input. Over an `int` or a `String` that means you cannot cover everything by listing cases, so the compiler insists on a `default`. Over an `enum` with every constant listed, it does not. That difference is the whole reason one of these two jobs fits a switch better than the other.

Module 04: Methods

Declaring a method holds no surprises. Three things in this module do: how Java passes arguments, how it picks between overloads, and what `static` actually means. The first of those is the most misunderstood thing in the language.

What You'll Learn

- Why Java is always pass-by-value, including for objects, and what follows from that
- Overload resolution order, and the real bug it causes in `List`
- Why the return type is not part of a method's signature
- Varargs, and the one case where `null` needs a cast
- When `static` is right and when it quietly blocks you later

Coming From Another Language

	Python / JS	Java
Same name, different parameters	not possible, use defaults	overloading
Default parameter values	<code>def f(x=1)</code>	no such thing, overload instead
Named arguments	<code>f(x=1)</code>	no such thing
Variable arguments	<code>*args</code>	<code>int... nums</code>
Argument passing	reference to object, name rebinding is local	identical semantics, different vocabulary

That last row matters. Python and Java behave **the same way** when you pass an object. Both let a function mutate it, and neither lets a function rebind the caller's variable. The argument only exists because the two communities use the phrase "pass by reference" to mean different things.

The Lesson

Pass-by-value, settled

The method gets a **copy of the variable**. When the variable holds a reference, the method gets a copy of the reference, so both point at the same object.

`PassByValue.java` shows the two cases that look identical and are not. Real output:

```
primitive after the call: 10
list after mutation:      [original, added inside the method]
list after reassignment:  [original]
string after the call:    before
```

Same type, same call shape, opposite results:

```
void mutateTheObject(List<String> items) {
    items.add("added inside the method"); // changes the shared object
}

void reassignTheParameter(List<String> items) {
    items = new ArrayList<>();           // rebinds the local copy only
}
```

In the first, the method followed its copy of the reference to the one shared object and changed it. In the second, it pointed its own copy somewhere else. Your variable never moved, because a method has no way to move it.

Three consequences you will actually use:

- To let a method change your data, pass a mutable object.
- To stop it, pass something immutable, or `List.copyOf`, or a defensive copy.
- A method can never repoint your variable. If it needs to give you a different object, it must **return** it.

Strings confuse this discussion because they are immutable, so there is no mutation case for them at all. Every String method returns a new object.

Overload resolution

When no overload matches exactly, the compiler tries in this fixed order:

```
1. widening    int -> long -> float -> double
2. boxing      int -> Integer
3. varargs     int -> int...
```

It takes the first that works, and it decides at **compile** time. From [Overloading.java](#):

```
pick(1)    -> long        widening beat boxing
choose(1)  -> Integer     boxing beat varargs
```

The order exists for backward compatibility. Widening and varargs predate autoboxing, so preferring boxing would have changed the meaning of code written before Java 5.

The bug this causes

`List` has both `remove(int index)` and `remove(Object o)`:

```
after remove(1):           [10, 30]
after remove(Integer.valueOf(30)): [10]
```

The first removed the element **at index 1**, which was `20`. Not the value `1`.

This has caused real production bugs. Whenever a `List` holds `Integer`, say explicitly which overload you mean.

Return type is not a signature

```
int    parse(String s) { ... }
double parse(String s) { ... }
```

```
error: method parse(String) is already defined in class ReturnIsNotASignature
```

The compiler chooses an overload from the arguments at the call site. In `parse("1");` as a bare statement there is nothing to choose from. This is exactly why the standard library has `parseInt` and `parseDouble` rather than two methods called `parse`.

Varargs

Inside the method, a varargs parameter is just an array. [Varargs.java](#):

```
int count(int... numbers) { return numbers.length; }
```

Three rules:

1. It must be the last parameter, and there can be only one.
2. Calling with no arguments gives an **empty array**, never `null`. No null check needed for ordinary calls.
3. An explicit `null` is ambiguous and needs a cast:

```
describe((Object[]) null) -> the array itself was null
describe((Object) null)   -> an array of length 1
```

`String.format` and `printf` are varargs methods, which is why they take any number of values.

static

`static` belongs to the class. Everything else belongs to an object.

You have been writing instance methods all along without noticing: a compact source file's `main` is one, which is why `this` works inside it.

Use `static` when the method needs nothing from any particular object, is a pure function of its arguments like `Math.max`, or is a factory like `List.of`.

Avoid it when the method touches per-object state, or when you might want to override it later. **Static methods are not polymorphic**, and Module 06 covers what that costs.

A useful test: if making a method static means passing in several things an object would already know, it should not be static.

Static *state* is one shared copy for the whole program. Convenient, and in a multi-threaded program a race condition waiting to happen. Module 18 returns to this.

Run It

```
java modules/04-methods/examples/PassByValue.java
java modules/04-methods/examples/Overloading.java
java modules/04-methods/examples/Varargs.java
java modules/04-methods/examples/StaticVsInstance.java

# Fails on purpose.
java modules/04-methods/examples/ReturnTypeIsNotASignature.java

./scripts/verify-examples.sh modules/04-methods
```

Common Mistakes

Expecting a method to repoint your variable. It cannot. Have it return the new value.

Calling `list.remove(someInt)` on a `List<Integer>`. You removed a position, not a value. Wrap it:
`list.remove(Integer.valueOf(x))`.

Handing out your internal collection from a getter. The caller now holds a reference to your object's state and can change it behind your back. Return `List.copyOf(items)` when that matters.

Trying to overload on return type alone. Give the methods different names.

Expecting default parameter values. Java has none. Write an overload that calls the fuller version:

```
String greet(String name) { return greet(name, "Hello"); }
String greet(String name, String word) { return word + ", " + name; }
```

Reaching for `static` because "it does not need an object right now". It is easy to make a method static and awkward to undo once callers depend on it, particularly when you later want to override or mock it.

Key Takeaways

- **Java is always pass-by-value.** The copy is of the variable, and for objects the variable is a reference.
- **Mutating a parameter's object is visible to the caller. Reassigning the parameter is not.** That single distinction resolves the entire argument.
- **Overloads resolve widening, then boxing, then varargs,** at compile time.
- `List.remove(int)` takes an index and `remove(Object)` takes a value. Be explicit when the list holds `Integer`.
- **Return type is not part of the signature.** Different names, not different returns.
- **Varargs is an array,** must come last, and is empty rather than null.
- `static` means no object, so no `this` and no overriding.

Homework

[homework/README.md](#)

Write a method that cannot change what it is given, prove it, then walk into the `List.remove` trap on purpose. Reference solution in [solutions/04-methods/](#).

Homework 04: What A Method Can And Cannot Change

Two programs. The first makes you prove the pass-by-value rule to yourself instead of taking it on trust. The second walks you into a real production bug on purpose.

Part One: Inventory

Create `Inventory.java`. Start with a mutable list:

```
var stock = new ArrayList<String>(List.of("bolt", "nut"));
```

Write three methods and call all of them, printing the list after each:

1. `addItem(List<String> items, String item)` — adds an item. This one **must** be visible to the caller.
2. `replaceAll(List<String> items, List<String> replacements)` — written so that it assigns a new list to the parameter. This one **must not** be visible to the caller, and your printed output has to show that it is not.
3. `replaceAllProperly(...)` — the fixed version.

Then write a comment answering: there are exactly two ways to fix `replaceAll`. Name both, and say when you would pick each.

Part Two: The Remove Trap

Create `RemoveTrap.java`. Build a `List<Integer>` and trigger both overloads:

```
var ids = new ArrayList<Integer>(List.of(100, 200, 2, 300));  
int idToDrop = 2;  
ids.remove(idToDrop);
```

Print the result. It will not be what the code appears to say.

Then write a comment, **in your own words**, explaining why the compiler chose the overload it did. Your explanation has to mention where boxing sits in the resolution order, because that is the actual reason.

Finish by showing at least two ways to write it so it does what it looks like.

Acceptance Criteria

- ☐ Both files run with `java Inventory.java` and `java RemoveTrap.java`

- ☐ The printed output of `Inventory.java` proves `addItem` was visible to the caller and `replaceAll` was not
- ☐ A comment names both fixes for `replaceAll` and says when each applies
- ☐ `RemoveTrap.java` shows `remove(int)` and `remove(Object)` giving different results from the same starting list
- ☐ The explanation mentions overload resolution order, in your own words
- ☐ At least two correct alternatives are shown

Stretch

Add a small class holding a `List<String>` field with a getter that returns the field directly. From `main`, get the list, modify it, and print the object's contents to show you changed its internals from the outside without permission.

Then fix the getter, and say what the fix costs. There is more than one fix and they do not cost the same thing.

Hint, if part one looks like it already works

If `replaceAll` appears to change the caller's list, check whether you assigned to the parameter or called a method on it. Only one of those is visible from outside, and the whole exercise turns on which one you wrote.

Module 05: Classes and Objects

This is graduation day. Every file so far has been a compact source file. From here on the examples are written the classic way, because that is what every real codebase, tutorial and Stack Overflow answer uses.

Nothing new is happening to the language. Module 01 used `javap` to show the compiler generating a class for you all along. Now you write it yourself.

What You'll Learn

- Every word in `public static void main(String[] args)`, and why each is there
- Fields, the four visibility levels, and why `private` is where you start
- Constructors, `this(...)`, and the free constructor you lose by writing one
- Why static factory methods often beat constructors
- Flexible constructor bodies, which let you validate before `super()`

Coming From Another Language

	Python	Java
Constructor	<code>__init__</code>	a method named after the class
Current instance	explicit <code>self</code> parameter	implicit <code>this</code>
Privacy	<code>_name</code> by convention	<code>private</code> , enforced by the compiler
Multiple constructors	one <code>__init__</code> , default args	overload, or static factories
Class-level data	class attribute	<code>static</code> field
File layout	many classes per file, freely	one <code>public</code> class per file, named to match

The privacy row is the real difference. Python's underscore is a request. Java's `private` is a rule the compiler enforces, and code that violates it does not build.

The Lesson

The classic form, word by word

```
public class TheClassicForm {  
    public static void main(String[] args) {  
        System.out.println("Running the classic way.");  
    }  
}
```

Word	Why
<code>public</code> (class)	usable from anywhere. Must match the file name.
<code>public</code> (main)	the JVM has to be able to call it
<code>static</code>	the important one. At startup no object exists yet, so <code>main</code> cannot belong to one.
<code>void</code>	returns nothing; exit codes come from <code>System.exit</code>
<code>main</code>	the exact name the JVM looks for
<code>String[] args</code>	command-line arguments. Never null; empty when there are none.

`this` does not exist in `main`, because it is static. To use instance state, `main` constructs an object first, which is what most real `main` methods do.

Which form should you use? For learning, scripts and single-file tools, the compact form is less noise. For anything multi-file, a library, or code other people touch, use the classic form. They are the same language; you are choosing how much to type.

Encapsulation

Four visibility levels, narrowest first:

Level	Reach
<code>private</code>	this class only
<i>(none)</i>	this package. Called package-private.
<code>protected</code>	this package, plus subclasses anywhere
<code>public</code>	everyone

Start at `private` and widen when you have a reason. Widening later is easy. Narrowing later breaks everyone who depended on it.

The point is not secrecy, it is **invariants**. From `Encapsulation.java`:

```
refused: insufficient funds: have 15000, asked for 100000  
balance is still: 15000
```

Because `balanceInCents` is private, every change goes through a method that enforces the rules, so the class can promise the balance is never negative and actually keep that promise. A class with public mutable fields has no invariants at all, since anything can change them at any time.

`final` on a field means assigned exactly once, in the constructor. Note that this is not the same as immutable: a `final` field can still point at a mutable object.

Constructors

Writing any constructor removes the free one. Java supplies a no-argument constructor only if you declare none at all. This is why adding a constructor to an existing class can break callers who wrote `new Thing()`.

`this(...)` delegates to another constructor:

```
Temperature() {  
    this(0.0);  
}
```

Static factories often beat constructors because they have names:

```
Temperature.fromFahrenheit(77.0)  
Temperature.fromKelvin(298.15)
```

Two constructors both taking a single `double` cannot coexist. Two factories can. A factory can also return a cached instance or a subclass, where a constructor always builds something new.

Fields get defaults, locals do not. A field is `0`, `false`, `null` or `0.0` automatically. A local variable used before assignment is a compile error. The asymmetry is deliberate: a field may legitimately be set later by another method, so the compiler cannot prove anything, while a local is used a few lines from its declaration, so the compiler can check and does.

Flexible constructor bodies

The old rule: `super(...)` had to be the very first statement. You constructed the parent, then discovered the arguments were invalid.

JEP 513, final in JDK 25, lets statements run first:

```
Positive(int value) {  
    if (value <= 0) {  
        throw new IllegalArgumentException("must be positive, got " + value);  
    }  
    super(value);  
}
```

`FlexibleConstructorBodies.java` output:

```
Measurement constructor ran with 5
built: 5

now with an invalid value:
rejected: must be positive, got -1
```

Read that carefully. The parent constructor ran **once**, for the valid case. For the invalid one it never ran at all, because the check threw first. Under the old rule that was impossible without smuggling the check into a static helper inside the `super()` call.

`this` is still off limits before `super()`, because the parent has not run and the object is not yet valid. The compiler enforces that.

Run It

```
java modules/05-classes-and-objects/examples/TheClassicForm.java
java modules/05-classes-and-objects/examples/TheClassicForm.java hello world
java modules/05-classes-and-objects/examples/Encapsulation.java
java modules/05-classes-and-objects/examples/Constructors.java
java modules/05-classes-and-objects/examples/FlexibleConstructorBodies.java

./scripts/verify-examples.sh modules/05-classes-and-objects
```

Common Mistakes

Public mutable fields. The class can no longer promise anything about its own state.

Adding a constructor and breaking `new Thing()` elsewhere. The free no-argument constructor vanishes the moment you declare any constructor.

Returning your internal collection from a getter. The caller can now modify your object's state without going through you. Module 04's homework covered the fix.

Assuming `final` means immutable.

```
private final List<String> items = new ArrayList<>();
items.add("still allowed"); // fine, the reference never changed
```

`final` fixes the reference, not the object.

A constructor that does real work. Constructors that open files or make network calls are hard to test and fail in ways callers cannot handle. Build the object, then call a method.

Writing getters and setters for every field reflexively. A setter for every field is a public mutable field with extra steps. Add one when something actually needs to change that value from outside.

Key Takeaways

- `static` on `main` exists because no object exists at startup.
- The classic and compact forms are the same language, differing only in what you type. Use classic for anything multi-file or shared.
- Start fields at `private`. Widening is easy, narrowing breaks callers.
- Encapsulation buys invariants, not secrecy.
- Declaring any constructor removes the free no-argument one.
- Static factories have names, so they can overload where constructors cannot.
- Fields default, locals do not.
- Statements may now precede `super()`, so you can reject bad arguments before the parent object is ever built.

Homework

[homework/README.md](#)

Build a class that cannot be put into an invalid state, then prove it by trying. Reference solution in [solutions/05-classes-and-objects/](#).

Homework 05: An Object That Cannot Be Broken

Build a class whose rules are enforced by the compiler and the constructor rather than by a comment asking callers to be careful. Then try to break it and show that you cannot.

The Task

Create `Booking.java` in the classic form: a `public class Booking` with a `public static void main(String[] args)`.

Inside it, write a `Reservation` class holding:

- a passenger name
- a row number
- a list of seat labels, such as `["A", "B"]`

The Rules It Must Guarantee

A `Reservation` may never exist in a state that breaks any of these:

1. The passenger name is never null and never blank
2. The row is always 1 or greater
3. There is always at least one seat
4. Its seat list can never be changed by anyone outside the object, **including whoever passed the list in**

Rule 4 is the interesting one, and it has two halves. Think about what the caller still holds after they call your constructor.

Prove It

In `main`, attempt each of these and print what happens:

- a null passenger
- a blank passenger
- row `0`
- an empty seat list
- keeping a reference to the list you passed in, then adding to it afterwards
- calling `add` on the list your getter hands back

All six attempts must fail or have no effect. Your output should make it obvious that the object defended itself each time.

Also Include

- At least one static factory method, plus a comment saying what it gives you that a constructor does not
- A `toString()` override so printing a reservation is readable

Acceptance Criteria

- ☐ Runs with `java Booking.java`
- ☐ `public class Booking` with `public static void main(String[] args)`
- ☐ Every field in `Reservation` is `private` and `final`
- ☐ All four rules are enforced in the constructor, before any field is assigned
- ☐ Mutating the caller's original list after construction does not change the reservation
- ☐ Mutating the list returned by the getter either fails or does not change the reservation
- ☐ A static factory exists, with a comment justifying it
- ☐ `toString()` is overridden

Stretch

Make the seat list a plain `ArrayList` field instead of an immutable one, and get rule 4 working anyway. Then write a comment comparing the two approaches: what does each cost, and on which side does the cost fall, the constructor or every call to the getter?

Hint, if rule 4 is slipping through

There are two separate leaks, and fixing one does not fix the other.

The first: after `new Reservation(..., seats)`, who else still holds a reference to that exact list object?

The second: after `reservation.seats()`, what did you just hand out?

`List.copyOf` addresses both at once, and working out why is the point.

Module 06: Inheritance and Polymorphism

Inheritance is the feature Java was sold on in 1995 and the one modern Java advises you to use sparingly. Both of those are true, and this module covers why.

The part you will use every single day is not `extends` at all. It is the `equals` / `hashCode` contract, and this module shows it breaking rather than describing it.

What You'll Learn

- Dynamic dispatch, and why it is the whole point of inheritance
- `super`, and why `@Override` is worth writing every time
- `abstract` and `final` as opposite instructions about your design
- The `equals` / `hashCode` contract, demonstrated by breaking a `HashSet`
- What is *not* polymorphic: fields and static methods

Coming From Another Language

	Python	Java
Inherit	<code>class Dog(Animal)</code>	<code>class Dog extends Animal</code>
Call parent	<code>super().__init__()</code>	<code>super(...)</code> , and it must come first
Multiple inheritance	yes	no, single only. Interfaces fill the gap.
Abstract	<code>abc.ABC</code>	<code>abstract</code> keyword
Preventing subclassing	not really	<code>final</code>
Equality	<code>__eq__</code> and <code>__hash__</code>	<code>equals</code> and <code>hashCode</code> , same pairing

Java allows only one parent class. That restriction is why interfaces exist, and Module 07 covers them.

The Lesson

Dynamic dispatch

One call site, three methods. From `Inheritance.java`:

```
[log] system reboot
[email to ada@example.com] system reboot
[sms to +44 7700 900000] system reboot
```

The loop is written against `Notification`. The compiler checks only that the type *has* a `deliver()` method. Which one runs is decided at runtime from the actual object.

The practical payoff: you can add a fourth notification type without touching that loop. Code written against the base type keeps working.

super and @Override

`super(...)` calls the parent constructor and must come first, with the exception Module 05 covered.

`super.method()` calls the parent's version, which is how you extend behaviour rather than replace it.

Always write `@Override`. It is not decoration, it is an instruction to the compiler to check that you really are overriding something. `OverrideTypo.java` misspells `deliver` as `delivar`:

```
error: delivar() in Email does not override or implement a method from a supertype
    @Override
    ^
```

Without the annotation that compiles perfectly, adds a method nobody calls, and leaves the base implementation running. The code looks right, so the bug survives review. One annotation removes the entire category.

abstract and final

`abstract` says subclasses must fill this in. `final` says nobody may change this. From `AbstractAndFinal.java`:

```
abstract class Shape {
    abstract double area();

    final boolean isLargerThan(Shape other) {
        return this.area() > other.area();
    }
}
```

An abstract class cannot be constructed, which is the point: a shape with no particular shape has no sensible area. Unlike an interface, it can also hold ordinary methods and state.

`final` on a method blocks overriding. `final` on a class blocks extension entirely. `String` and `Integer` are both final, and not out of unhelpfulness: if anyone could subclass `String` and override `equals`, no code anywhere could trust a `String` again.

Make classes final more often than feels natural. A class not designed for extension usually cannot be extended safely, because a subclass can override any method and break invariants the class relies on. Module 09 introduces `sealed`, the middle ground between all and nothing.

The equals/hashCode contract

This is the part you will actually hit. From `EqualsHashCodeContract.java`, a class that overrides `equals` and forgets `hashCode`:

```
two equal points added, set size: 2
contains an equal point:           false
but the two ARE equal:            true
```

Read that again. The set holds two items it agrees are equal, and cannot find an item equal to one it already contains.

`HashSet` does not call `equals` on everything. It computes `hashCode` to choose a bucket and compares only within that bucket. `Object.hashCode` is identity-based, so two distinct instances get different codes regardless of what `equals` says, land in different buckets, and never meet.

Add `hashCode` and it behaves:

```
two equal points added, set size: 1
contains an equal point:           true
```

The full contract:

1. If `a.equals(b)` then `a.hashCode() == b.hashCode()`. This is the one that breaks things.
2. Equal hash codes do not imply equality. Collisions are legal.
3. Reflexive: `a.equals(a)`
4. Symmetric: `a.equals(b) == b.equals(a)`
5. Transitive: `a=b` and `b=c` implies `a=c`
6. Consistent: the same answer every time, if nothing changed
7. `a.equals(null)` is always false

Use `Objects.hash(...)` and pass **exactly** the fields `equals` compares. Using a field in one and not the other breaks rule 1 just as thoroughly as omitting `hashCode` altogether.

`toString` is the third of the family. Not part of the contract, but the difference between a readable log line and `BadPoint@4034c28c`.

The shortcut: a record writes all three correctly from its components. Module 08 covers them, and for a data-carrying class a record is almost always the better answer.

What is not polymorphic

Methods dispatch on the object. Fields and statics do not. From `WhatIsNotPolymorphic.java`, where the object is a `Child` held in a `Parent` variable:

```
instance method: Child.instanceName
field access:    parent field
as a Child:     child field
static method:  Parent.staticName
```

Both fields exist on the object at once. The subclass did not replace the parent's field, it added a second one that hides it. Static methods are likewise *hidden*, not overridden.

The rule that follows: **never shadow a field, never redeclare a parent's static method**. Both are legal, neither does what it looks like, and there is no good reason to write either.

This is also why Module 04 warned that `static` blocks you later. A static method cannot be overridden, so it cannot take part in any of this.

Run It

```
java modules/06-inheritance-and-polymorphism/examples/Inheritance.java
java modules/06-inheritance-and-polymorphism/examples/AbstractAndFinal.java
java modules/06-inheritance-and-polymorphism/examples/EqualsHashCodeContract.java
java modules/06-inheritance-and-polymorphism/examples/WhatIsNotPolymorphic.java

# Fails on purpose.
java modules/06-inheritance-and-polymorphism/examples/OverrideTypo.java

./scripts/verify-examples.sh modules/06-inheritance-and-polymorphism
```

Common Mistakes

Overriding `equals` without `hashCode`. Your objects will go into a `HashMap` and never come out. This is the most common serious bug in this module.

Including a field in `hashCode` that `equals` ignores, or the reverse. Same breakage, harder to spot.

Using a mutable field in `equals` and `hashCode`. Put the object in a set, change the field, and it is now lost inside its own collection, because it is filed under a bucket its new hash code no longer matches.

Omitting `@Override`. Free bug detection, declined.

Calling an overridable method from a constructor. The subclass override runs before the subclass constructor has initialised its fields, so it sees nulls and zeros. Make such methods `final` or `private`.

Reaching for inheritance to reuse code. `extends` means "is a", not "borrows from". If the subclass cannot honestly be used everywhere the parent can, use composition: hold the other object as a field.

Key Takeaways

- **Dynamic dispatch picks the method from the object**, not the variable's type.
- **Always write `@Override`**. It converts silent bugs into compile errors.
- **`abstract` forces subclasses to fill in. `final` forbids change.** Prefer `final` by default.
- **Override `equals` and `hashCode` together, over the same fields, always.**
- **Never shadow fields or redeclare static methods.** They are resolved by declared type, not by object.
- **Prefer composition to inheritance** unless the "is a" relationship is honest.

Homework

[homework/README.md](#)

Break a `HashSet` on purpose, diagnose it, then fix it. Reference solution in [solutions/06-inheritance-and-polymorphism/](#).

Homework 06: The Set That Loses Things

Part one is a bug hunt you cause on purpose. Part two builds a small hierarchy that uses dispatch honestly.

Part One: Break A HashSet

Create `Library.java` in the classic form.

Write a class `BrokenIsbn` holding a single `String code`. Override `equals` so that two `BrokenIsbn` objects with the same code are equal. **Do not override** `hashCode`.

Then, in `main`:

1. Add the same ISBN to a `HashSet` twice
2. Print the set's size
3. Print whether the set `contains` an equal ISBN
4. Print whether the two objects are `equals` to each other

The output will be contradictory. The set will hold two items it agrees are equal, and will fail to find an item it already contains.

Diagnose It

Write a comment explaining **in your own words** why this happens. A correct answer has to mention buckets. Saying "because hashCode is missing" describes the symptom, not the mechanism.

Then write `Isbn`, the fixed version, and show the same four lines behaving.

The Mutation Trap

Write one more class with a **non-final** field used by both `equals` and `hashCode`. Then:

1. Create an instance and put it in a `HashSet`
2. Print `contains(theObject)`
3. Change the field
4. Print `contains(theObject)` again, and the set's size

The object will be lost inside a set that still counts it. Explain why in a comment, and state what that implies about fields used in `equals`.

Part Two: A Mixed Shelf

Write an abstract `LibraryItem` with a title, and three subclasses: `Book`, `Audiobook` and `Magazine`. Each measures its size differently. A book has pages, an audiobook has a duration, a magazine has an issue number.

Requirements:

- `LibraryItem` has one `abstract` method the subclasses must implement
- `LibraryItem` has one ordinary method they inherit
- `LibraryItem` has one `final` method, with a comment saying why it is final
- One subclass calls `super.someMethod()` to extend rather than replace behaviour
- All three subclasses are `final`, with a comment saying why that is the default

Then loop over an array of `LibraryItem` and print all three. One loop, three behaviours.

Acceptance Criteria

- ☐ Runs with `java Library.java`
- ☐ `BrokenIsbn` output shows size `2`, `contains` `false`, and `equals` `true`
- ☐ A comment explains the bucket mechanism in your own words
- ☐ `Isbn` output shows size `1` and `contains` `true`
- ☐ The mutation trap shows `contains` flipping from true to false without the set's size changing
- ☐ `LibraryItem` has an abstract, an ordinary and a `final` method
- ☐ Every `@Override` is annotated
- ☐ The loop is written against `LibraryItem`, never against a subclass

Stretch

Rewrite `Isbn` as a record and delete `equals`, `hashCode` and `toString`. Confirm the behaviour is identical, then write a comment on what you gave up by switching. There is something, and Module 08 will name it.

Hint, if part one refuses to misbehave

Check that you are constructing **new** objects for the second add and for the `contains` call. Reusing the same variable compares an object to itself, which succeeds by identity alone and hides the bug you are trying to produce.

Module 07: Interfaces

A class may extend exactly one class and implement any number of interfaces. Everything in this module follows from that asymmetry.

The module ends with the strongest argument against inheritance you will see in this curriculum, and it is not a slogan. It is two classes from the standard library giving different answers to identical code.

What You'll Learn

- Interfaces as capabilities, and why to accept the narrowest one that works
- When an interface beats an abstract class, and when it does not
- Default methods, and the specific problem they were invented to solve
- Static and private interface methods
- Why the diamond problem is a compile error rather than a guess
- What "prefer composition" actually protects you from

Coming From Another Language

	Python	Java
Contract	duck typing, or <code>Protocol</code>	<code>interface</code> , checked at compile time
Multiple parents	full multiple inheritance	one class, many interfaces
Mixins	via multiple inheritance	via interfaces with default methods
Conflict resolution	MRO decides silently	compile error, you decide

Python resolves ambiguity for you through the method resolution order. Java refuses to, on the grounds that a silent choice between two equally plausible behaviours is how you get bugs nobody can explain.

The Lesson

Interfaces are capabilities

From `Interfaces.java` :

```
private static void launch(Flies thing) {  
    System.out.println("launching: " + thing.fly());  
}
```

That method works on a duck, an aeroplane, and anything anyone writes later. It does not care what the object *is*, only that it can fly.

```
launching: duck flapping
launching: jet engines
```

A `Penguin` that implements `Swims` and `Speaks` but not `Flies` will not compile if passed in. The capability is checked, not assumed.

Accept the narrowest capability that does the job. A parameter typed `Flies` can take types that do not exist yet. One typed `Duck` cannot.

Interface or abstract class?

	interface	abstract class
How many per class	any number	exactly one
Instance fields	no	yes
Constructors	no	yes
State	no	yes

Ask what you are modelling. **"Can do" is an interface:** `Flies`, `Comparable`, `AutoCloseable`. **"Is a" is an abstract class**, and only when subclasses genuinely share state and construction logic.

When both fit, prefer the interface. It leaves the implementer's single inheritance slot free, and that slot is spent easily and regretted later.

Default methods, and why they exist

From `DefaultMethods.java`:

```
interface Collection {
    List<String> tracks();                // must implement

    default int count() { return tracks().size(); } // free
    default boolean isEmpty() { return count() == 0; }
}
```

The history matters here. Before Java 8, adding a method to an interface broke every existing implementation everywhere, immediately. Interfaces in public libraries were effectively frozen forever.

Java 8 needed to add `stream()` to `java.util.Collection`. Without default methods that one addition would have broken every collection class ever written by anyone.

Default methods exist for library evolution. They are not a way to sneak multiple inheritance in, and using them as one produces the next section.

Two smaller additions:

- **Private interface methods** (Java 9) let several defaults share a helper without exposing it to implementers.
- **Static interface methods** belong to the interface itself. This is why the library has `List.of`, `Map.of` and `Comparator.comparing`.

The diamond problem

Two interfaces, the same default, one class implementing both. `DiamondConflict.java`:

```
error: types Timestamped and Versioned are incompatible;
class Entry implements Timestamped, Versioned {
^
    class Entry inherits unrelated defaults for describe() from types Timestamped and
    Versioned
```

Java banned multiple class inheritance for twenty years to avoid exactly this. Default methods reintroduced a narrow version, so the language handles it the only safe way: it makes you decide.

```
@Override
public String describe() {
    return Timestamped.super.describe() + " / " + Versioned.super.describe();
}
```

The `InterfaceName.super.method()` syntax exists solely for this. There is no conflict when only one interface supplies a default, or when the class supplies its own.

What composition actually protects you from

This is the part worth remembering. `CompositionOverInheritance.java` writes one counting class three ways. Real output:

```
--- inheritance, extending HashSet ---
added 3, counter says: 6    <- wrong

--- the SAME code, extending ArrayList ---
added 3, counter says: 3    <- right, by luck

--- composition ---
added 3, counter says: 3    <- right, by design
```

The override is identical in both subclasses:

```
@Override public boolean add(String item)    { added++; return super.add(item); }
@Override public boolean addAll(Collection<? extends String> items) {
    added += items.size();
    return super.addAll(items);
}
```

`HashSet.addAll` calls `this.add` for each element, so the override runs again and the count doubles.

`ArrayList.addAll` copies in bulk, so it does not.

Two classes from the same library, the same code, different answers. And the subclass cannot see why from its own source.

That is the **fragile base class** problem. The correctness of your class depends on an implementation detail the parent never promised and may change in any release. Composition removes the dependency entirely: hold the collection, do not become one.

The cost is that you write the methods you want to expose rather than inheriting forty. That cost is usually worth paying, and it forces a good question: which of those forty did this type actually want to offer?

Run It

```
java modules/07-interfaces/examples/Interfaces.java
java modules/07-interfaces/examples/DefaultMethods.java
java modules/07-interfaces/examples/CompositionOverInheritance.java

# Fails on purpose.
java modules/07-interfaces/examples/DiamondConflict.java

./scripts/verify-examples.sh modules/07-interfaces
```

Common Mistakes

Typing a parameter to a class when an interface would do. It shuts out every type written after yours.

Using default methods to build mixins. They were made for library evolution. Interfaces still cannot hold state, so a "mixin" with anything to remember does not work anyway.

Extending a library class to add behaviour. The counting example is what happens. Wrap it instead.

Forgetting that interface methods are implicitly public. Implementing one with weaker visibility is a compile error, and the message is not obvious the first time.

Reaching for an abstract class because "there might be shared code later". Start with the interface. Adding an abstract class beneath it later is easy; reclaiming an implementer's inheritance slot is not.

Key Takeaways

- **One superclass, many interfaces.** That limit is the reason interfaces exist.
- **Accept the narrowest capability that does the job**, so future types can satisfy it.
- **"Can do" is an interface, "is a" is a class.** When both fit, take the interface.
- **Default methods exist so libraries can grow** without breaking implementers.
- **Conflicting defaults are a compile error**, resolved with `Interface.super.method()`.
- **Inheritance couples you to the parent's implementation**, not just its API. The same override is correct under `ArrayList` and wrong under `HashSet`.

Homework

[homework/README.md](#)

Design with capabilities, then reproduce the fragile base class bug and fix it by composing. Reference solution in [solutions/07-interfaces/](#).

Homework 07: Capabilities, Conflicts and a Fragile Base

Three parts. Design with capabilities, resolve a conflict Java refuses to resolve for you, then reproduce the bug that makes "prefer composition" more than a slogan.

Part One: Smart Devices

Create `Devices.java` in the classic form.

Define two interfaces:

- `Switchable` — has a `name()` the implementer must supply, plus `turnOn()` and `turnOff()` as **default** methods
- `Dimmable` — has `setLevel(int)` and `full()` as defaults, and uses a **private interface method** to clamp the level into 0..100 so the rule is written once

Then three device classes:

Device	Implements
Lamp	both
Thermostat	<code>Switchable</code> only
Speaker	both, but overrides <code>setLevel</code>

In `main`, loop over the devices **through the interface type**, never through the concrete class. Show that passing a `Thermostat` where a `Dimmable` is expected is impossible, and say in a comment how you know.

Part Two: A Conflict You Must Resolve

Define `Timestamped` and `Versioned`, each with a **default** method of the same name and signature. Write a class implementing both.

It will not compile. Read the error, then resolve it explicitly.

In a comment, quote the error you got and explain why Java refuses to choose for you when Python's method resolution order would have picked silently.

Part Three: Break A Subclass

Write `LoggingHashSet` extends `HashSet<String>` that keeps a log of everything added, overriding both `add` and `addAll`.

Add three items using `addAll` and print the log size. It will not be 3.

Then write `LoggingCollection` that **holds** a `HashSet` instead of extending one, with the same two methods. Add the same three items and print the log size.

Explain It

Write a comment, in your own words, covering:

1. The mechanism. Why does the inherited version over-count?
2. Why nothing in `LoggingHashSet` 's own source looks wrong.
3. What the composed version does differently that removes the problem.

Acceptance Criteria

- ☐ Runs with `java Devices.java`
- ☐ `Switchable` and `Dimmable` each have at least one `default` method
- ☐ `Dimmable` has a `private` interface method used by its defaults
- ☐ `Speaker` overrides a default, `Lamp` does not
- ☐ The loops in `main` are typed to the interface, not the class
- ☐ The conflicting-defaults class resolves with `Interface.super.method()`
- ☐ A comment quotes the real compiler error you saw
- ☐ `LoggingHashSet` prints a log size that is not 3
- ☐ `LoggingCollection` prints 3
- ☐ Your explanation names the mechanism, not just the symptom

Stretch

Change `LoggingHashSet` to extend `ArrayList<String>` instead, leaving everything else identical. The count will come out right.

Write a comment on what that tells you about the original bug, and about how much you can trust a subclass you have reviewed but whose parent you have not read.

Hint, if part three gives you 3 straight away

Check which collection you extended, and check that you added the items with `addAll` rather than three separate `add` calls. Both details matter, and one of them is the entire point of the exercise.

Module 08: Records and Enums

Module 06 made you write `equals`, `hashCode` and `toString` by hand, and showed a `HashSet` losing its contents when you got it wrong. This module is the answer to that problem.

Records carry data. Enums fix a set of instances. Between them they replace a large amount of the hand-written boilerplate that older Java is famous for.

What You'll Learn

- What a record generates, shown with `javap` rather than `claimed`
- Compact constructors, for validating and normalising
- Why "immutable" for a record is shallower than it sounds
- Enums as classes with fixed instances, carrying state and behaviour
- Why `ordinal()` is a trap
- How to choose between a class, a record and an enum

Coming From Another Language

	Python	Java
Data holder	<code>@dataclass</code> or <code>NamedTuple</code>	<code>record</code>
Equality	generated by the decorator	generated by the record
Fixed instance set	<code>enum.Enum</code>	<code>enum</code> , but each constant is a full object
Per-constant behaviour	awkward	an abstract method each constant implements
Immutability	<code>frozen=True</code> , still shallow	records, also still shallow

The shallowness caveat applies in both languages, and for the same reason. A frozen container holding a mutable list still holds a mutable list.

The Lesson

What a record is

```
record Point(int x, int y) { }
```

That one line gives you a constructor, `equals`, `hashCode`, `toString`, and an accessor per component. From [Records.java](#):

```
toString: Point[x=3, y=4]
equals:    true
hashCode: true
set size: 1
```

The last line is the point. Module 06's hand-written `BadPoint` put two equal objects in a `HashSet` and then could not find either. A record cannot make that mistake.

Use the Module 01 technique to see what was generated. For `public record Money(long cents, String currency) { }`:

```
public final class Money extends java.lang.Record {
    public Money(long, java.lang.String);
    public final java.lang.String toString();
    public final int hashCode();
    public final boolean equals(java.lang.Object);
    public long cents();
    public java.lang.String currency();
}
```

Three things follow from that:

- **final class** — records cannot be extended, ever.
- **extends Record** — every record already has a superclass, so it cannot extend anything else. It can implement interfaces freely.
- **Accessors have no get prefix.** It is `p.x()`, not `p.getX()`.

A record may have extra methods, static factories and interfaces. What it may **not** have is extra instance state:

```
record Bad(int x) { private int cached; } // does not compile
```

That restriction is the guarantee. A record *is* its components, so two records with equal components are interchangeable.

Compact constructors

Where a record validates and normalises. From `CompactConstructors.java`:

```
record Email(String address) {
    Email {
        if (address == null || !address.contains("@")) {
            throw new IllegalArgumentException("not an email address: " + address);
        }
        address = address.strip().toLowerCase();
    }
}
```

No parameter list, no field assignment. The compiler assigns the parameters to the fields **after** this body runs, so reassigning the parameter changes what gets stored. Writing `this.address = ...` here is a compile error.

Normalisation matters more than it looks. A record's `equals` compares components, so if two logically equal values can have different components, the record is lying:

```
new Fraction(6, 8)    -> 3/4
new Fraction(-1, -2) -> 1/2
```

Without reducing and sign-normalising in the constructor, `6/8` and `3/4` would be unequal despite being the same number. **When a record represents a concept rather than a raw tuple, normalise in the compact constructor.**

Shallow immutability

A record's fields are `final`. That is all the language promises. From [ShallowImmutability.java](#):

```
--- the leak ---
built with: Leaky[title=post, tags=[draft]]
after the caller edited their own list: Leaky[title=post, tags=[draft, smuggled-in]]
after editing the accessor's result:    Leaky[title=post, tags=[draft, smuggled-in,
smuggled-out]]

--- sealed off ---
built with: Safe[title=post, tags=[draft]]
after the caller edited their own list: Safe[title=post, tags=[draft]]
the accessor's list refused modification
```

Two separate leaks, closed by one line:

```
record Safe(String title, List<String> tags) {
    Safe {
        tags = List.copyOf(tags);
    }
}
```

`List.copyOf` copies, so the caller's list is no longer yours, and returns an immutable list, so the accessor is safe to hand out directly.

This matters more for records than for ordinary classes, because records are used as map keys and set elements precisely because their `equals` is free. Module 06's mutation trap applies in full: put a `Leaky` in a `HashSet`, edit its list, and it is lost inside the set that still counts it.

Write `List.copyOf` in the compact constructor every time a component is a collection.

Enums

An enum is not a list of integer constants. Each constant is a full object with its own state and, if you want, its own behaviour. From `Enums.java`:

```
MERCURY  gravity  3.70  a 75kg person weighs  277.7 N
EARTH    gravity  9.80  a 75kg person weighs  735.2 N
MARS     gravity  3.71  a 75kg person weighs  278.4 N
```

Constant-specific behaviour is the feature worth knowing:

```
enum Operation {
    PLUS("+") { @Override double apply(double a, double b) { return a + b; } },
    MINUS("-") { @Override double apply(double a, double b) { return a - b; } };

    abstract double apply(double a, double b);
}
```

Adding a constant without its `apply` is a compile error. The alternative, a `switch` somewhere else, has to be found and updated by hand.

`ordinal()` is a trap. It is the declaration position, so inserting a constant silently renumbers everything after it:

```
PENDING -> stored as P, ordinal happens to be 0
ACTIVE   -> stored as A, ordinal happens to be 1
FAILED   -> stored as X, ordinal happens to be 2
```

Never persist an ordinal and never switch on one. Give the constant an explicit code field, as `Status` does.

Enums are the one place where `==` is idiomatic, because each constant is a singleton.

Choosing the shape

Use	When
<code>record</code>	the type is its data, and two instances with the same data are the same thing
<code>enum</code>	the set of instances is fixed and known at compile time
<code>class</code>	it has identity, mutable state, or behaviour that outweighs its data

The question that settles it: **if two instances hold identical values, are they interchangeable?** Two `Point(3, 4)` are. Two bank accounts with the same balance are not.

Run It

```
java modules/08-records-and-enums/examples/Records.java
java modules/08-records-and-enums/examples/CompactConstructors.java
java modules/08-records-and-enums/examples/ShallowImmutability.java
java modules/08-records-and-enums/examples/Enums.java

./scripts/verify-examples.sh modules/08-records-and-enums
```

Common Mistakes

Putting a mutable collection in a record and calling it immutable. Two leaks, one line to fix.

Persisting `ordinal()`. Someone inserts a constant in the middle and every stored row now means something else.

Using a record for something with identity. A `User` record means two users with the same name and email are the same user. Sometimes true, usually not.

Forgetting to normalise. A record whose `equals` compares raw components will treat `6/8` and `3/4` as different fractions.

Writing `this.x = x` in a compact constructor. It does not compile. Assign to the parameter.

Writing a `switch` over an enum in five places instead of a method on the enum. Constant-specific behaviour keeps the logic with the data and turns an omission into a compile error.

Key Takeaways

- **A record is its components.** The compiler writes `equals`, `hashCode`, `toString` and accessors, correctly.
- **Records are final and already extend `Record`**, so no inheritance, but interfaces are fine.
- **Compact constructors validate and normalise**, and assign to the parameter.
- **Immutability is shallow.** `List.copyOf` a collection component every time.
- **Enum constants are objects** and can carry state and per-constant behaviour.
- **Never use `ordinal()` for anything durable.**
- **Pick a record when equal data means equal thing**, an enum when the instances are fixed, a class otherwise.

Homework

[homework/README.md](#)

Model a small order system with records and enums, then close two immutability leaks you created on purpose. Reference solution in [solutions/08-records-and-enums/](#).

Homework 08: An Order System

Model a small order system, then close two immutability leaks you create on purpose.

Part One: Money

Create `Orders.java` in the classic form.

Write a `Money` record with `long cents` and `String currency`. Its compact constructor must:

- reject a negative amount
- reject a null currency
- **normalise** the currency by stripping whitespace and lowercasing it
- reject any currency outside a supported set of `gbp`, `eur`, `usd`

Then show that `new Money(1250, "gbp")` and `new Money(1250, "GBP ")` are `equals`, and that putting both in a `HashSet` gives one element.

One ordering detail is worth getting right: normalise before checking against the supported set, not after. Write a comment saying why.

Also add `plus(Money)` which refuses to add different currencies, and `percent(int)` which computes a percentage in whole cents.

Part Two: Two Leaks

Write a `LeakyOrder` record with a `String reference` and a `List<String> notes`, with **no** compact constructor.

Then, in `main`:

1. Build one from a list you keep a reference to
2. Add to *your* list afterwards, and print the record
3. Add to the list its accessor returns, and print the record

Both attacks will succeed. Then write `Order`, the fixed version, and show both attacks failing.

Explain It

In a comment, describe **both** leaks separately. They are not the same leak, and a fix for one does not automatically fix the other. Then say why this matters more for a record than for an ordinary class, referring back to Module 06.

Part Three: Shipping and Status

Write a `Shipping` enum with three options, each carrying a description and **its own costing rule** as constant-specific behaviour. Standard shipping is free over 50.00 and costs 3.99 otherwise. Express is always 8.99. Collection is always free.

Write an `OrderStatus` enum with four constants, each carrying an explicit **stable code** such as `"PLC"`. Add a comment saying why the code exists rather than using `ordinal()`.

Finally, write a method that gives advice per status using a `switch` expression with **no default branch**, and a comment explaining what you gain by omitting it.

Acceptance Criteria

- ☐ Runs with `java Orders.java`
- ☐ `Money` normalises currency in the compact constructor
- ☐ A comment explains the normalise-then-validate ordering
- ☐ `Money(1250, "gbp")` equals `Money(1250, "GBP ")`, shown in output
- ☐ `plus` refuses mismatched currencies
- ☐ `LeakyOrder` output shows both attacks succeeding
- ☐ `Order` output shows both attacks failing
- ☐ A comment describes the two leaks separately
- ☐ `Shipping` uses constant-specific behaviour, not a `switch`
- ☐ `OrderStatus` carries an explicit code, with a comment on why
- ☐ The advice `switch` has no `default`

Stretch

Add a `CANCELLED` handling branch to your advice method, then add a fifth constant to `OrderStatus` and **do not** update the switch. Compile it, record the exact error, and put it in a comment.

Then add a `default` branch, recompile, and describe what changed about the safety of the code.

Hint, if part two's second attack does not work

Check what your accessor returns. A record generates an accessor that hands back the field exactly as stored. If the field is a plain `ArrayList`, so is what the caller receives, and they can do whatever they like to it.

Module 09: Sealed Types and Pattern Matching

This module is where three features you have already met stop being separate things. Sealed interfaces say what the cases are, records say what data each case carries, and an exhaustive switch says what to do with them. The compiler checks that all three agree.

The result is what other languages call algebraic data types, written in ordinary Java.

What You'll Learn

- `sealed`, and the three modifiers every permitted subtype must choose from
- Pattern matching with `instanceof`, and the cast it removes
- Switch patterns, guards with `when`, and explicit `case null`
- Record patterns, including nested destructuring
- Why exhaustiveness is the whole payoff, and what `default` costs you

Coming From Another Language

	Elsewhere	Java
Closed set of cases	Rust <code>enum</code> , Kotlin <code>sealed</code> , TS union	<code>sealed interface</code>
Destructuring	Python <code>match</code> , JS destructuring	record patterns
Guard	<code>if</code> inside the arm	<code>when</code> clause
Exhaustiveness	Rust enforces it	enforced for sealed types

If you have written Rust or Kotlin this will feel immediately familiar. If you have written older Java, this is the feature that most changes how you model data.

The Lesson

sealed

```
final      nobody may extend this
sealed     only these named types may extend this
(neither)  anybody, anywhere, forever
```

Module 06 argued for `final` by default. `sealed` is the answer when you *do* want subtypes, but a known, fixed set:

```
sealed interface Shape permits Circle, Rectangle, Triangle { }
```

Every permitted subtype must declare its own intent, choosing one of `final`, `sealed`, or `non-sealed`. Leaving all three off is a compile error:

```
error: sealed, non-sealed or final modifiers expected
```

Records are implicitly final, which is why a record needs no modifier. That pairing is deliberate, and it is the shape most modern Java data modelling takes.

An uninvited type gets refused:

```
error: class is not allowed to extend sealed class: Result (as it is not listed in its
'permits' clause)
record Maybe(String hint) implements Result {
^
```

Seal when the set of cases is part of the design. A payment is a card, a transfer or a voucher. A parse either succeeded or it did not. In those, "someone might add another later" is a risk you want caught, not an extension point.

Do not seal a plugin interface. Sealing something third parties should implement is how you ship a library nobody can extend.

Pattern matching

`case String s` tests the type, casts, and names the result, all at once. From `PatternMatching.java`, which writes the same logic twice:

```
// modern
case String s when s.isEmpty() -> "an empty string";
case String s -> "text of length " + s.length();

// old
if (o instanceof String) {
    String s = (String) o;
    return s.isEmpty() ? "an empty string" : "text of length " + s.length();
}
```

The old form names the type twice and performs a separate cast. That cast is where bugs lived: nothing stops you writing `if (o instanceof String) { Integer i = (Integer) o; }`, which compiles and throws.

Three details worth knowing:

- **Guards use `when`**, and guarded cases must come *before* the unguarded case for the same type. First match wins, so an unguarded `case String s` placed first would swallow everything.
- **`case null` is explicit.** Without it, switching on null throws `NullPointerException`, the pre-existing behaviour kept for compatibility.
- **`instanceof` patterns work outside switch**, which is where you will use them most:

```
if (o instanceof String s && s.length() > 5) {
    return s.substring(0, 5) + "...";
}
```

The scoping is smarter than it looks. After an early return, the variable is in scope for the rest of the method:

```
if (!(o instanceof String s)) {
    return -1;
}
return s.length(); // s is in scope here
```

Record patterns

Destructuring in the pattern itself. From `RecordPatterns.java`:

```
case Circle(Point(var x, var y), var r) when x == 0 && y == 0 ->
    "circle of radius " + r + " at the origin";

case Rectangle(Point(var x1, var y1), Point(var x2, var y2))
    when (x2 - x1) == (y2 - y1) ->
    "a square of side " + (x2 - x1);
```

Real output:

```
circle of radius 5.0 at the origin
a square of side 4
a 6 by 3 rectangle
```

Detecting a square needed no method on `Rectangle` at all. The pattern matched the type, pulled out two nested points, bound four values and compared them, in one case label.

Exhaustiveness is the payoff

A switch over a sealed type needs no `default`, because the compiler can prove it is complete. Leave a case out and `NotExhaustive.java` happens:

```
error: the switch expression does not cover all possible input values
    return switch (payment) {
           ^
missing patterns:
    Voucher _
```

It names the missing type. The `Voucher _` is an unnamed pattern: the compiler is telling you a branch that ignores the value would satisfy it.

This is the whole argument for sealing. Add a constant to the `permits` clause and every incomplete switch in the entire codebase becomes a build failure that points at itself. Writing `default -> 0` instead

would have shipped a voucher with no fee and no warning.

Module 03 made this case for enums. Sealed types extend it to any closed set of shapes, with data attached.

Run It

```
java modules/09-sealed-and-pattern-matching/examples/SealedTypes.java
java modules/09-sealed-and-pattern-matching/examples/PatternMatching.java
java modules/09-sealed-and-pattern-matching/examples/RecordPatterns.java

# Both fail on purpose. Read which type each one names.
java modules/09-sealed-and-pattern-matching/examples/NotExhaustive.java
java modules/09-sealed-and-pattern-matching/examples/NotPermitted.java

./scripts/verify-examples.sh modules/09-sealed-and-pattern-matching
```

Common Mistakes

Adding `default` to a switch over a sealed type. You have just turned every future compile error into a silent wrong answer. This is the single most important habit in this module.

Putting the unguarded case first. `case String s` before `case String s when ...` makes the guarded branch unreachable, and the compiler says so.

Forgetting `case null`. A switch on a null reference throws unless you list it. If null is possible, handle it visibly.

Sealing an interface third parties need to implement. Sealed is for closed sets you own, not for extension points.

Declaring a permitted subtype with no modifier. Every one must say `final`, `sealed` or `non-sealed`. Records already say `final`.

Reaching for deep inheritance where a sealed hierarchy fits. If the subtypes differ in data rather than behaviour, a sealed interface of records plus a switch is usually clearer than an abstract class with overrides.

Key Takeaways

- `sealed` names the complete set of subtypes, and every one of them must declare `final`, `sealed` or `non-sealed`.
- **Patterns test, cast and bind in one step**, removing the cast that used to go wrong.
- **Guards use `when` and must precede the unguarded case** for the same type.

- `case null` is explicit; otherwise a null throws.
- **Record patterns destructure, and they nest** as deep as the data.
- **Omit `default` over a sealed type.** The missing-case compile error is the feature you are paying for.
- **Sealed interface + records + exhaustive switch** is how modern Java models a closed set of data shapes.

Homework

[homework/README.md](#)

Model an expression evaluator as a sealed hierarchy, then prove the compiler catches a forgotten case.

Reference solution in [solutions/09-sealed-and-pattern-matching/](#).

Homework 09: An Expression Evaluator

Model arithmetic expressions as a sealed hierarchy, then let the compiler catch a case you forget on purpose.

This is the classic use for sealed types plus records plus pattern matching, and it is worth writing once by hand.

Part One: The Model

Create `Calculator.java` in the classic form.

Define a `sealed interface Expr` permitting five record types:

Record	Components
<code>Literal</code>	an <code>int</code> value
<code>Add</code>	two <code>Expr</code>
<code>Multiply</code>	two <code>Expr</code>
<code>Negate</code>	one <code>Expr</code>
<code>Divide</code>	two <code>Expr</code>

Note that `Add`, `Multiply`, `Negate` and `Divide` hold `Expr`, not `int`. That is what makes the structure a tree and the methods recursive.

Part Two: Three Operations

Write three methods, each a **single switch expression with no default**:

- `evaluate(Expr)` returning the `int` result
- `print(Expr)` returning a readable string, parenthesising binary operations
- `countNodes(Expr)` returning how many nodes the tree contains

Build `(3 + 4) * -(2)` and run all three against it.

Use **record patterns** to pull out the components. Write `case Add(Expr left, Expr right)`, not `case Add a` followed by `a.left()`.

Part Three: Prove The Compiler Is Watching

Add a sixth record, `Power`, to the `permits` clause. Do **not** update `evaluate`.

Compile it. Record the exact error, including the missing pattern it names, in a comment. Then either handle `Power` or remove it again.

Write one sentence on what would have happened instead if `evaluate` had a `default` branch.

Part Four: Simplify

Write `simplify(Expr)` returning a simpler but equivalent `Expr`, using **guards** for these rules:

Rule	Example
<code>0 + x</code> and <code>x + 0</code>	<code>(0 + 7)</code> becomes <code>7</code>
<code>1 * x</code> and <code>x * 1</code>	<code>(1 * 9)</code> becomes <code>9</code>
<code>0 * x</code> and <code>x * 0</code>	<code>(0 * 9)</code> becomes <code>0</code>
<code>--x</code>	<code>--5</code> becomes <code>5</code>
constant folding	<code>(2 + 3)</code> becomes <code>5</code>

The double-negation rule is the interesting one. It wants a **nested** record pattern, matching two levels in a single case label.

Ordering matters here. Say in a comment why the guarded cases must come before the unguarded case for the same type.

Acceptance Criteria

- ☐ Runs with `java Calculator.java`
- ☐ `Expr` is `sealed` with an explicit `permits` clause
- ☐ `evaluate`, `print` and `countNodes` are each one switch with **no default**
- ☐ Record patterns are used to destructure, not accessor calls
- ☐ A comment quotes the real "missing patterns" error you produced
- ☐ A sentence explains what `default` would have cost
- ☐ `simplify` implements all five rules using guards
- ☐ The double-negation rule uses a nested record pattern
- ☐ A comment explains the guarded-before-unguarded ordering

Stretch

Division by zero cannot be handled by returning an `int`. Model the outcome as a second sealed interface, `Result`, permitting `Success(int value)` and `Failure(String reason)`, and write `evaluateSafely` returning it.

Then write a comment comparing this to throwing an exception. Module 10 covers exceptions next, so note what you think each approach costs the caller before you read it.

Hint, if the nested pattern will not compile

`Negate` holds an `Expr`, and `Negate` is itself an `Expr`. So a `Negate` whose component is another `Negate` is matched by naming the type twice and destructuring both. Write the outer pattern first and let the inner one sit inside its parentheses.

Module 10: Exceptions

Java splits exceptions into two families and makes the compiler police one of them. It is nearly alone in this, and the argument about whether that was a good idea has run for twenty-five years.

This module covers the mechanics, states the argument fairly, and shows the three failure modes that cost people real debugging time: a swallowed exception, a lost cause, and a `close()` that hides the real problem.

What You'll Learn

- Checked versus unchecked, and how to decide which to throw
- `try / catch / finally`, multi-catch, and catch ordering
- try-with-resources, reverse close order, and suppressed exceptions
- Why `return` inside `finally` is banned in every style guide
- Custom exceptions, and the constructor argument people forget

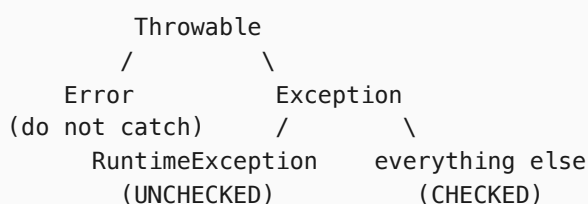
Coming From Another Language

	Python / JS	Java
Compiler-enforced handling	none	checked exceptions
Catch several types	<code>except (A, B)</code>	<code>catch (A B e)</code>
Cleanup	<code>finally</code> , <code>with</code>	<code>finally</code> , try-with-resources
Resource protocol	<code>__enter__ / __exit__</code>	<code>AutoCloseable</code>
Exception chaining	<code>raise X from e</code>	<code>new X(msg, cause)</code>

The one genuinely new idea is checked exceptions. Everything else you already know under a different name.

The Lesson

The two families



- **Unchecked** (`RuntimeException` and below): the compiler says nothing. These mean a programming mistake. The fix is to change the code, not to handle it.
- **Checked** (everything else under `Exception`): the compiler forces you to catch it or declare it.
- **Error**: the JVM is in trouble. `OutOfMemoryError`, `StackOverflowError`. Do not catch these, you cannot fix them from inside.

A detail worth knowing, from `CheckedVsUnchecked.java`:

```
unchecked: NumberFormatException -> For input string: "12x"
unchecked: NumberFormatException -> Cannot parse null string
```

`Integer.parseInt(null)` throws `NumberFormatException`, **not** `NullPointerException`. Guessing the type rather than reading the javadoc is how you write a `catch` block that never fires.

Multi-catch handles two types the same way, and the variable is implicitly final. **Catch order matters**: a supertype must come after its subtypes, and the compiler rejects an unreachable catch rather than letting it sit there.

The argument, fairly stated

For: a failure the caller can recover from should be visible in the type system. A file might not exist. That is not a bug, and the compiler making you think about it is a feature.

Against: in practice they push people towards `catch (Exception e) {}` to silence the compiler, which is strictly worse than no checking. They leak through abstractions, since adding a `throws` changes every caller's signature all the way up. And they compose badly with lambdas, which is why nothing in the streams API accepts a throwing function.

No language designed after Java has copied them. That is evidence, not proof.

What to do today:

- Unchecked for programming errors: `IllegalArgumentException`, `IllegalStateException`.
- Checked only when the caller can genuinely do something other than log and give up.
- Never declare `throws Exception`. It tells the caller nothing and forces them to catch everything.

try-with-resources

The happy path is unremarkable. The failure path is where the design shows. From

`TryWithResources.java`:

```
open A
open B
body running
close B (and failing)
close A (and failing)
caught: body failed
    suppressed: close B failed
    suppressed: close A failed
```

Two things to take from that:

1. **Resources close in reverse order.** B opened last, closed first. That matters when one resource wraps another.
2. **Close failures are suppressed, not substituted.** The body's exception is the one you catch; failures from `close()` attach to it and are readable with `getSuppressed()`.

Both exist because the hand-written equivalent got them wrong:

```
Resource r = null;
try {
    r = open();
    use(r);
} finally {
    if (r != null) r.close(); // if this throws, it REPLACES the real exception
}
```

A `close()` failure would hide the actual problem and the real cause was gone.

`close()` also runs when the body returns early, so you never need a `finally` for it.

finally, and how it eats exceptions

`finally` always runs. That is the promise and the trap. From [FinallySwallows.java](#):

```
static int swallowsTheException() {
    try {
        throw new RuntimeException("you will never see this");
    } finally {
        return 42;
    }
}
```

Output: `42`. The exception is gone. No log, no stack trace, nothing.

A `return` inside `finally` discards whatever the `try` block was doing, including an exception in flight. The same mechanism silently overwrites a good return value, which is much easier to miss in review.

Never put `return`, `break` or `continue` inside a `finally` block. Every style guide bans it and this is why.

Note also that `finally` does not run if the JVM exits. `System.exit(0)` inside a `try` skips it entirely. "Always runs" means within a normal method exit.

Custom exceptions and the cause

From `CustomExceptions.java`, the same wrapping done twice:

```
--- with the cause preserved ---
  caused by: java.io.IOException: users/7.json: no such file
  root cause: java.io.IOException: users/7.json: no such file

--- with the cause thrown away ---
  caused by: null
  the original IOException is gone. good luck.
```

The cause is the second constructor argument, and forgetting it is the most common mistake in exception handling. Without it, the stack trace starts where you rethrow and says nothing about what actually failed.

Always give a custom exception a constructor that takes a `Throwable` cause.

Carry structured data rather than encoding it in the message, so callers can ask for the id instead of parsing it back out of a string.

Checked or unchecked? `extends Exception` is checked, `extends RuntimeException` is not. Ask whether a caller can realistically do something other than log and give up.

Run It

```
java modules/10-exceptions/examples/CheckedVsUnchecked.java
java modules/10-exceptions/examples/TryWithResources.java
java modules/10-exceptions/examples/FinallySwallows.java
java modules/10-exceptions/examples/CustomExceptions.java

# Fails on purpose.
java modules/10-exceptions/examples/UnhandledChecked.java

./scripts/verify-examples.sh modules/10-exceptions
```

Common Mistakes

The empty catch block.

```
try { readConfig(path); } catch (IOException e) { }
```

It compiles, it silences the compiler, and it converts a visible failure into a program that quietly does the wrong thing. This is the worst line you can write in Java, and the fact that checked exceptions produce it is the strongest argument against the feature.

Losing the cause. `throw new MyException("failed")` inside a catch block discards everything you needed to debug it.

`return` in a `finally` block. Covered above. It eats exceptions.

Catching `Exception` to cover everything. You have just caught bugs alongside expected failures, including ones you have no idea how to handle.

Catching `Throwable` or `Error`. You cannot recover from `OutOfMemoryError`, and catching it turns a crash into a hang.

Using exceptions for control flow. Throwing to exit a loop is slow, because building a stack trace is expensive, and it hides the logic from the reader.

Declaring `throws Exception`. It tells callers nothing except that everything is now their problem.

Key Takeaways

- **Unchecked means a bug, checked means a foreseeable failure.** Choose by asking what the caller could actually do.
- **Never write an empty catch block.** If you truly cannot handle it, wrap it in an unchecked exception and let it travel.
- **`try-with-resources` closes in reverse order and suppresses rather than replaces,** which hand-written cleanup got wrong for years.
- `return` inside `finally` silently discards exceptions.
- **Always pass the cause** when wrapping, and always give custom exceptions a cause constructor.
- **Never declare `throws Exception`.**

Homework

[homework/README.md](#)

Build a small config loader, then reproduce three exception anti-patterns and fix each one. Reference solution in [solutions/10-exceptions/](#).

Homework 10: A Config Loader That Tells The Truth

Build something that fails in several ways, then reproduce three anti-patterns so you have written each one at least once deliberately rather than by accident.

Part One: The Loader

Create `ConfigLoader.java` in the classic form.

Write a `load(String name)` method returning a `Map<String, String>` parsed from `key=value` lines. Simulate three files:

Name	Behaviour
<code>app.conf</code>	returns <code>host=localhost</code> and <code>port=8080</code>
<code>missing.conf</code>	the reader throws <code>IOException</code>
<code>broken.conf</code>	parses, but <code>port</code> is not a number

Write a `ConfigException` that:

- is **checked**, with a comment saying why checked is the right call here
- carries the **source file name** as a field, not glued into the message
- has a constructor taking a `Throwable` cause

Load all three files in `main`. For each failure, print the message, the source, and the cause.

The `broken.conf` case is the interesting one. `Integer.parseInt` throws an *unchecked* `NumberFormatException`. Decide whether to let that escape as-is or convert it, and justify your choice in a comment.

Part Two: Three Anti-Patterns

Write each of these twice, once broken and once fixed, and print both results.

1. The empty catch block. Catch something and do nothing. Then fix it.

In a comment, explain in your own words why this is worse than letting the program crash.

2. `return` inside `finally`. Throw from the `try`, `return` from the `finally`, and show that the exception vanishes. Then fix it.

Explain how this is the empty catch block wearing different clothes.

3. Catching `Exception` too broadly. Write a method that *intends* to handle a parsing failure but whose real bug is a `NullPointerException`, and show the handler misreporting it. Then fix it.

Explain what the broad catch cost whoever has to debug it.

Part Three: Resources

Open two `AutoCloseable` resources in one try-with-resources. Make **both** `close()` methods throw, and make the **body** throw as well.

Catch the result and print the caught exception plus everything from `getSuppressed()`.

Then write a comment covering:

1. Which resource closed first, and why that order is the correct one
2. Which exception you caught, and what happened to the other two
3. What hand-written cleanup in a `finally` block would have done instead

Acceptance Criteria

- ☐ Runs with `java ConfigLoader.java`
- ☐ `ConfigException` is checked, carries the source as a field, and has a cause constructor
- ☐ All three config cases are exercised and print their cause
- ☐ A comment justifies checked over unchecked for `ConfigException`
- ☐ A comment justifies the `NumberFormatException` decision
- ☐ All three anti-patterns appear broken and fixed, with output showing the difference
- ☐ Each anti-pattern has an explanation in your own words
- ☐ The try-with-resources output shows reverse close order and two suppressed exceptions
- ☐ No empty catch block survives in the final version except the one deliberately labelled as the anti-pattern

Stretch

Rewrite `load` so it never throws at all, returning a sealed `Result` type instead, as Module 09's stretch did.

Then write a short comparison. Which is easier for the caller to get wrong? Which reads better at the call site? Which one survives being called from a lambda inside a stream, and why does that matter given what Module 10 said about checked exceptions and the streams API?

Hint, if the suppressed list comes back empty

Suppressed exceptions only appear when the **body** throws as well. If the body completes normally and a `close()` throws, that close exception becomes the primary one and there is nothing to suppress it against.

Module 11: Generics

Generics moved a whole class of failure from runtime to compile time. Before Java 5, every collection held `Object` and every read needed a cast you simply had to get right.

The feature is easy to use and easy to misread, because one design decision made in 2004 explains nearly every limitation you will hit. That decision is erasure, and this module leads with it.

What You'll Learn

- Generic classes and methods, and why bounds let them do anything useful
- Erasure, what it forbids, and why it was chosen anyway
- Raw types, heap pollution, and where the resulting crash actually lands
- Wildcards through PECS
- Why generics are invariant and arrays are not, and which choice was the mistake

Coming From Another Language

	Python	Java
Type parameters	hints, unenforced	enforced at compile time
At runtime	available via <code>__orig_class__</code>	erased, nothing survives
Variance	annotated on the TypeVar	wildcards at the use site
Primitives	no distinction	<code>List<int></code> is illegal

The erasure row is the one that will surprise you. In Java the type argument genuinely does not exist at runtime, and a surprising number of limitations follow from that one fact.

The Lesson

The basics

```
class Box<T> {  
    private final T value;  
    T get() { return value; }  
}  
  
static <T> T first(List<T> items) { return items.get(0); }
```

For a generic method the angle brackets come **before** the return type, and the type is inferred from the arguments.

Bounded type parameters are what let a generic method do more than shuffle values around:

```
static <T extends Comparable<T>> T max(List<T> items)
```

Without the bound, `T` is treated as `Object` and `compareTo` does not exist. Multiple bounds use `&`, with the class first: `<T extends Number & Comparable<T>>`.

Conventional names are `T` type, `E` element, `K` key, `V` value, `R` result. They are only conventions.

Erasure

From `Erasure.java`:

```
List<String> class: java.util.ArrayList
List<Integer> class: java.util.ArrayList
same class: true
```

The type argument left no trace. **Why it works this way:** generics arrived fifteen years after the JVM. Erasure meant generic and pre-generic code could call each other and run on an unchanged JVM, so existing libraries did not need rewriting. It was a deliberate trade, compatibility then for these limits forever.

What erasure forbids:

Forbidden	Because
<code>x instanceof List<String></code>	no such type exists at runtime
<code>new T[]</code> , <code>new T()</code>	nothing to allocate
<code>show(List<String>)</code> and <code>show(List<Integer>)</code>	same erasure, they collide
<code>List<int></code>	primitives are not objects
<code>static T shared;</code> inside <code>Box<T></code>	one static field per class, not per type argument

The standard workaround when you genuinely need the type at runtime is to pass a `Class<T>`, which is why so many library methods take one.

Raw types and heap pollution

`HeapPollution.java` compiles with a warning and then fails:

```
List<String> words = new ArrayList<>();
List raw = words;    // raw type, legal for backward compatibility
raw.add(42);         // unchecked warning, allowed
```

```
the list now contains: [legitimate, 42]
reading element 0 is fine: legitimate
reading element 1 will not be:
Exception in thread "main" java.lang.ClassCastException:
    class java.lang.Integer cannot be cast to class java.lang.String
```

Look at where the crash lands. The line that broke the invariant was `raw.add(42)`. The failure is at the `get`, because that is where the compiler inserted the cast the declared type promised. In real code those two lines are often in different classes written by different people, and that distance is what makes heap pollution hard to debug.

Never use a raw type in new code. Unchecked warnings mean the compiler has stopped being able to guarantee anything. When you must suppress one, put `@SuppressWarnings` on the narrowest possible scope and say why it is safe.

Wildcards: PECS

Producer Extends, Consumer Super.

Generics are **invariant**: a `List<Integer>` is not a `List<Number>`, even though an `Integer` is a `Number`. Without wildcards you would need one overload per element type.

```
// PRODUCER: reads from the list, so any subtype of Number will do
static double sum(List<? extends Number> numbers)

// CONSUMER: writes into the list, so any supertype of Integer will do
static void addAll(List<? super Integer> destination, List<Integer> source)
```

Each direction costs you the other:

- `? extends` is **read-only**. The list might be a `List<Double>`, so adding an `Integer` would corrupt it. Only `null` may be added.
- `? super` **reads as `Object`**. The list might be a `List<Object>`, so that is all the compiler can promise.

`List<?>` is the unbounded form: read `Object`s, check the size, add nothing. It differs from the raw `List` in the way that matters, since `List<?>` is type-safe and enforced while the raw type simply turns checking off.

Invariance was the right call

Arrays chose the other answer. From [Wildcards.java](#):

```
Object[] array = new String[2]; // compiles: arrays are covariant
array[0] = 42;                 // throws at runtime
```

```
ArrayStoreException: java.lang.Integer
```

Every array write carries a runtime type check because of that decision. Generics refuse the equivalent at compile time, so no check is needed and the failure cannot reach production.

Run It

```
java modules/11-generics/examples/Generics.java
java modules/11-generics/examples/Erasure.java
java modules/11-generics/examples/Wildcards.java

# These two fail on purpose, one at runtime and one at compile time.
java modules/11-generics/examples/HeapPollution.java
java modules/11-generics/examples/CannotCreateGenericArray.java

./scripts/verify-examples.sh modules/11-generics
```

Common Mistakes

Using a raw type to make a warning go away. You have turned off the checking the feature exists to provide.

@SuppressWarnings on a whole class or method. It hides every future mistake in that scope too. Put it on the single declaration that needs it.

Trying `new T[size]`. Use a `List`, or allocate `Object[]` and cast while keeping the array private to the class.

Writing `List<? extends Number>` for a parameter you need to add to. You cannot. If you both read and write, use a plain `List<T>`.

Wildcards on a return type. It forces every caller to deal with the wildcard. Return the concrete type.

Reaching for `Object` where a type parameter would do. Every caller then needs a cast, which is exactly the pre-2004 situation.

Key Takeaways

- **Generics are checked at compile time and erased at runtime.** Nearly every limitation follows from that.
- **Bounds are what let generic code do anything** beyond moving values around.
- **Raw types defeat the whole feature**, and the resulting crash appears far from the line that caused it.
- **PECS:** producer `extends` for reading, consumer `super` for writing.
- **`? extends` cannot be written to; `? super` reads as `Object`.**

- **Generics are invariant and arrays are covariant.** The array choice was the mistake, and `ArrayStoreException` is the bill.

Homework

[homework/README.md](#)

Build a generic container, hit three erasure limits, and cause heap pollution on purpose. Reference solution in [solutions/11-generics/](#).

Homework 11: A Typed Cache

Build a generic container, run into three erasure limits on purpose, and cause heap pollution so you can see exactly where the blame lands.

Part One: The Cache

Create `TypedCache.java` in the classic form.

Write a `Cache<K, V>` with a fixed capacity:

- `put(K key, V value)` — when full, evict the **oldest** entry before inserting
- `get(K key)` — returns the value or `null`
- `size()` and `keys()`

Show it working with `Cache<String, Integer>`, prove eviction happens, then use the same class as `Cache<Integer, List<String>>` to show one implementation serving unrelated types.

Hit The Limits

While writing it, try each of these, confirm it does not compile, then solve it another way. Document all three in a comment on the class:

1. Store the values in a `V[]` array
2. Validate with `if (value instanceof V)`
3. Share a fallback via `static V defaultValue;`

For each, say what the error was and what you did instead.

Part Two: Bounds and PECS

Write three methods:

Method	Signature shape
<code>largest(List<T>)</code>	needs a bound so it can call <code>compareTo</code>
<code>average(...)</code>	reads numbers, never writes
<code>drainInto(...)</code>	writes integers, never reads

Give `average` and `drainInto` the correct wildcards. Prove `average` works on both a `List<Integer>` and a `List<Double>`, and that `drainInto` works into both a `List<Number>` and a `List<Object>`.

Explain It

Write a comment covering, in your own words:

- Why `largest` cannot work without its bound
- Why `average` uses one wildcard and `drainInto` the other
- What each wildcard **gives up**, and why giving that up is what makes it safe

Answers that just restate "producer extends, consumer super" do not count. Say what the compiler would refuse to let you do in each case.

Part Three: Pollute The Heap

Build a `List<String>`, assign it to a **raw** `List`, and add an `Integer` through the raw reference.

Then print, in order:

1. The list's contents
2. Element 0, read as a `String`
3. Element 1, read as a `String`, catching what happens

The Question That Matters

Write a comment answering: **which line broke the invariant, and which line threw?**

Then explain why they are different lines, what the compiler inserted where, and why this is harder to debug than a normal crash.

Acceptance Criteria

- ☐ Runs with `java TypedCache.java`
- ☐ `Cache<K, V>` is generic over both parameters and evicts the oldest entry
- ☐ The same `Cache` class is used with two unrelated type argument pairs
- ☐ A comment documents all three erasure limits and the workaround used
- ☐ `largest` declares a bound, with a comment on why it is required
- ☐ `average` and `drainInto` use the correct wildcards
- ☐ Both are proven against two different element types
- ☐ The explanation names what each wildcard forbids
- ☐ Heap pollution is reproduced and the `ClassCastException` caught
- ☐ A comment identifies the offending line and the throwing line as different

Stretch

Make `Cache` evict the **least recently used** entry rather than the oldest inserted, so a `get` counts as a use.

`LinkedHashMap` can do this for you with one constructor argument and one overridden method. Find it, use it, and write a comment on what you would have had to write by hand otherwise.

Hint, if part three does not throw

Check that you are reading element 1 into a `String` variable, not into `var` or `Object`. The cast is inserted because of the **declared type at the read site**. Read it as `Object` and there is no cast, so there is no failure, and the polluted list sits there quietly until someone else reads it properly.

Module 12: Collections

Four shapes, several implementations each. Choosing correctly is most of the skill, and the rest is knowing which three or four traps exist.

Two of those traps are the same rule you have already met twice. Module 06 lost an object inside a `HashSet`; Module 08 warned about a mutable component in a record. Both were this module's rule, seen from different angles.

What You'll Learn

- `List`, `Set`, `Map`, `Deque`, and which implementation to reach for
- Sequenced collections, and the thirty-year gap they closed
- `List.of` and friends, with four behaviours that surprise people
- Why removing during iteration throws, and the four correct alternatives
- The rule about keys, stated plainly at last

Coming From Another Language

	Python	Java
Ordered sequence	<code>list</code>	<code>ArrayList</code>
Unique	<code>set</code>	<code>HashSet</code> , or <code>LinkedHashSet</code> for order
Key to value	<code>dict</code> , insertion-ordered	<code>HashMap</code> unordered, <code>LinkedHashMap</code> ordered
Sorted	<code>sorted()</code> on demand	<code>TreeMap</code> , <code>TreeSet</code> keep it sorted
Immutable literal	<code>tuple</code> , <code>frozenset</code>	<code>List.of</code> , <code>Set.of</code> , <code>Map.of</code>
Stack / queue	<code>list</code> , <code>deque</code>	<code>ArrayDeque</code>

The row that catches Python developers: **`HashMap` does not preserve insertion order.** Python dictionaries have since 3.7. If you need the order, say so by using `LinkedHashMap`.

The Lesson

Choosing

From `ChoosingACollection.java`:

Need	Use
index, or duplicates	<code>ArrayList</code>
uniqueness, order irrelevant	<code>HashSet</code>
uniqueness, stable output	<code>LinkedHashSet</code>
sorted	<code>TreeSet</code> / <code>TreeMap</code>
key to value	<code>HashMap</code> , or <code>LinkedHashMap</code>
stack or queue	<code>ArrayDeque</code>

`ArrayList` versus `LinkedList` : use `ArrayList` . A linked list gives $O(1)$ insertion *at a known position*, but finding that position is $O(n)$, and every element costs an extra object plus two pointers. Sequential memory is fast and pointer chasing is not, so `ArrayList` wins in practice even where the big-O table says it should lose.

Never use `java.util.Stack` . It extends `Vector` , synchronises every method for a threading model abandoned in 1998, and iterates bottom-to-top, which is backwards for a stack. `ArrayDeque` is the answer for both stacks and queues.

A `HashSet` 's iteration order is not a promise. It looking stable in your tests means nothing; it can change between JDK versions.

Three `Map` methods worth learning immediately:

```
map.getOrDefault(key, 0)
map.computeIfAbsent(key, k -> new ArrayList<>()).add(value)    // map of lists
map.merge(word, 1, Integer::sum)                               // counting
```

Sequenced collections

Added in Java 21 to close a gap that had been open since 1998. Getting the first element used to be three different expressions depending on the type, and getting the **last** element of a `LinkedHashSet` meant iterating the whole thing.

From `SequencedCollections.java` :

```
getFirst:  a
getLast:   c
reversed:  [c, b, a]
after add, the view shows: [d, c, b, a]
```

`reversed()` is a **view**, not a copy. It costs nothing to create and reflects later changes to the original.

`SequencedMap` adds `firstEntry` , `lastEntry` , `putFirst` and `putLast` :

```
firstEntry: x=1
putFirst:   {w=0, x=1, y=2, z=3}
```

Interface	Implemented by
SequencedCollection	List , Deque , LinkedHashSet , SortedSet
SequencedSet	LinkedHashSet , TreeSet
SequencedMap	LinkedHashMap , TreeMap

HashSet and HashMap are deliberately **not** sequenced. They have no defined order, so the compiler now refuses the question rather than letting you iterate and hope.

The immutable factories

From `ImmutableFactories.java`, four surprises, all deliberate:

1. **Genuinely immutable.** Every mutating method throws. Not "unmodifiable by convention".
2. **null is rejected.** `List.of("a", null)` throws `NullPointerException`. Converting old code to `List.of` can surface nulls you did not know you had.
3. **Duplicates are an error.** `Set.of(1, 1)` throws `duplicate element: 1`, where `new HashSet<>(List.of(1, 1))` would quietly give you one element. In a literal, a duplicate is almost always a typo.
4. **Set.of iteration order is unspecified** and deliberately varies. If you need order, use `List.of` or `LinkedHashSet`.

Three things that look alike and are not:

	add	set
<code>List.of</code>	throws	throws
<code>Arrays.asList</code>	throws	ok
<code>Collections.unmodifiableList</code>	throws	throws

`Arrays.asList` is **fixed size, not immutable**, and writes through to the backing array.

`Collections.unmodifiableList` is a **view** over a mutable list:

```
after mutating the source:
  unmodifiable VIEW sees it: [x, y]
  List.copyOf does not:      [x]
```

`List.copyOf` is usually what you want, which is why Modules 05 and 08 both reached for it.

Mutation during iteration

`MutationDuringIteration.java` throws:

```
Exception in thread "main" java.util.ConcurrentModificationException
  at java.base/java.util.ArrayList$Itr.checkForComodification
```

"Concurrent" is misleading. There is one thread. It means the collection changed concurrently with an iteration, not from another thread.

`ArrayList` keeps a `modCount` that increments on every structural change. The iterator records it and checks before every step, failing fast rather than skipping elements or reading past the end. It is documented as **best-effort**, so never write code that relies on the exception being thrown.

Four correct alternatives:

```
numbers.removeIf(n -> n % 2 == 0);           // 1. usually this

var it = numbers.iterator();                  // 2. when a predicate is not enough
while (it.hasNext()) {
    if (it.next() % 2 == 0) it.remove();
}

// 3. collect, then remove after
// 4. build a new collection (Module 14)
```

The rule about keys

`KeysMustNotChange.java` puts a mutable object in a map and renames it:

```
after renaming the key object:
get(key):      null
containsKey:   false
size:          1
but iterating still finds it: [MutableKey(invoices)]
after remove(key), size is still: 1
```

The entry sits in the bucket for its **old** hash code. Lookup computes the new one, searches a different bucket, and finds nothing. The map still counts it and iteration still yields it, so the data is not gone. It is merely unreachable by lookup, which is the one thing a map exists to do. It is also now unremovable by key.

Map keys and set elements must be immutable in every field that `equals` and `hashCode` use.

A record whose components are themselves immutable satisfies this for free. That is most of why records and maps go together so well, and it is Module 08's warning restated from the other side.

Run It

```
java modules/12-collections/examples/ChoosingACollection.java
java modules/12-collections/examples/SequencedCollections.java
java modules/12-collections/examples/ImmutableFactories.java
java modules/12-collections/examples/KeysMustNotChange.java

# Fails on purpose.
java modules/12-collections/examples/MutationDuringIteration.java

./scripts/verify-examples.sh modules/12-collections
```

Common Mistakes

Depending on `HashMap` or `HashSet` **iteration order**. It is not defined and it does change.

Removing inside a for-each loop. Use `removeIf`.

Using a mutable object as a key. The entry becomes unreachable the moment a relevant field changes.

Expecting `Arrays.asList` to be immutable. It is fixed-size, and `set` writes through to the array behind it.

Returning `Collections.unmodifiableList(internalList)` and believing it is safe. It is a view. Whoever holds the original can still change what the caller sees.

Reaching for `LinkedList` because insertion is $O(1)$. Finding the insertion point is not, and the constant factors are bad.

Using `java.util.Stack`. Synchronised for no reason, and it iterates the wrong way round.

Key Takeaways

- `ArrayList`, `HashMap`, `ArrayDeque` **cover most needs**. Reach elsewhere for a stated reason.
- Use `LinkedHashSet` or `LinkedHashMap` **whenever order is observable**, such as in output or tests.
- **Sequenced collections** give `getFirst`, `getLast` and `reversed()`, and `reversed()` is a free view.
- `List.of` **rejects null, rejects duplicate set entries, and is truly immutable**. `Arrays.asList` and `unmodifiableList` are neither.
- `removeIf`, **not removal inside a loop**.
- **Keys must not change** in any field `equals` or `hashCode` reads.

Homework

[homework/README.md](#)

Build a word index, pick the right collection at each step, and make a map entry unreachable on purpose. Reference solution in [solutions/12-collections/](#).

Homework 12: A Word Index

Build a small text index. Every step asks you to pick a collection and justify it, because picking correctly is the skill this module is actually teaching.

The Text

```
private static final String TEXT = ""  
    the quick brown fox jumps over the lazy dog  
    the dog barks and the fox runs  
    a quick fox is a happy fox"";
```

Part One: Counting

Create `WordIndex.java` in the classic form.

Count how many times each word appears, then print the three most frequent.

Requirements:

- Use `merge` for the counting. Do **not** write a `get`, null check, `put` sequence.
- The counts must print in a **predictable** order, and a comment must say which map type you chose and why a `HashMap` would have been the wrong call.
- `the` and `fox` both appear four times. Your top-three must be **deterministic** across runs, so decide how to break the tie and say so in a comment.

Part Two: Indexing

Build an index from each initial letter to the set of words starting with it.

Requirements:

- Initials must come out **alphabetically**, without a separate sorting step
- Each word appears **once** per initial, and the words are **sorted**
- Use `computeIfAbsent`, not a null check

Then use sequenced-collection methods to print the first entry, the last entry, and the keys in reverse.

Finally, build the set of unique words **three ways** using `HashSet`, `LinkedHashSet` and `TreeSet`, print all three, and write a comment on what each one promises. One of them promises nothing at all.

Part Three: Lose An Entry

Write a `MutableTag` class with a single renameable field, correct `equals` and `hashCode`, and a `toString`.

Put one in a `HashMap` as the **key**, then rename it. Print, in this order:

1. `get`, `containsKey` and `size` before the rename
2. The same three after
3. The key set, by iterating
4. `size` after calling `remove(tag)`

Explain It

Write a comment covering, in your own words:

- Why `get` returns null when the entry is demonstrably still there
- Why iteration finds it but lookup does not
- Why `remove` cannot delete it either
- What the only remaining way to get rid of it is

Then do the same with a record as the key, and show a freshly constructed equal key still finding the value.

Acceptance Criteria

- ☐ Runs with `java WordIndex.java`
- ☐ Counting uses `merge`
- ☐ A comment justifies the map type over `HashMap`
- ☐ The top-three output is deterministic, with the tiebreak explained
- ☐ The initial index is alphabetical with sorted, deduplicated values
- ☐ `computeIfAbsent` is used
- ☐ `firstEntry`, `lastEntry` and `reversed` all appear
- ☐ All three set types are printed with a comment on what each promises
- ☐ The mutable-key demo shows `get` null, `containsKey` false, `size` 1
- ☐ Iteration still yields the lost entry, and `remove` fails to delete it
- ☐ The explanation covers all four questions above

Stretch

Recover from the lost entry. Without rebuilding the map, remove the unreachable key using an `Iterator`.

Then write a comment on why `Iterator.remove` can do what `Map.remove` cannot, and connect it back to what Module 12 said about `ConcurrentModificationException` being the same machinery viewed from the other side.

Hint, if the top-three keeps changing

Sorting by count alone leaves equal counts in whatever order they happened to be in. That order can come from a hash table, which means it is not stable. Add a second comparison so ties resolve the same way every time.

Module 13: Lambdas

A lambda is an implementation of an interface with exactly one abstract method, written without the ceremony of a class.

That definition is worth holding onto. A lambda is not a new kind of value in Java. It is still an object implementing an interface. Module 07 said an interface with a single abstract method can be implemented without writing a class; this module is that sentence made practical.

What You'll Learn

- Lambda syntax, and what the compiler infers for you
- The four method reference forms, including the two that look identical
- Capture, and why a captured local must be effectively final
- The standard functional interfaces and how they compose
- Why `java.util.function` has so many nearly identical types

Coming From Another Language

	Python / JS	Java
Anonymous function	<code>lambda</code> , <code>=></code>	<code>-></code>
Multi-statement body	JS yes, Python no	yes, with braces
Captured variable may change	yes	no , must be effectively final
Type of a lambda	a function object	an interface implementation
Function type	first class	there is none; use an interface

The two rows that matter: Java has **no function type**, so every lambda needs a target interface, and a captured local **cannot be reassigned**.

The Lesson

The same thing, three ways

From `Lambdas.java`:

```
Greeter named = new PoliteGreeter();           // a named class
Greeter anonymous = new Greeter() {           // an anonymous class
    @Override public String greet(String n) { return "Good day, " + n; }
};
Greeter lambda = name -> "Good day, " + name; // a lambda
```

All three produce an object implementing `Greeter`. The lambda omits the interface name, the method name, the modifiers and the parameter type, because the compiler can work all of them out from the target type.

Syntax forms:

```
() -> doSomething()           // no parameters, parentheses required
name -> "hello " + name       // one parameter, parentheses optional
(String name) -> ...          // explicit type when inference needs help
name -> { ...; return x; }     // block body needs braces and return
(a, b) -> a + b               // two parameters always need parentheses
```

A functional interface has exactly one abstract method. Default and static methods do not count, which is why `Comparator` can have dozens of methods and still work as a lambda target. Write `@FunctionalInterface` on your own: it makes the compiler enforce the rule, so adding a second abstract method later fails at the interface rather than at every lambda that used it.

Method references

Four forms, from `MethodReferences.java`:

Form	Example	Equivalent lambda
static	<code>Integer::parseInt</code>	<code>s -> Integer.parseInt(s)</code>
constructor	<code>ArrayList::new</code>	<code>() -> new ArrayList<>()</code>
bound instance	<code>prefix::concat</code>	<code>s -> prefix.concat(s)</code>
unbound instance	<code>String::toUpperCase</code>	<code>s -> s.toUpperCase()</code>

The unbound form is the one people misread. The lambda's first argument becomes the **receiver**, not an argument. That is why `Function<String, String> upper = String::toUpperCase` works even though `toUpperCase` takes no parameters.

Forms three and four look the same and are not:

```
bound, arg "fix":           pre-fix
bound, arg "amble":        pre-amble
unbound, ("pre-", "fix"):   pre-fix
unbound, ("post-", "fix"):  post-fix
```

The bound one always concatenates onto `prefix`. The unbound one takes its receiver from the first argument, so what it concatenates onto changes per call. Tell them apart by what sits left of the `::`: a

variable means bound, a type name means unbound or static.

When not to use one. `words.forEach(System.out::println)` is clearer. But once anything else happens, a lambda reads better. Contorting code so a method reference fits usually makes it worse.

Capture

A captured local must be **final or effectively final**.

Effectively final means you never assign to it after initialisation. You do not write the keyword; the compiler checks the behaviour. `NotEffectivelyFinal.java`:

```
error: local variables referenced from a lambda expression must be final or effectively final
```

Why the rule exists: a lambda can outlive the method that made it. By the time it runs, that stack frame may be gone, so the lambda holds a **copy of the value**, not a reference to the variable. If reassignment were allowed, two questions would have no good answer: does an already-created lambda see the new value, and what happens when two threads assign at once?

The rule is about the variable, not the object. This is fine:

```
List<String> items = new ArrayList<>();
Runnable r = () -> items.add("x");    // the variable never changes
```

Which is also the loophole people use to get around it:

```
int[] counter = {0};
list.forEach(x -> counter[0]++);    // compiles, and is a smell
```

It compiles because the variable never changes. It is a smell because the lambda is being used for mutation, which is what Module 14 exists to replace.

Fields have no such rule. A lambda captures `this` and reads the field through it, so a mutable field works. That also means a stored lambda keeps the whole enclosing object alive, which matters for long-lived listener lists.

this differs between a lambda and an anonymous class:

```
inside a lambda, this is:          Capture
inside an anonymous class, this is: Capture$1
```

The `$1` is a real, separate class. A lambda creates no such thing, so `this` inside it means the enclosing object. This is a behavioural difference, not a style preference.

The standard interfaces

Interface	Shape	Method
<code>Function<T,R></code>	T in, R out	<code>apply</code>
<code>Predicate<T></code>	T in, boolean out	<code>test</code>
<code>Supplier<T></code>	nothing in, T out	<code>get</code>
<code>Consumer<T></code>	T in, nothing out	<code>accept</code>

Plus `BiFunction`, `BiPredicate`, `BiConsumer`, `UnaryOperator<T>` (a `Function<T,T>`) and `BinaryOperator<T>`.

Composition, from `StandardInterfaces.java`:

```
twice.andThen(plusOne) on 5: 11
twice.compose(plusOne) on 5: 12
```

`andThen` means "do me first, then the argument". `compose` means "do the argument first, then me". Getting them the wrong way round is a classic bug whenever the operations are not commutative.

Predicates combine with `and`, `or` and `negate`.

Primitive specialisations exist because `Function<Integer,Integer>` boxes every value.

`IntPredicate`, `ToIntFunction`, `IntUnaryOperator` and their `long` and `double` equivalents avoid it. This is Module 02's boxing cost showing up in API design, and it is why `java.util.function` is so large.

Run It

```
java modules/13-lambdas/examples/Lambdas.java
java modules/13-lambdas/examples/MethodReferences.java
java modules/13-lambdas/examples/Capture.java
java modules/13-lambdas/examples/StandardInterfaces.java

# Fails on purpose.
java modules/13-lambdas/examples/NotEffectivelyFinal.java

./scripts/verify-examples.sh modules/13-lambdas
```

Common Mistakes

Trying to reassign a captured local. Compute the final value first, or use a second variable.

The `int[] box = {0}` trick to accumulate. It compiles. It also says the lambda wants to be a reduction, which Module 14 covers.

Confusing `andThen` with `compose`. Read them aloud: "and then" is second, "compose" wraps around.

Writing `a.length() - b.length()` as a comparator. It overflows for large values. Use `Comparator.comparingInt`.

Expecting `this` in a lambda to be the lambda. There is no lambda object to refer to.

Declaring your own functional interface when a standard one fits. A reader who knows `Function<String,Integer>` has to go and look up your `StringScorer`.

Storing lambdas in long-lived collections without thinking about `this`. The lambda holds the enclosing object, so it cannot be collected.

Key Takeaways

- **A lambda implements a one-abstract-method interface.** Java has no function type.
- `@FunctionalInterface` makes the compiler enforce the rule on interfaces you own.
- **Four method reference forms**, and the unbound one turns the first argument into the receiver.
- **Captured locals must be effectively final**, because the lambda copies the value and may outlive the frame.
- **The rule is about the variable, not the object**, so mutating a captured collection is legal.
- `andThen` and `compose` run in opposite orders.
- **Primitive specialisations exist to avoid boxing.**

Homework

[homework/README.md](#)

Build a small rules engine out of composed predicates and functions, then hit the capture rule on purpose. Reference solution in [solutions/13-lambdas/](#).

Homework 13: A Rules Engine

Build eligibility rules out of composed predicates, use all four method reference forms, then walk into the capture rule on purpose.

The Data

```
record Applicant(String name, int age, int creditScore, boolean employed) { }

List.of(
    new Applicant("ada",      36, 780, true),
    new Applicant("grace",    45, 610, true),
    new Applicant("alan",     17, 800, false),
    new Applicant("edsger",   52, 550, true),
    new Applicant("barbara",  29, 720, false));
```

Part One: Composed Predicates

Create `RulesEngine.java` in the classic form.

Write **three separate named predicates**:

Name	Rule
<code>isAdult</code>	age 18 or over
<code>hasGoodCredit</code>	credit score 700 or over
<code>isEmployed</code>	employed is true

Then build two composite rules **by combining them**, not by writing a new lambda:

- `eligible` — adult, good credit, and employed
- `needsReview` — adult, but credit below 700

Print a row per applicant showing all three results plus whether they are a minor, using `negate()` for the last one.

Write a comment on why three named predicates beat one large lambda. Your answer should mention reuse: one of the two composite rules shares a clause with the other.

Part Two: All Four Method Reference Forms

Use each form at least once, each with a comment naming which it is:

1. A **static** method reference

2. A **constructor** reference
3. A **bound instance** reference
4. An **unbound instance** reference

Then Explain The Hard Part

Forms 3 and 4 are both written `Something::method`. Write a comment explaining, in your own words:

- What sits to the left of the `::` in each case, and why that is the tell
- Which one picks its receiver once, and which one picks it per call
- **The observable consequence for arity.** A bound reference to a one-argument method needs a functional interface with how many parameters? What about the unbound reference to the same method? Show both.

Finally, use `andThen` to build a pipeline turning an `Applicant` into a credit band of A, B or C, and print it for everyone.

Part Three: The Capture Rule

First, write the version that does **not** compile: a local `int approved = 0;` incremented inside a `forEach`. Confirm the error, then put it in a comment along with an explanation of why the rule exists.

Then write the same count **three** ways that do work:

1. The `int[] box = {0}` trick
2. An ordinary `for` loop
3. A stream, using `filter` and `count`

The Question

Write a comment explaining why the array trick compiles when the plain `int` did not, and why it is still a bad answer. Your explanation must distinguish between the **variable** and the **object it points at**, because that distinction is the whole rule.

Finish by capturing a `List` in a lambda and mutating it, showing that this is legal, then say why `.toList()` is better anyway.

Acceptance Criteria

- ☐ Runs with `java RulesEngine.java`
- ☐ Three separately named predicates exist
- ☐ `eligible` and `needsReview` are built by composing them, not rewritten
- ☐ `negate()` is used

- ☐ A comment justifies composition, mentioning the shared clause
- ☐ All four method reference forms appear, each labelled
- ☐ The bound-versus-unbound explanation covers the arity consequence and shows both signatures
- ☐ An `andThen` pipeline produces credit bands
- ☐ The non-compiling capture attempt is quoted with its real error
- ☐ The count is produced three working ways, all agreeing
- ☐ The explanation distinguishes the variable from the object

Stretch

Write a `Predicate<Applicant>` factory: a method taking a minimum score and returning a predicate that tests against it.

Then write a comment on what that method captures, whether the captured parameter is effectively final, and why a method parameter is allowed to be captured at all given everything this module said about stack frames.

Hint, if `compose` will not type-check

`a.andThen(b)` runs `a` first. `a.compose(b)` runs `b` first. Only one ordering lines up when `a` takes an `Applicant` and `b` takes an `int`. If the compiler refuses, you have them the wrong way round, and that refusal is doing you a favour.

Module 14: Streams

A stream is a pipeline, not a collection. It holds no data and it is consumed once. Nearly everything surprising about streams follows from those two facts.

Module 13 built the parts. Every lambda in a stream pipeline is one of the interfaces from that module, so this is where they start paying off.

What You'll Learn

- Pipeline anatomy, and laziness shown rather than asserted
- Why elements flow through one at a time, not stage by stage
- Collectors, with `groupingBy` as the one that justifies the API
- Gatherers, which fill the gap where custom intermediate operations belong
- Parallel streams, when they help, and the trap that makes them dangerous

Coming From Another Language

	Python	Java
Transform	comprehension, <code>map</code>	<code>.map(...)</code>
Filter	comprehension <code>if</code> , <code>filter</code>	<code>.filter(...)</code>
Lazy	generators	streams, always
Reuse	a list can be iterated twice	a stream cannot
Group	<code>itertools.groupby</code> needs sorting	<code>Collectors.groupingBy</code> does not
Parallel	<code>multiprocessing</code>	<code>.parallel()</code> , and think first

The reuse row is the one that bites. A Python list comprehension gives you a list. A Java stream gives you a pipeline that evaporates once used.

The Lesson

Anatomy

```
words.stream()           // SOURCE
    .filter(w -> w.length() > 4) // INTERMEDIATE, lazy
    .map(String::toUpperCase) // INTERMEDIATE, lazy
    .toList();            // TERMINAL, runs everything
```

Intermediate operations return a stream and do nothing. The terminal operation is what actually runs the pipeline.

Laziness, demonstrated

From `StreamBasics.java`, building a pipeline with a `peek` in it prints nothing:

```
pipeline built. notice nothing was peeked.  
now adding a terminal operation:  
peeked at banana  
...
```

Laziness is what makes **short-circuiting** possible:

```
examined 1  
examined 2  
examined 3  
found Optional[3]
```

Elements 4 and 5 were never touched. It is also why an infinite stream works:

```
Stream.iterate(1, n -> n + 1).map(n -> n * n).limit(5).toList()
```

Element-wise, not stage-wise

A common misreading is that `filter` runs over everything, then `map` runs over everything. It does not. Each element goes through the **whole pipeline** before the next one starts:

```
filter sees a  
map sees a  
forEach sees a  
filter sees b  
...
```

Collectors

From `CollectingResults.java`, `groupingBy` alone justifies learning the API:

```
STAFF.stream().collect(Collectors.groupingBy(Employee::department))
```

The **downstream collector** is the second argument, and it is where the API stops being verbose and starts being worth it:

```
groupingBy(Employee::department, Collectors.counting())  
groupingBy(Employee::department, Collectors.averagingInt(Employee::salary))  
groupingBy(Employee::department, TreeMap::new, Collectors.counting()) // ordered
```

Without the map factory you get a `HashMap`, whose order is undefined. Module 12 applies here too.

`partitioningBy` splits into exactly two groups and always has both keys, even when one side is empty.

`toMap` **throws on a duplicate key** rather than overwriting:

```
duplicate key threw: Duplicate key engineering ...
```

That is usually what you want. Supply a merge function to say otherwise.

`.toList()` **on the stream returns an unmodifiable list** and is the modern form.

`Collectors.toList()` returns a modifiable list of unspecified type. Prefer the former.

Gatherers

Finalised in JDK 24, so no flags here. They fill a real gap: you could always write a custom **terminal** operation as a `Collector`, but never a custom **intermediate** one. Anything stateful across elements meant leaving the stream.

From `UsingGatherers.java`:

Gatherer	Result on <code>1,2,3,4,5</code>
<code>windowFixed(2)</code>	<code>[[1, 2], [3, 4], [5]]</code>
<code>windowSliding(2)</code>	<code>[[1, 2], [2, 3], [3, 4]]</code>
<code>scan(() -> 0, Integer::sum)</code>	<code>[1, 3, 6, 10]</code>

`windowFixed` keeps the short final window rather than dropping it, which is what you want for batching so no records go missing. `scan` is `reduce` that emits every intermediate value, which is what a running balance needs.

`mapConcurrent(n, fn)` maps with a bounded concurrency limit **and preserves order**, running each on a virtual thread. That is a better default than `.parallel()` for IO-shaped work, for reasons the next section explains.

When to reach for one: state across elements, or emitting a different number of elements than you consumed. One in, one out, no memory is just `map`.

Parallel streams

From `ParallelStreams.java`. Adding `.parallel()` is one word, and it is almost never the right call.

The trap, run three times on the same machine:

```
the list now holds: 23839
the list now holds: 22694
the list now holds: 84097
```

100,000 elements were added to an `ArrayList` in parallel. Every run lost a different number, and a fourth run might throw instead, when one thread resizes the backing array while another is mid-write.

It is not reliably wrong, which is worse than being reliably wrong. On a small input or an unloaded machine it often works. That is how this reaches production.

The fix is not a lock, it is `collect` :

```
IntStream.range(0, 100_000).parallel().boxed().toList(); // 100000, always
```

Each thread accumulates into its own container and the results merge at the end, so nothing is shared while being written. Synchronising `add` would also be correct and would be *slower* than the sequential version, because every thread would queue on one lock.

`forEach` does not preserve order in parallel; `forEachOrdered` does.

Consider <code>.parallel()</code>	Avoid it
large N, genuinely expensive per element	small N: fork/join overhead dominates
source splits cheaply: arrays, <code>ArrayList</code> , <code>IntStream.range</code>	<code>LinkedList</code> or iterator sources
no shared state, no ordering requirement	anything doing IO

The IO case deserves emphasis. The common pool is sized for CPU cores, and blocking it starves every other parallel stream in the JVM. Use `Gatherers.mapConcurrent` or Module 19's virtual threads instead.

The honest default: do not write `.parallel()` until a profiler names this pipeline as the bottleneck.

Single use

`StreamsAreSingleUse.java` :

```
IllegalStateException: stream has already been operated upon or closed
```

A stream holds no elements of its own to replay. Keep the **source**, not the stream, and a collection will give you as many streams as you want.

Run It

```
java modules/14-streams/examples/StreamBasics.java
java modules/14-streams/examples/CollectingResults.java
java modules/14-streams/examples/UsingGatherers.java
java modules/14-streams/examples/ParallelStreams.java

# Fails on purpose.
java modules/14-streams/examples/StreamsAreSingleUse.java

./scripts/verify-examples.sh modules/14-streams
```

Common Mistakes

Storing a stream in a field or variable and using it twice. Store the source.

Mutating a shared collection from `forEach`. Use `collect`. In parallel it is a data race; even sequentially it is harder to read than the collected version.

Reaching for `.parallel()` to make something faster. Measure first. It is frequently slower and occasionally wrong.

Blocking inside a parallel stream. You have just taken threads from a pool the whole JVM shares.

Using `peek` for real side effects. It is a debugging window. Implementations are allowed to skip it when the result is not needed.

Forgetting `toMap` throws on duplicates. Supply a merge function when keys can repeat.

A pipeline longer than the loop it replaced. Streams are for expressing *what*, not a rule that loops are forbidden. If the loop is clearer, write it.

Key Takeaways

- **A stream is a lazy pipeline, consumed once.** Keep the source, not the stream.
- **Nothing runs until the terminal operation**, which is what enables short-circuiting and infinite sources.
- **Elements flow one at a time through the whole pipeline**, not stage by stage.
- **`groupingBy` with a downstream collector** is the most useful thing in the API.
- **Gatherers give you custom intermediate operations**, for windowing and running totals.
- **`.parallel()` with shared mutable state loses data unpredictably.** Use `collect`, and measure before parallelising at all.

Homework

[homework/README.md](#)

Analyse a log with a stream pipeline, use a gatherer, and reproduce the parallel data race. Reference solution in [solutions/14-streams/](#).

Homework 14: Log Analysis

Analyse a log with stream pipelines, reach for a gatherer where `map` cannot go, and reproduce the parallel data race so you have watched it happen.

The Data

```
List.of(
    "INFO  auth    120",
    "ERROR auth    450",
    "INFO  billing 80",
    "WARN  billing 900",
    "ERROR auth    380",
    "INFO  search  60",
    "ERROR billing 1200",
    "INFO  auth    95",
    "WARN  search  700",
    "INFO  billing 110");
```

Parse each line into `record Entry(String level, String service, int millis)`.

Part One: Grouping

Create `LogAnalysis.java` in the classic form. Produce all of these, each as a **single** stream pipeline:

1. Entry count per service
2. Average response time per service
3. The distinct levels seen per service, sorted
4. A two-way split of entries into slow (over 500ms) and fast
5. The slowest time per service

Requirements:

- Every grouped result must print in a **predictable** key order. Say in a comment which map type you used and why the default was not acceptable.
- Use a downstream collector for at least three of them, not a second pass.
- For the two-way split, use `partitioningBy` and write a comment on what it guarantees that `groupingBy` on a boolean does not.
- For the slowest time, first try a plain `toMap` and record what happens, then fix it.

Part Two: Gatherers

Extract just the timings, then produce:

1. Batches of three
2. The difference between each consecutive pair of readings
3. A running total

The Question

Write a comment explaining, in your own words, why **none** of these three can be written with `map`. Your answer needs to address all three, and they fail the same test for different reasons. Name the rule `map` obeys and say which part each one breaks.

Part Three: Race It

Add 50,000 integers to a plain `ArrayList` from a parallel stream. Do it **three times** and print the resulting size each run.

Guard it with a try/catch, because losing elements is not the only failure mode.

Then produce the correct count two ways: with `.toList()` and with `.count()`.

Explain It

Write a comment covering:

1. **The mechanism.** `ArrayList.add` is more than one step. Name the steps and say what two threads interleaving can do to each other.
2. **Why `collect` is immune.** What does it do differently with the containers?
3. **Why adding `synchronized` would be the wrong fix**, even though it would be correct.

Part Four: Laziness and Single Use

Build a pipeline with a `peek` in it but no terminal operation, and show that nothing runs.

Add `findFirst`, and show from the peek output that it **stopped early** rather than examining all ten entries.

Then call a second terminal operation on the same stream variable, catch what happens, and print the message. Finish by getting the same answer correctly from the source.

Acceptance Criteria

- ☐ Runs with `java LogAnalysis.java`
- ☐ All five part-one results come from single pipelines
- ☐ Key order is predictable, with a comment justifying the map type

- ☐ At least three downstream collectors are used
- ☐ A comment states what `partitioningBy` guarantees
- ☐ The `toMap` duplicate-key failure is triggered and then fixed
- ☐ All three gatherer operations produce correct output
- ☐ The comment explains why `map` cannot do any of the three
- ☐ Three parallel runs print their sizes, guarded against throwing
- ☐ The correct count is shown via `.toList()` and `.count()`
- ☐ The race explanation covers mechanism, why `collect` is safe, and why a lock is the wrong fix
- ☐ The peek output proves both laziness and short-circuiting
- ☐ Stream reuse is caught and its message printed

Stretch

Replace the parallel stream in part three with `Gatherers.mapConcurrent(4, ...)`.

Then write a comment comparing the two. Which one bounds its concurrency? Which preserves order? Which one would you use for work that calls a network service, and what does Module 14 say about why the other is a bad idea there?

Hint, if your deltas come out one element too long

`windowSliding(2)` over ten elements yields nine windows, not ten. There are nine gaps between ten readings. If you got ten, check whether you are windowing or mapping.

Module 15: Optional

`Optional` exists for one job: **a return type for a method that may legitimately find nothing**. Used for that, it is excellent. Used anywhere else it is usually worse than the null it replaced, because it adds ceremony without adding safety.

This module is mostly about the difference.

What You'll Learn

- What `Optional` actually buys you, and who it buys it for
- The operations that never ask "is it there"
- `map` versus `flatMap`, the same rule as streams
- Why `orElse` and `orElseGet` are not interchangeable
- Five anti-patterns, including the one that reintroduces the crash

Coming From Another Language

	Elsewhere	Java
Absent value	<code>None</code> , <code>null</code> , <code>undefined</code>	<code>Optional.empty()</code>
Safe navigation	<code>?.</code>	<code>.map(...)</code>
Default	<code>x ?? y</code> , <code>or</code>	<code>.orElse(y)</code>
Compiler-enforced	Rust <code>Option</code> , Kotlin <code>T?</code>	no

That last row matters. In Rust or Kotlin the compiler stops you ignoring absence. Java's `Optional` is an ordinary class, and nothing prevents `.get()`. It is a **convention supported by a type**, not a guarantee.

The Lesson

What it is for

Compare two signatures:

```
User findUser(int id)           // might return null. Read the source to find out.
Optional<User> findUser(int id) // says so, in the type.
```

That is the entire value proposition. It is about **the caller's obligation being visible**, not about avoiding null internally.

`Optional.of` rejects null on purpose. Writing it asserts the value is there, so an NPE means your assertion was wrong. Use `ofNullable` when you genuinely do not know.

Using it without unwrapping

The idiomatic style never asks whether a value is present. It describes what to do with it and what to fall back to. From `OptionalBasics.java`:

```
findUser(1).map(User::email).orElse("no email")
findUser(1).filter(u -> u.name().startsWith("a")).map(User::name).orElse("no match")
findUser(9).or(() -> findUser(1))           // an alternative Optional, so it chains
```

`ifPresentOrElse` handles the two-branch case. `Optional.stream()` yields zero or one element, so `flatMap(Optional::stream)` drops the misses inside a pipeline without a separate filter.

map versus flatMap

Same rule as streams. If your function returns a plain value, use `map`. If it already returns an `Optional`, `map` gives you a nested one:

```
map with an Optional-returning fn: Optional[Optional[ada@example.com]]
flatMap:                           Optional[ada@example.com]
```

When a chain suddenly has nested `Optional`s in it, you wanted `flatMap`.

orElse versus orElseGet

The single most common `Optional` bug. From `OrElseVsOrElseGet.java`, both calls made on a **present** `Optional`:

```
calling orElse:
>>> expensiveDefault ran, called from orElse
result: the real value
calling orElseGet:
result: the real value
```

Both returned the real value. Only one computed the default it then threw away.

	Takes	Evaluated
<code>orElse(T other)</code>	a value	always, because Java evaluates arguments before the call
<code>orElseGet(Supplier<T>)</code>	a lambda	only when empty

It stops being a performance note the moment the default has side effects:

```
after orElse on a PRESENT optional, counter = 1
after orElseGet on the same,               counter = 0
```

Now it is a correctness bug. A default that inserts a row, sends a request or logs will do so when it should not.

The rule: `orElse("")` and `orElse(0)` are fine. `orElse(buildDefault())` and `orElse(repo.findDefault())` are bugs.

`orElseThrow(supplier)` is the honest way to say "absent here is a bug", and it lets you write a message explaining what was expected.

Five anti-patterns

From `AntiPatterns.java`:

1. **`isPresent()` then `get()`**. This is an if-null check with more typing. Nearly every such pair becomes `map`, `filter`, `ifPresent`, `ifPresentOrElse`, `orElse` or `orElseGet`.
2. **`Optional` as a field**. It is not `Serializable`, it costs an object per instance, and a field can simply be null. Keep the field plain and return an `Optional` from the **accessor**.
3. **`Optional` as a parameter**. Every caller must now write `Optional.of(x)` or `Optional.empty()`, and they can still pass null, so you gained nothing. Use an overload.
4. **`Optional<List<T>>`**. The caller now has three cases: absent, present-and-empty, present-and-populated. An empty list already means nothing here.
5. **`Optional` in a hot loop**. Every one is an allocation. This is one of the few places where that argument genuinely holds.

The crash it was meant to prevent

`UnguardedGet.java`:

```
Exception in thread "main" java.util.NoSuchElementException: No value present
```

This is worse than a `NullPointerException`, not better. Since Java 14 an NPE names the exact expression that was null. "No value present" names nothing. And the code *looks* safe, because `Optional` appears in the type, so a reviewer skims past it.

`get()` has no use that `orElseThrow()` does not cover better. JDK 10 added the no-argument `orElseThrow()` for exactly this reason: identical behaviour, but it reads as a deliberate choice rather than an oversight.

Run It

```
java modules/15-optional/examples/OptionalBasics.java
java modules/15-optional/examples/OrElseVsOrElseGet.java
java modules/15-optional/examples/AntiPatterns.java

# Fails on purpose.
java modules/15-optional/examples/UnguardedGet.java

./scripts/verify-examples.sh modules/15-optional
```

Common Mistakes

`orElse` with a computed default. It runs every time, present or not.

`isPresent()` followed by `get()`. You have written a null check wearing a costume.

`Optional` fields and parameters. Neither was the intent, and both add noise.

Returning `Optional<List<T>>`. Return an empty list.

Calling `get()` at all. Use `orElseThrow` with a message.

Returning `null` from a method that returns `Optional`. It happens, and it is the worst of both worlds: the caller trusts the type and gets an NPE anyway.

Key Takeaways

- `Optional` is a return type for lookups, not a general null wrapper.
- It is a convention, not a guarantee. Java's compiler does not enforce it.
- Describe what to do, never ask whether it is there.
- `flatMap` when the function already returns an `Optional`.
- `orElse` always evaluates its argument; `orElseGet` does not. With side effects, that is a correctness bug.
- Never `get()`. `orElseThrow` with a message says what you expected.

Homework

[homework/README.md](#)

Build a lookup chain, prove the `orElse` trap with a counter, and fix four anti-patterns. Reference solution in [solutions/15-optional/](#).

Homework 15: A Contact Book

Build a lookup API, prove the `orElse` trap with a counter you can watch, and fix four anti-patterns.

The Data

```
record Contact(String name, String email, String phoneOrNull) { }

Map.of(
    "ada",    new Contact("ada",    "ada@example.com",    "555-0100"),
    "grace",  new Contact("grace",  "grace@example.com", null),
    "alan",   new Contact("alan",   "alan@example.com",   "555-0199"));
```

Note that `grace` has no phone, and the field is called `phoneOrNull` for a reason you will implement in part three.

Part One: Lookups Without Unwrapping

Create `ContactBook.java` in the classic form.

Write `find(String name)` returning `Optional<Contact>`.

Then, for `ada`, `grace` and `nobody`, print each contact's phone number and their email domain. Missing values get a readable fallback.

Rules:

- No `isPresent` anywhere in this part
- No `get` anywhere in the whole file
- The phone lookup must need `flatMap`, not `map`. Show both and say why the `map` version does not even compile into an `orElse(String)`.

Then chain three fallbacks with `or()` and explain in a comment why `orElse` could not have been used to build that chain.

Finish by mapping a list of names through `find` and collecting only the hits, using `Optional::stream` rather than a filter.

Part Two: Prove The Trap

Add a `static int defaultLookups` counter and a `lookupDefaultEmail()` method that increments it.

Then run three cases and print the counter after each:

Case	Expect
<code>orElse(lookupDefaultEmail())</code> on a present Optional	?
<code>orElseGet(ContactBook::lookupDefaultEmail)</code> on the same	?
<code>orElseGet(...)</code> on an empty Optional	?

Write down what you predicted before running it, then what actually happened.

Explain It

In a comment, in your own words:

1. Why the argument to `orElse` runs at all when the value is present
2. Why `orElseGet` does not
3. **Why this is a correctness bug and not just a performance note.** Your answer must refer to the counter, and say what it would be in real code.

Part Three: Fix Four Anti-Patterns

Show the fixed form of each, with a comment naming what was wrong:

1. `isPresent` then `get` — replace with a single chained call
2. `Optional` as a field — keep `phoneOrNull` plain, return `Optional` from the accessor, and say what an `Optional` field would have cost
3. `Optional` as a parameter — write a method taking an optional label using an **overload** instead
4. `Optional<List<T>>` — write a search returning an empty list, and say how many states the wrapped version would have forced on the caller

Then replace a `get()` with `orElseThrow` and a message. Print both that message and what `get()` would have said, and comment on the difference.

Part Four: What It Does Not Protect You From

Write a method declared to return `Optional<String>` that returns `null`.

Call it, print what came back, then call any method on the result and catch what happens.

In a comment, explain why this is worse than a plain nullable return, and why it proves `Optional` is a convention rather than a guarantee. Compare with Kotlin or Rust, where the compiler is involved.

Acceptance Criteria

- ☐ Runs with `java ContactBook.java`

- ☐ `get()` appears nowhere in the file
- ☐ `isPresent` appears nowhere except where you are demonstrating the anti-pattern
- ☐ The phone lookup uses `flatMap`, with both forms shown
- ☐ `or()` chains three fallbacks, with a comment on why `orElse` cannot
- ☐ `Optional::stream` is used to drop misses
- ☐ The counter prints three times with the three different results
- ☐ The explanation covers argument evaluation and why it is a correctness bug
- ☐ All four anti-patterns are shown fixed, each with a comment
- ☐ `orElseThrow`'s message is compared with `get()`'s
- ☐ The null-returning `Optional` method is demonstrated and explained

Stretch

Write `Optional<Contact> findAny(String... names)` returning the first name that exists, evaluating no more lookups than necessary.

Then write a comment on whether your version is lazy, and if you used a stream, what `findFirst` did that a loop over `or()` would not have.

Hint, if part one's `flatMap` point is not landing

Try writing `find("ada").map(Contact::phone).orElse("none")` and read the compiler error. The type is `Optional<Optional<String>>`, and `orElse` on that wants an `Optional<String>`, not a `String`. The error is the lesson.

Module 16: Files and IO

Two types do most of the work. `Path` is a **name**, and every method on it is pure string manipulation that touches no disk. `Files` is what actually reads, writes and asks questions of the filesystem.

Keeping those separate in your head prevents a surprising number of bugs, starting with the security one.

What You'll Learn

- `Path` as pure name manipulation, and the traversal bug it prevents
- The one-line `Files` methods for reading and writing
- Why `Files.lines` needs closing and `readString` does not
- What `java.io.File` gets wrong, with the failure shown side by side
- The exception family that names the file and the reason

Coming From Another Language

	Python	Java
Path object	<code>pathlib.Path</code>	<code>java.nio.file.Path</code>
Join	<code>/</code> operator	<code>.resolve(...)</code>
Read all	<code>path.read_text()</code>	<code>Files.readString(path)</code>
Read lines lazily	<code>with open(...) as f</code>	<code>Files.lines(path)</code> , must close
Missing file	<code>FileNotFoundException</code>	<code>NoSuchFileException</code>
Default encoding	UTF-8	UTF-8 since Java 18

`pathlib` and `java.nio.file` are close relatives, so most of this will feel familiar. The closing requirement on `Files.lines` is the one to watch.

The Lesson

Path is a name, not a file

From `PathBasics.java`. `resolve` appends, and gets the separator right on every platform:

```
Path.of("/var/data").resolve("reports/q3.csv")    // /var/data/reports/q3.csv
```

An absolute argument replaces the base entirely:

```
Path.of("/var/data").resolve("/etc/passwd")    // /etc/passwd
```

That behaviour is how unvalidated user input becomes a path traversal bug. The defence is to normalise and then check where you landed:

```
Path attempted = uploads.resolve(userSupplied).normalize();
attempted.startsWith(uploads)    // false for ../../etc/passwd
```

Use `Path.startsWith`, not `String.startsWith`. It compares name elements rather than characters, so it cannot be fooled:

```
String.startsWith would have said: true    // "/srv/uploads-evil/x"
Path.startsWith says:                  false
```

`normalize()` is pure and does **not** follow symlinks, so removing `b/..` can change which file you reach if `b` is a link. `toRealPath()` is the one that touches the disk, resolves links, and throws if the file is absent.

`Path.equals` is lexical. `Files.isSameFile` asks the filesystem.

Reading and writing

From `ReadingAndWriting.java`:

```
Files.writeString(path, "content");           // create or
        truncate
Files.writeString(path, "more\n", StandardOpenOption.APPEND); // append
String whole = Files.readString(path);
List<String> lines = Files.readAllLines(path);
```

Since Java 18 the default charset is UTF-8 everywhere. Before that the same code read different bytes on different machines, which produced a long tail of encoding bugs.

Useful queries: `exists`, `isRegularFile`, `isDirectory`, `size`, `isReadable`.

A note on `exists`. It answers a question about the past. The file can vanish before you act on the answer. Prefer attempting the operation and handling the exception over checking first. That race is called TOCTOU and it is a real source of bugs.

`Files.createDirectories` creates every missing parent. `copy`, `move` and `deleteIfExists` do what they say, and the copy options control overwriting.

Streaming, and the closing rule

`readString` and `readAllLines` load the **entire file**. Fine for a config file, fatal for a log.

From `StreamingLargeFiles.java`:

```
try (var lines = Files.lines(log)) {  
    long errors = lines.filter(l -> l.startsWith("ERROR")).count();  
}
```

`Files.lines`, `Files.list`, `Files.walk` and `Files.find` all hold an open file handle. The stream is the only thing that can close it, so try-with-resources is not optional. This is the most commonly leaked resource in Java file code, and on a busy server it means running out of descriptors.

Memory stays at one line regardless of file size, and the pipeline can short-circuit, so finding the first match reads only as far as it needs to.

One wrinkle when writing from inside a stream:

```
lines.forEach(line -> {  
    try { out.write(line); } catch (IOException e) { throw new UncheckedIOException(e); }  
});
```

A lambda cannot throw a checked exception. That is Module 10's complaint about checked exceptions and lambdas, met in the wild, and wrapping is the usual fix.

Why not java.io.File

From `WhyNotLegacyFile.java`, the same failure twice:

```
File.delete() returned: false  
why did it fail? no permission? not there? locked?  
the API does not say, and cannot be made to.  
  
Files.delete() threw NoSuchFileException  
naming the file: /var/.../does-not-exist.txt
```

That is the central complaint. Every `File` method that can fail returns `false`, so error handling becomes guesswork.

Three more: `File` cannot handle symbolic links, cannot read file attributes in one call, and `listFiles()` returns an **array**, so a directory with a million entries becomes a million-element array.

`path.toFile()` and `file.toPath()` convert freely. **Convert at the boundary with old APIs, and use `Path` everywhere inside your own code.**

The exception family

`MissingFile.java` throws `NoSuchFileException`, which carries the path as **structured data**:

```
catch (NoSuchFileException e) {  
    log.warn("missing config at {}", e.getFile());  
}
```

Exception	Meaning
<code>NoSuchFileException</code>	the path does not exist
<code>FileAlreadyExistsException</code>	it does, and you asked to create it
<code>AccessDeniedException</code>	permissions
<code>DirectoryNotEmptyException</code>	delete on a non-empty directory
<code>NotDirectoryException</code>	you treated a file as a directory

All are `IOException` subclasses. Catch the specific one when you can react differently, and `IOException` when you cannot.

Run It

```
java modules/16-files-and-io/examples/PathBasics.java  
java modules/16-files-and-io/examples/ReadingAndWriting.java  
java modules/16-files-and-io/examples/StreamingLargeFiles.java  
java modules/16-files-and-io/examples/WhyNotLegacyFile.java  
  
# Fails on purpose.  
java modules/16-files-and-io/examples/MissingFile.java  
  
./scripts/verify-examples.sh modules/16-files-and-io
```

Every example works in a temporary directory and deletes it before exiting, so running them leaves nothing behind.

Common Mistakes

Not closing `Files.lines` or `Files.walk`. A leaked file descriptor per call.

`readAllLines` on a large file. Use `Files.lines`.

Building paths with string concatenation. Use `resolve`, and normalise anything derived from user input.

`String.startsWith` for a containment check on paths. `/srv/uploads-evil` passes.
`Path.startsWith` does not.

Checking `exists` before every operation. The answer is stale the moment you have it.

Catching `Exception` around file work. The specific subclasses tell you whether to retry, create the file, or give up.

New code using `java.io.File`. Silent booleans instead of reasons.

Key Takeaways

- `Path` is a name; `Files` touches the disk. Path methods are pure.
- `resolve` with an absolute argument replaces the base, which is a traversal bug waiting to happen. Normalise and check with `Path.startsWith`.
- `readString` loads everything; `Files.lines` does not and must be closed.
- Every directory-walking method returns a closeable stream.
- Default charset has been UTF-8 since Java 18, so encoding no longer varies by machine.
- `Files` throws exceptions that name the file and the reason. `File` returns `false`.

Homework

[homework/README.md](#)

Build a log analyser that streams, then write a path validator that resists traversal. Reference solution in [solutions/16-files-and-io/](#).

Homework 16: A Log Tool

Build something that reads a file properly, writes one safely, and refuses a path traversal attempt.

Every file you create must be deleted before the program exits. Work inside

`Files.createTempDirectory(...)` and clean it up in a `finally`. Leaving files behind fails this assignment.

Part One: Stream, Do Not Slurp

Create `LogTool.java` in the classic form.

Generate a log of 1,000 lines shaped like:

```
ERROR request=0 ms=10
INFO request=1 ms=47
```

Then answer four questions, **each with its own** `Files.lines` **call**:

1. How many lines of each level, in alphabetical order of level
2. The first `ERROR` line
3. The line with the highest `ms=` value
4. Summary statistics over all the `ms=` values

Every one of those streams must be closed. Write a comment saying what would leak if you omitted the `try-with-resources`, and how many times over in this method.

For question 2, add a comment on what `findFirst` does that `readAllLines` could never do, whatever the file size.

Part Two: Write A Filtered Copy

Write only the `ERROR` lines to a second file, reading and writing **in one try-with-resources with two resources**.

You will hit a problem: `forEach` takes a `Consumer`, and `write` throws a checked `IOException`. Solve it, and write a comment naming the general problem and why the standard fix preserves the original failure.

Then:

- Append a trailing line, and comment on why the append needed an explicit option
- Create a nested directory and `move` the file into it
- Show the original is gone

Part Three: The Path Validator

Create an `uploads` directory. Write a method that takes a user-supplied filename and decides whether it is allowed.

Test it with all five of these:

Input	Should be
<code>photo.jpg</code>	allowed
<code>nested/../photo.jpg</code>	allowed
<code>../../etc/passwd</code>	rejected
<code>/etc/passwd</code>	rejected
<code>..</code>	rejected

The fourth one is the interesting case. Work out what `resolve` does with an absolute argument, and make sure your check still catches it.

Then Prove The String Trap

Create a sibling directory called `uploads-evil`, and a file inside it.

Print whether that file is "inside uploads" according to `String.startsWith` and according to `Path.startsWith`. They disagree.

Write a comment explaining why, what `Path` compares that `String` does not, and why this is a real bypass rather than a curiosity.

Part Four: Exceptions That Say Something

Trigger and catch each of these, printing the filename from the exception itself rather than from a variable you already had:

1. `NoSuchFileException`
2. `FileAlreadyExistsException`
3. `DirectoryNotEmptyException`

Then call `delete()` on a missing file through the **legacy** `java.io.File` API and print what it returns.

Write a comment covering:

- What the legacy return value tells you, and what it cannot
- **Why checking `exists()` first is not a fix.** Name the race and say why the exception can still arrive from a line that looks guarded

Acceptance Criteria

- ☐ Runs with `java LogTool.java`
- ☐ The temp directory is deleted, and the program prints proof
- ☐ Four separate `Files.lines` calls, each in try-with-resources
- ☐ A comment states what leaks without them, and how many times
- ☐ A comment explains what `findFirst` gains over `readAllLines`
- ☐ The filtered copy uses two resources in one try-with-resources
- ☐ The checked-exception-in-a-lambda problem is solved and explained
- ☐ The append uses an explicit open option, with a comment on why
- ☐ All five validator cases produce the correct verdict
- ☐ `String.startsWith` and `Path.startsWith` are shown disagreeing
- ☐ The explanation says what `Path` compares instead
- ☐ All three file exceptions are caught and their filename printed from the exception
- ☐ A comment names the TOCTOU race and why `exists()` does not fix it

Stretch

Make the filtered copy **atomic**: write to a temporary file in the same directory, then move it into place with `StandardCopyOption.ATOMIC_MOVE`.

Then write a comment on what a reader of the destination file could observe during a non-atomic write, and why "same directory" matters for the move to be atomic at all.

Hint, if `/etc/passwd` slips through

Check what `base.resolve("/etc/passwd")` returns before you normalise. An absolute argument does not append to the base, it replaces it entirely, so the result has nothing to do with your uploads directory. Your containment check is what has to catch that, which is exactly why the check exists rather than trusting `resolve`.

Module 17: Standard Library Tour

Four corners of the library you will use constantly, chosen because each has a trap that has cost people real time.

Every example here is **reproducible**: fixed clocks, fixed dates, seeded random generators. That is partly so the lesson is stable, and partly because making time and randomness reproducible is itself the technique you need for testing.

What You'll Learn

- The String methods worth memorising, and `formatted`
- `java.time`: which type to pick, and why its arithmetic clamps
- Daylight saving, where date handling actually breaks
- Five `Math` behaviours that have produced real bugs
- Reproducible random numbers, and when never to use them

Coming From Another Language

	Python	Java
Format	f-strings	<code>"%s".formatted(x)</code>
Date	<code>datetime</code>	<code>LocalDate</code> , <code>LocalDateTime</code> , <code>ZonedDateTime</code>
Timezone-aware	opt-in via <code>tzinfo</code>	opt-in via the type you choose
Duration	<code>timedelta</code>	<code>Duration</code> and <code>Period</code> , and they differ
Random	<code>random.seed(42)</code>	<code>RandomGeneratorFactory.of(...).create(42)</code>

The row that matters: Java makes you pick a type that either carries a zone or does not.

`LocalDateTime` is **not** a moment in time, and treating it as one is the most common `java.time` bug.

The Lesson

Strings

From `Strings.java`:

```

" x ".strip()           // Unicode-aware; trim() is not
"a,b,c".split(",")      // takes a REGEX, not a literal
String.join("-", "2026", "09")
"ab".repeat(3)
"%-10s|%8.2f|%,d".formatted("name", 3.14159, 1_234_567)

```

Two traps:

- **split** takes a regex. Splitting on `.` or `|` without escaping gives the wrong answer, and it is a classic one-hour bug.
- **compareTo** returns a negative number, zero, or a positive one. Not `-1, 0, 1`. Code that assumes otherwise is wrong.

Format conversions worth memorising: `%s` anything, `%d` integer, `%.2f` two decimal places, `%n` platform newline, `%,d` thousands separator, `%%` a literal percent. `%10s` right-aligns, `%-10s` left-aligns.

java.time: choosing the type

Type	Holds	Use for
<code>LocalDate</code>	date only	a birthday
<code>LocalTime</code>	time only	opening hours
<code>LocalDateTime</code>	both, no zone	a wall clock reading
<code>ZonedDateTime</code>	a real moment somewhere	a meeting
<code>Instant</code>	a point on the UTC timeline	a timestamp
<code>Duration</code>	hours, seconds	elapsed time
<code>Period</code>	years, months, days	calendar amounts

LocalDateTime has no zone. `2026-09-23T10:15` is a moment in some unstated place. Storing one and assuming UTC is the bug this module most wants you to avoid.

Every `now()` takes an optional `Clock`. Code calling `now()` with no argument cannot be tested for "what happens on 29 February". Take a `Clock` as a dependency and the problem disappears.

Arithmetic that clamps

From `DateAndTime.java`:

```

31 Jan + 1 month: 2026-02-28
29 Feb + 1 year:  2025-02-28
and back again:   2026-01-28
so +1 month then -1 month is NOT the identity.

```

Both clamp to the last valid day rather than rolling forward. That is deliberate, and it is **not reversible**. If your billing code adds a month and subtracts one, it does not land where it started.

`Duration` is time-based and exact. `Period` is calendar-aware, so one month is 28, 29, 30 or 31 days depending on where you apply it. That is why both exist.

Always pass a `Locale` to `ofPattern`. Without one it uses the machine default, so the same code formats differently on a server abroad. Same class of bug as the default charset before Java 18.

Daylight saving

From `TimeZonesAndDst.java`:

```
--- the spring forward gap ---
01:30 EST:      2026-03-08T01:30-05:00[America/New_York]
plus one hour:  2026-03-08T03:30-04:00[America/New_York]
02:30 does not exist; atZone gives: 2026-03-08T03:30-04:00[America/New_York]

--- the autumn fall-back overlap ---
first  (EDT): 2026-11-01T01:30-04:00[America/New_York]
second (EST): 2026-11-01T01:30-05:00[America/New_York]
```

Three things happen there:

1. Adding one hour to 01:30 gives **03:30**, because 02:00 never existed. The arithmetic is right; one real hour passed.
2. A local time inside the gap does not exist. Java does not throw, it moves forward.
3. In autumn, 01:30 happens **twice**. `atZone` picks the earlier offset. If you need the later one you must say so, and silently assuming either is how bookings get double-counted.

And the distinction that matters most:

`plusDays(1)` keeps the wall-clock time. `plusHours(24)` adds 24 real hours. On two days a year they give different answers. "Same time tomorrow" and "twenty-four hours from now" are different operations.

The rules that keep you safe: store an `Instant` or UTC and convert only for display, use a region `ZoneId` like `Europe/London` rather than a fixed offset, and never store a `LocalDateTime` for a real moment.

Math

From `MathSurprises.java`:

```

Math.abs(MIN_VALUE):    -2147483648
is it negative?         true
  abs(MIN_VALUE) % 7    = -2    <- a negative array index
  floorMod(MIN_VALUE, 7) = 5    <- always in range

-7 / 2                  = -3
Math.floorDiv           = -4

Math.round(2.5)         = 3
Math.round(-2.5)        = -2

MAX_VALUE + 1           = -2147483648
Math.addExact throws: integer overflow

```

abs can return a negative number. The `int` range is asymmetric, so negating the minimum wraps back to itself. This matters because `Math.abs(hash) % buckets` is a common idiom that produces a **negative index**, as the output above shows. Use `Math.floorMod(hash, buckets)` instead, which is always in range, or `Math.absExact` which throws.

Note the example uses 7 buckets, not 8. `MIN_VALUE` is divisible by 8, so with 8 both forms return `0` and the bug stays hidden. Choosing an example where the failure actually appears is part of the work.

Integer division truncates toward zero; floorDiv rounds toward negative infinity. They differ for negatives.

round rounds half toward positive infinity, so `-2.5` becomes `-2`, not `-3`.

Overflow is silent. `addExact`, `subtractExact` and `multiplyExact` refuse to wrap. Use them wherever a wrapped value is worse than a crash, which for anything counting money or memory is always.

Random numbers

From `RandomNumbers.java`:

```
RandomGenerator rng = RandomGeneratorFactory.of("L64X128MixRandom").create(42);
```

Note the shape: the **factory** creates the generator. `RandomGenerator.of(name)` returns a generator directly and has no `create`.

Same seed, same sequence. A test that shuffles or samples needs a fixed seed, or it fails once a month for reasons nobody can reproduce. Take the generator as a parameter, exactly as you would take a `Clock`.

`nextInt(bound)` excludes the bound; `nextInt(low, high)` includes low and excludes high.

Never use these for anything security-relevant. They are pseudo-random and predictable given enough output. Use `java.security.SecureRandom` for tokens, session ids and keys.

Do not share one generator across threads. `java.util.Random` is synchronised and becomes a bottleneck; `ThreadLocalRandom.current()` gives each thread its own.

Run It

```
java modules/17-standard-library-tour/examples/Strings.java
java modules/17-standard-library-tour/examples/DateAndTime.java
java modules/17-standard-library-tour/examples/TimeZonesAndDst.java
java modules/17-standard-library-tour/examples/MathSurprises.java
java modules/17-standard-library-tour/examples/RandomNumbers.java

./scripts/verify-examples.sh modules/17-standard-library-tour
```

Run any of them twice and the output is identical, because none depends on the current time or an unseeded generator.

Common Mistakes

Storing a `LocalDateTime` for a real moment. It has no zone. Store an `Instant`.

Using a fixed offset instead of a region id. `+01:00` is wrong for half the year in London.

Calling `now()` with no `Clock`. The code becomes untestable for any date-dependent branch.

`ofPattern` without a `Locale`. Different output on a server abroad.

`Math.abs(hash) % n` for a bucket index. Negative once in four billion. Use `floorMod`.

Comparing doubles with `==`. Compare within a tolerance, or avoid floating point for money.

An unseeded generator in a test. A flake once a month that nobody can reproduce.

`split(".")`. That is a regex matching any character.

Key Takeaways

- **Pick the `java.time` type by whether it needs a zone.** `LocalDateTime` does not carry one.
- **Take a `Clock` and a `RandomGenerator` as dependencies,** so time and randomness become testable.
- **Date arithmetic clamps and is not reversible.**
- **`plusDays(1)` and `plusHours(24)` differ** on daylight saving days.
- **`abs` can be negative, division truncates, `round` is asymmetric, overflow is silent.** The `Exact` methods throw instead.
- **Seed generators for tests; use `SecureRandom` for anything that matters.**

Homework

[homework/README.md](#)

Build a scheduling tool that survives daylight saving, then reproduce four `Math` traps. Reference solution in [solutions/17-standard-library-tour/](#).

Homework 17: A Scheduler That Survives March

Build something that handles dates correctly, then reproduce four `Math` traps.

Every line your program prints must be identical on every run, and identical regardless of which time zone the machine is set to. That means a fixed `Clock`, fixed dates, and a seeded generator. Output that varies fails this assignment.

Part One: Monthly Billing

Create `Scheduler.java` in the classic form.

A customer signs up on **31 January 2026**. Read that date from a fixed `Clock`, not from `LocalDate.now()` with no argument.

Print their next four billing dates **two ways**:

1. **Naively**, by repeatedly adding one month to the previous result
2. **Anchored**, by always computing from the original signup date

The two disagree from the second month onward.

Explain It

Write a comment covering:

- What `plusMonths` does to 31 January, and why that part is correct
- Why the naive version's error is **permanent** rather than a one-off
- What the anchored version does differently, in one sentence

Part Two: A Meeting Across Daylight Saving

Schedule a 09:00 New York meeting on **Friday 6 March 2026**, then move it three days forward two ways:

- `plusDays(3)`
- `plusHours(72)`

Print both, plus the real hours elapsed between the original and the `plusDays` result. It will not be 72.

Then show what London sees for both moments.

Three More Cases

1. **The ambiguous hour.** 01:30 on 1 November 2026 in New York happens twice. Print both, and how far apart they are.
2. **The nonexistent hour.** 02:30 on 8 March 2026 never happens. Print what `atZone` does with it.
3. **The shifting gap.** London and New York change on different dates. Say what that means for anyone who hard-codes a five-hour difference.

Explain It

A comment covering why `plusDays` and `plusHours` differ, which question each one answers, and which you would use for a recurring meeting versus a timeout.

Part Three: Four Math Traps

Reproduce and explain each:

1. `Math.abs(Integer.MIN_VALUE)` and the **negative bucket index** it causes. Show `abs(hash) % buckets` alongside `floorMod(hash, buckets)`.
2. `-7 / 2` versus `Math.floorDiv(-7, 2)`
3. `Math.round(2.5)` versus `Math.round(-2.5)`
4. Silent overflow, and what `Math.addExact` does instead

Careful with trap 1. Pick your bucket count so the bug actually shows. Some values make both forms agree and hide it entirely, and working out which is part of the exercise.

Part Four: Reproducible Sampling and Formatting

Write a method that picks N names at random from a list, taking a **seed** as a parameter.

Call it twice with the same seed and once with a different one. Show the first two agree.

Write a comment on what an unseeded generator would do to a test, and name one thing you must never use these generators for.

Finish with a small formatted report using `formatted`, with:

- A date formatted through an explicit `Locale`
- A left-aligned label column
- A thousands-separated number

Say in a comment why the `Locale` is not optional.

Acceptance Criteria

- ☐ Runs with `java Scheduler.java`
- ☐ Running it twice produces byte-identical output
- ☐ A fixed `Clock` is used; no bare `now()` anywhere
- ☐ Both billing schedules are printed and disagree from month two
- ☐ A comment explains why the naive error is permanent
- ☐ `plusDays(3)` and `plusHours(72)` are shown giving different results
- ☐ Real elapsed hours are printed and are not 72
- ☐ The ambiguous hour is shown twice, an hour apart
- ☐ The nonexistent hour is shown being moved forward
- ☐ All four Math traps are reproduced with visible output
- ☐ Trap 1's bucket count actually exposes the negative index
- ☐ Sampling is seeded, and two same-seed runs agree
- ☐ `ofPattern` is given an explicit `Locale`, with a comment on why

Stretch

Write `List<ZonedDateTime> weeklyMeetings(ZonedDateTime first, int count)` returning a weekly recurrence that stays at the same **local** time across a daylight saving transition.

Then write a comment on what would go wrong if you stored each occurrence as an `Instant` computed by adding seven days' worth of seconds, and what you would store instead for a meeting a year from now whose zone rules might change before it happens.

Hint, if your output changes between runs

Search for `now()` with no argument, and for any generator created without a seed. Those are the only two sources of variation here, and both have a form that takes the thing they depend on as a parameter. That is not a trick for homework; it is how time-dependent code is made testable in general.

Module 18: Concurrency Basics

Two things go wrong when threads share data, and they are not the same thing.

Races are about interleaving: two threads interfere mid-operation. **Visibility** is about one thread never seeing another's write at all. This module demonstrates both, then covers the tools that fix each.

What You'll Learn

- Threads, interruption, and why you should not create threads yourself
- Why `count++` loses increments, shown with hundreds of thousands lost
- `synchronized`, atomics and `LongAdder`, and what each costs
- The visibility problem, where a loop never exits
- `ExecutorService`, and the failure mode that stays silent

Coming From Another Language

	Python	Java
Real parallelism	blocked by the GIL	yes, genuinely
Race on <code>x += 1</code>	possible	certain , at scale
Lock	<code>threading.Lock</code>	<code>synchronized</code> , <code>ReentrantLock</code>
Atomic counter	no built-in	<code>AtomicInteger</code>
Thread pool	<code>ThreadPoolExecutor</code>	<code>ExecutorService</code>
Visibility keyword	none needed	<code>volatile</code>

Python's global interpreter lock hides most of this. Java runs threads on real cores in parallel, so every hazard here is one you will actually meet.

The Lesson

Threads directly

```
Thread worker = new Thread(() -> doWork(), "worker-1");
worker.start();
worker.join();
```

`start()` and `run()` are not the same. `start()` creates a thread; `run()` just calls a method on the current one. The mistake looks like it works because the output is identical when there is nothing to

interleave.

Interruption is a request, not a kill. Catching `InterruptedException` clears the flag, so restore it:

```
catch (InterruptedException e) {  
    Thread.currentThread().interrupt();  
}
```

`Thread.stop()` was deprecated in 2000 and has been removed, because killing a thread mid-operation leaves whatever it was mutating in an unknown state.

Why not to create threads yourself: a platform thread costs about 1MB of stack, there is no bound on how many exist, and nothing collects results or propagates failures.

The race

From `TheRaceCondition.java`, four threads incrementing 200,000 times each:

```
expected total: 800000  
actual total:   242657  
-> 557343 increments were LOST
```

`count++` is three operations: read, add, write. Two threads can both read `100` before either writes `101`, and one increment vanishes.

The same shape appears everywhere: `x += 1`, `list.add(item)`, `balance -= amount`, and every check-then-act pair:

```
if (!map.containsKey(k)) { map.put(k, v); }
```

Both threads can pass the check before either puts. That is why `Map` has `putIfAbsent` as a single atomic operation.

Three fixes

From `FixingTheRace.java`:

unsafe ++	796131	WRONG
synchronized	800000	correct
AtomicInteger	800000	correct
LongAdder	800000	correct

`synchronized` gives mutual exclusion *and* a happens-before edge: everything a thread did before releasing the lock is visible to the next thread that acquires it. **That second guarantee is the one people forget**, and it is why `synchronized` fixes visibility too. The cost is that blocked threads queue.

Atomics use a compare-and-swap instruction: write back only if nothing changed, otherwise retry. No blocking, but under contention the retries are the cost.

`LongAdder` keeps several internal cells so threads mostly do not touch the same memory. Faster than `AtomicLong` under heavy write contention, slower to read. Use it for metrics; use `AtomicLong` when you need the value after every update.

And the fourth option, usually best: do not share the state. Module 14 made the same point about parallel streams, where the fix was `collect` giving each thread its own container. A race you cannot have is cheaper than one you synchronise.

What not to lock on:

Don't	Because
<code>synchronized (this)</code>	anyone with a reference can lock you out
<code>synchronized (SomeClass.class)</code>	the same, globally
a <code>String</code> literal	literals are pooled and shared with unrelated code
a boxed <code>Integer</code>	values below 128 are cached

Use a `private final Object`.

Visibility

This is the one people do not expect. From `MemoryVisibility.java`, one thread spins on a flag and another sets it:

```
main set plainFlag = true
after waiting 1.5s, reader still running: true

main set volatileFlag = true
after waiting 1.5s, reader still running: false
```

The plain reader **never saw the write**. There is no interleaving here at all.

Nothing in that loop touches shared state, so the JIT may assume `plainFlag` cannot change and hoist the read out entirely, turning it into `while (true)`. That is a **legal** optimisation. The Java Memory Model only promises one thread sees another's write when a happens-before relationship connects them, and a plain field creates none.

What `volatile` does: every read goes to main memory, and it establishes happens-before, so everything written before a volatile write is visible after the matching read.

What it does not do: make `count++` atomic. Three operations remain three operations.

`volatile` is for flags and for publishing a reference. `synchronized`, atomics and locks are for compound actions.

ExecutorService

```
try (ExecutorService pool = Executors.newFixedThreadPool(3)) {  
    Future<Integer> result = pool.submit(() -> compute());  
    System.out.println(result.get());  
}
```

`ExecutorService` has been `AutoCloseable` since Java 19, so `try-with-resources` replaces the `shutdown` and `awaitTermination` pair people routinely got wrong.

A real trap, from `UsingExecutors.java`:

```
submitted, and nothing has been reported yet  
get() threw ExecutionException  
caused by: java.lang.IllegalStateException: the task failed
```

A task submitted with `submit()` that throws and whose `Future` is never inspected **fails silently**. Nothing logs, nothing crashes. Use `execute()` if you want the default handler to report it, or always inspect the `Future`.

Pool	Use
<code>newFixedThreadPool(n)</code>	bounded; the safe default for CPU work
<code>newCachedThreadPool()</code>	unbounded; a burst of tasks makes a burst of threads
<code>newSingleThreadExecutor()</code>	one at a time, in order
<code>newScheduledThreadPool(n)</code>	delayed and repeating

For IO-bound work all of these force a choice between too few threads and too many. Module 19 removes the choice.

Run It

```
java modules/18-concurrency-basics/examples/ThreadsAndRunnables.java  
java modules/18-concurrency-basics/examples/TheRaceCondition.java  
java modules/18-concurrency-basics/examples/FixingTheRace.java  
java modules/18-concurrency-basics/examples/MemoryVisibility.java  
java modules/18-concurrency-basics/examples/UsingExecutors.java  
  
./scripts/verify-examples.sh modules/18-concurrency-basics
```

Run the race example several times. The number lost changes every run, and it is never zero on a multi-core machine. That inconsistency is the lesson.

Common Mistakes

Calling `run()` instead of `start()`. No thread is created.

Assuming `volatile` makes `++` safe. It does not.

Swallowing `InterruptedException`. Restore the flag or the shutdown signal is lost.

Locking on `this`, a class, a `String` or a boxed `Integer`. All are shared more widely than you think.

Not checking a `Future`. Exceptions disappear entirely.

`newCachedThreadPool` in a server. Unbounded thread creation under load.

Check-then-act on a concurrent map. `containsKey` then `put` is two operations. Use `putIfAbsent` or `compute`.

Synchronising everything to be safe. Contention can make the parallel version slower than the sequential one, and Module 14 showed exactly that.

Key Takeaways

- `count++` is three operations, so it loses increments under contention.
- `synchronized` gives mutual exclusion and visibility. Atomics give lock-free updates. `LongAdder` wins under heavy write contention.
- Visibility is a separate hazard. A plain field can be cached or hoisted, and the reader may never see the write.
- `volatile` fixes visibility, not atomicity.
- Prefer not sharing state at all over synchronising access to it.
- Use an `ExecutorService`, close it with `try-with-resources`, and always inspect the `Future`.

Homework

[homework/README.md](#)

Build a counter four ways, break it, then fix a check-then-act race. Reference solution in [solutions/18-concurrency-basics/](#).

Homework 18: A Counter That Cannot Count

Build the same counter four ways, watch one of them fail, then fix a race that a thread-safe collection does **not** protect you from.

Your program must exit cleanly every time. Any thread that might not finish has to be a daemon, and any `join` has to have a timeout.

Part One: Four Counters

Create `BankAccount.java` in the classic form.

Define a `Counter` interface with `increment()`, `value()` and `name()`, then four implementations:

Name	Mechanism
<code>UnsafeCounter</code>	a plain <code>long count</code> and <code>count++</code>
<code>SynchronizedCounter</code>	a <code>synchronized</code> block
<code>AtomicCounter</code>	<code>AtomicLong</code>
<code>AdderCounter</code>	<code>LongAdder</code>

Run each through the same workload: 4 threads, 100,000 increments each. Print the result, whether it is correct, and how long it took.

One requirement on the synchronized version: lock on a `private final Object`, not on `this`.

Write a comment saying what could go wrong with `this`, and name two other things that are also bad to lock on and why.

Explain It

A comment covering, in your own words:

- What `count++` actually compiles to
- A concrete interleaving of two threads that loses exactly one increment
- Why the number lost **differs every run**, and why that is worse for you than a consistently wrong answer

Part Two: A Loop That Never Ends

Write a class holding two fields: a plain `boolean` and a `volatile boolean`.

For each one:

1. Start a reader thread that spins until the flag is true
2. Sleep briefly, set the flag from `main`
3. `join` with a timeout, then print whether the reader is still alive

The plain one will still be alive. The volatile one will not.

Explain It

This is the part worth getting right. Write a comment covering:

- **Why this is not a race.** How many threads write, and how many times?
- What the compiler is permitted to do to that loop, and why it is permitted
- What `volatile` changes
- **Why `volatile` would not fix part one's counter.** Name the two separate problems and say which one `volatile` solves

Part Three: Check Then Act

Use a `ConcurrentHashMap`. Have 8 threads all try to claim the same key:

```
if (!map.containsKey("winner")) {
    map.put("winner", id);
    winners.add(id);
}
```

Print how many threads believed they won. It will be more than one.

Then fix it so exactly one can win, and print that too.

The Question That Matters

The map is a `ConcurrentHashMap`, which is thread-safe. Write a comment explaining why that did **not** prevent the race, what a thread-safe collection does and does not guarantee, and why `putIfAbsent`, `compute` and `merge` exist at all if the map is already safe.

Hint on making the race actually fire: if each thread starts and finishes before the next begins, nothing collides. Two `CountDownLatch` objects will let you hold every thread at the gate and release them together.

Part Four: The Failure Nobody Hears

Submit a task to an `ExecutorService` that throws, and **never inspect the Future**. Show that nothing at all is reported.

Then submit the same task and call `get()`, catching what comes back. Print the cause.

List three ways to make sure a failure is never lost.

Acceptance Criteria

- ☐ Runs with `java BankAccount.java` and exits cleanly every time
- ☐ All four counters run the identical workload with timings
- ☐ The unsafe one is visibly wrong; the other three are exactly correct
- ☐ `synchronized` locks on a private final object, with a comment on why
- ☐ Two other bad lock targets are named with reasons
- ☐ The explanation gives a concrete losing interleaving
- ☐ The plain flag reader is still alive after the join; the volatile one is not
- ☐ Reader threads are daemons and joins have timeouts
- ☐ The explanation says why this is not a race and why `volatile` is not enough for part one
- ☐ The broken check-then-act reports more than one winner
- ☐ The fixed version reports exactly one
- ☐ A comment explains what `ConcurrentHashMap` does and does not guarantee
- ☐ The silent failure is demonstrated and then surfaced

Stretch

Replace the fixed check-then-act with `compute`, so the winning thread also gets to record a timestamp atomically alongside its id.

Then write a comment on what `compute` holds while your lambda runs, and why calling another map's method, doing IO, or acquiring a second lock inside that lambda would be a bad idea.

Hint, if the unsafe counter comes out correct

Check that the threads genuinely overlap. If your workload is small enough, or the pool is size 1, each task finishes before the next starts and there is nothing to interleave. More iterations and at least as many threads as cores will do it.

Module 19: Virtual Threads

A virtual thread is a thread the JVM schedules rather than the operating system. Same `Thread` API, same code, same debugger. What changed is the price.

A platform thread reserves about a megabyte of stack and an OS scheduling slot. A virtual thread is a heap object that grows as needed. That single change makes blocking cheap again, and this module is mostly about what follows from it.

A warning about other sources: most material written about virtual thread pinning is out of date. This module measures the current behaviour rather than repeating it.

What You'll Learn

- Mounting, unmounting, and what a carrier thread is
- The benchmark that makes the case, measured on your own machine
- What pins today, which is far less than it was
- `ScopedValue`, and why `ThreadLocal` does not scale to a million threads
- Four ways people misuse them, three from old habits

Coming From Another Language

	Go	Python	Java
Lightweight concurrency	goroutine	<code>asyncio</code> task	virtual thread
Blocking call	fine, scheduler handles it	blocks the loop	fine, thread unmounts
Colour problem	none	<code>async</code> / <code>await</code> splits your API	none
Cost each	~2KB	small	small, grows as needed

Virtual threads put Java where Go has been since the start: you write ordinary blocking, sequential code, and the runtime makes it scale. No `async` keyword splits the library in two.

The Lesson

Mounting and carriers

```
Thread.ofVirtual().name("worker").start(() -> ...);
```

Printing the thread shows something like `VirtualThread[#33,worker]/runnable@ForkJoinPool-1-worker-1`. That `ForkJoinPool-1-worker-1` is the **carrier**: a real platform thread the virtual thread is

currently mounted on.

When the virtual thread blocks, it unmounts and the carrier picks up another one. That is the whole mechanism. Blocking stopped costing a thread.

The executor you will actually use creates **one thread per task**, not a pool:

```
try (var executor = Executors.newVirtualThreadPerTaskExecutor()) { ... }
```

There is nothing to size, because the threads are not the scarce resource. From `VirtualThreads.java`:

```
100000 virtual threads, each sleeping 10ms: 216ms
```

The benchmark

From `Benchmark.java`, 5,000 tasks each blocking for 100ms on an 8-core machine:

virtual thread per task	127ms
platform pool of 200	2716ms
platform pool of 50	10750ms

The arithmetic is simple. A pool of 200 can have at most 200 tasks blocked at once, so 5,000 tasks take 25 batches of 100ms. A pool of 50 takes 100 batches. Virtual threads have no such limit, so all 5,000 block at once.

The choice this removes. Sizing a platform pool for IO was always a bad trade: too few threads and requests queue behind blocked ones, too many and you pay a megabyte of stack each while the OS scheduler thrashes.

The usual escape was asynchronous code, which works and costs you readable stack traces, step debugging, and ordinary `try / catch`. Virtual threads give the async scalability back to plain blocking code.

Where they do not help: CPU-bound work. A task that never blocks never unmounts, and you cannot run more work in parallel than you have cores. A fixed platform pool is still correct there.

Pinning, today

Pinning is when a virtual thread cannot unmount, so its carrier stays blocked and a platform thread is wasted.

In Java 21 to 23, blocking inside a `synchronized` block pinned the carrier, and every article from that era tells you to replace `synchronized` with `ReentrantLock`.

JEP 491, delivered in JDK 24, removed that. From `PinningToday.java`, 2,000 tasks each holding their own lock while sleeping 100ms:

```
cores, and therefore carriers: 8
elapsed: 131ms
```

If synchronized still pinned, only 8 tasks could sleep at once, so this would take about 25000ms.

What still pins:

1. **A native method or foreign function call.** The JVM cannot unmount a stack it does not control.
2. **A class initialiser that blocks.** Rare, and usually a design problem.

Both are far narrower than the old rule.

Pinning and contention are different things. A lock is still a lock: if a thousand virtual threads contend for one monitor, 999 wait. Pinning wastes a *carrier*; contention wastes *time*. JEP 491 fixed the first and could not touch the second, because serialising is what a lock is for.

Check your own code with `-Djdk.tracePinnedThreads=full` rather than trusting any article, including this one.

ScopedValue

Finalised in JDK 25. It replaces `ThreadLocal` for the common case.

The problem: a `ThreadLocal` is a mutable, unbounded, per-thread entry that lives until someone remembers to `remove()` it. With a few hundred pooled threads that was survivable. With a million virtual threads it is not.

```
private static final ScopedValue<String> CURRENT_USER = ScopedValue.newInstance();

ScopedValue.where(CURRENT_USER, "ada").run(() -> {
    handleRequest();           // CURRENT_USER.get() works at any depth
});
// binding is gone here, including if the body threw
```

There is no `remove()` to forget. The binding lasts exactly as long as `run()`.

It is immutable, which is the point. A `ThreadLocal` can be set by anything that can reach it, at any depth, and the change persists for the thread's life. A `ScopedValue` can only be shadowed by a nested scope, which is visible in the code.

One limitation worth knowing:

```
child thread sees it bound? false
```

A plain child thread does **not** inherit the binding, deliberately: the parent's scope could end while the child still runs. Inheritance requires structured concurrency, which is JEP 533 and still in preview on this

release. Module 22 covers it.

Use	For
<code>ScopedValue</code>	read-only value for the duration of a call: current user, request id
<code>ThreadLocal</code>	genuinely mutable per-thread state: a reusable buffer

Four misuses

From `CommonMisuses.java`:

- 1. Pooling them.** `newFixedThreadPool(200, Thread.ofVirtual().factory())` caps you at 200 and gains nothing. Pools reuse an expensive resource; these are not expensive.
- 2. Forgetting the pool was also your rate limit.** A platform pool did two jobs: providing threads and bounding concurrency. Virtual threads replace only the first. If a pool of 10 was protecting your database, removing it removes the protection, and you find out at 3am. Use a `Semaphore`, which states the constraint instead of implying it.
- 3. CPU-bound work.** Ten thousand virtual threads doing arithmetic will not beat your core count; they add scheduling overhead.
- 4. `ThreadLocal` at scale.** One entry per thread, and now there are a million.

Virtual threads make blocking cheap. They do not make sharing safe, and they are not a CPU multiplier. Everything in Module 18 about races, visibility and locks applies word for word.

Run It

```
java modules/19-virtual-threads/examples/VirtualThreads.java
java modules/19-virtual-threads/examples/Benchmark.java
java modules/19-virtual-threads/examples/PinningToday.java
java modules/19-virtual-threads/examples/ScopedValues.java
java modules/19-virtual-threads/examples/CommonMisuses.java

./scripts/verify-examples.sh modules/19-virtual-threads
```

The benchmark takes about fifteen seconds, most of it deliberately spent waiting in the platform-pool cases.

Common Mistakes

Pooling virtual threads. One per task; there is nothing to reuse.

Losing an unintended rate limit. Bound concurrency explicitly with a `Semaphore`.

Using them for CPU-bound work. Cores are still the limit.

Keeping `ThreadLocal` at scale. One entry per thread, and there are now millions.

Following advice to avoid `synchronized`. That was true before JDK 24 and is not now. Measure rather than assume.

Assuming they fix concurrency bugs. They fix thread *cost*. Races and visibility are untouched.

Expecting a child thread to inherit a `ScopedValue`. It does not, without structured concurrency.

Key Takeaways

- **A virtual thread unmounts when it blocks**, freeing its carrier. That is the whole idea.
- **One thread per task**, not a pool.
- **JEP 491 removed `synchronized` pinning in JDK 24.** Native calls still pin.
- **Pinning wastes a carrier; contention wastes time.** Different problems.
- **`ScopedValue` is immutable and scope-bounded**, with nothing to clean up.
- **They make blocking cheap, not sharing safe.** Module 18 still applies in full.

Homework

[homework/README.md](#)

Measure the difference yourself, then find the rate limit you accidentally removed. Reference solution in [solutions/19-virtual-threads/](#).

Homework 19: A Download Service

Measure the difference yourself, then find the rate limit you removed without noticing.

Part One: Measure It

Create `DownloadService.java` in the classic form.

Run 2,000 simulated downloads, each blocking for 100ms, three ways:

1. `newVirtualThreadPerTaskExecutor()`
2. `newFixedThreadPool(100)`
3. `newFixedThreadPool(25)`

Print all three timings.

Then **predict** the two platform numbers before looking at them. The formula is simple, and it comes from how many tasks can be blocked at once.

Print your predicted floors next to the measurements.

Explain It

A comment covering:

- Why the platform numbers are `(tasks / poolSize) × latency`
- Why the virtual number has no `poolSize` in it
- **The actual point.** The platform timings depend on a number you had to guess. Say what guessing it too low costs, and what guessing it too high costs.

Part Two: The Limit You Removed

A downstream service tolerates **5 concurrent calls**. Your old code used `newFixedThreadPool(5)`, so that limit was enforced by accident.

Write a `ConcurrencyTracker` that records the **peak** number of calls in flight at once. Then run 40 calls three ways and print the peak each time:

1. Platform pool of 5
2. Virtual threads, **no other change**
3. Virtual threads plus an explicit `Semaphore(5)`

The second one will hit 40. Nothing will fail, nothing will warn.

Your tracker's peak must itself be race-free. Two threads raising the peak at the same moment must not lose an update, and working out how to do that is part of the exercise.

Explain It

A comment covering:

- What two separate decisions the old pool size was making at once
- Why the `Semaphore` is better than the pool, even though both give 5
- **Where `release()` must go, and what happens if it is skipped on an exception.** Be specific about the failure mode

Part Three: Scoped Values

Bind a request id with `ScopedValue`, then read it **two frames deep** without passing it as a parameter.

Show that it is unbound outside the scope.

Then start a child thread inside the scope and show whether it sees the binding.

Explain It

A comment on:

- What you did **not** have to write, compared to a `ThreadLocal`
- What a `ThreadLocal` would still hold after the request finished, and why that matters when the thread is reused
- Why a child thread does not inherit the binding, and what would be needed for it to

Part Four: What Did Not Change

Run four virtual threads incrementing a plain `int` field 100,000 times each.

Print expected against actual.

Write a comment on why virtual threads did not help here, and on whether the risk of this bug went **up or down** now that spawning ten thousand threads is easy.

Acceptance Criteria

- ☐ Runs with `java DownloadService.java`
- ☐ Three timings printed, with predicted floors alongside
- ☐ The explanation covers both directions of guessing the pool size wrong

- ☐ `ConcurrencyTracker` records a peak and is itself race-free
- ☐ The three peaks are 5, 40 and 5
- ☐ A comment names the two decisions the pool size was conflating
- ☐ `release()` is in a `finally`, with a comment on the leak if it is not
- ☐ A scoped value is read two frames deep with no parameter passing
- ☐ It is shown unbound outside the scope
- ☐ The child-thread behaviour is shown and explained
- ☐ The unsafe counter is shown losing increments under virtual threads

Stretch

Replace the `Semaphore` with a bounded queue: a producer submitting work and a fixed number of virtual-thread consumers draining it.

Then write a comment comparing the two. Which one applies backpressure to the submitter? What happens under each when work arrives faster than it can be processed, and which failure would you rather debug?

Hint, if your tracker's peak looks too low

`peak = Math.max(peak, current)` is a read-modify-write, which is Module 18's race with different variable names. `AtomicInteger` has a method that retries until it wins; find it.

Module 20: Packages and Modules

Packages organise names. The classpath tells the JVM where to look. Modules add enforcement on top of both.

Most of what you need here is practical: enough to read a build file, diagnose a startup failure, and understand why `--add-opens` exists.

What You'll Learn

- The package-to-directory rule, and package-private as a real access level
- The classpath, and the two startup errors that are not the same thing
- Module import declarations, and when not to use them
- Building a jar, and what a module adds that a jar cannot

Coming From Another Language

	Python	Java
Namespace	module = file, package = dir	package, declared in the file
Layout enforced	by import path	by the compiler
Search path	<code>sys.path</code>	classpath or module path
Import everything	<code>from x import *</code>	<code>import x.*</code> , or <code>import module m</code>
Privacy across files	underscore convention	package-private, enforced
Distribution unit	wheel	jar

The enforced-layout row is the one that catches people. In Java the `package` statement and the directory path **must** match, and the compiler will tell you.

The Lesson

Packages

```
package com.example.util;    ->    com/example/util/Greeter.java
```

The compiler uses the directory to find sources it was not given, and the JVM uses it to find classes at runtime.

`Packages.java` builds a small two-package project in a temporary directory, compiles it with the real `javac`, and runs it:

```
javac exit code: 0
produced: [classes/com/example/app/Main.class, classes/com/example/util/Greeter.class]
<< Hello, world >>
```

Naming: reverse domain name, all lower case. The reversal exists so two organisations cannot collide.

Never use the default package for real code. A class with no `package` statement cannot be imported by any class that has one, so it is unusable from anywhere organised.

Package-private is a real access level, and the one people forget:

Level	Reach
<code>private</code>	this class
<code>(none)</code>	this package
<code>protected</code>	this package plus subclasses anywhere
<code>public</code>	everyone

It is how collaborating classes share something without exposing it to the world.

The classpath

A list of directories and jars, **searched in order**:

```
java -cp classes:lib/one.jar:lib/two.jar com.example.Main
java -cp "classes;lib/*" com.example.Main      (Windows uses ;)
```

The two errors are not the same. From `TheClasspath.java`:

ClassNotFoundException is a checked exception. Something asked for a class *by name* at runtime and it was not found. Usually reflection, a JDBC driver, or a framework loading a configured class.

NoClassDefFoundError is an Error. The class was present at compile time and is missing or unusable now. And here is the case people misdiagnose:

```
first attempt: ExceptionInInitializerError, caused by java.lang.IllegalStateException:
the static initialiser failed
second attempt: NoClassDefFoundError: Could not initialize class TheClasspath$Broken
```

A class whose static initialiser threw reports the real cause **once**. Every attempt after that says `NoClassDefFoundError` with no hint at all.

If you see `NoClassDefFoundError` for a class you are certain is on the classpath, scroll **up** the log. The real failure was reported earlier and looked unrelated.

The failure mode nobody expects: two jars containing the same class. The classpath is searched in order, the first wins, the second is silently ignored. Symptoms are a method that does not exist, or behaviour from a version you thought you had replaced. This is the problem the module system was built to solve.

Module import declarations

JEP 511, final in JDK 25:

```
import module java.base;
```

One line imports every public package that module exports. From `ModuleImports.java`, with no other import:

```
names:    [ada, grace, alan]
path:     /tmp/example.txt
time:     2026-09-23
```

It is a **compile-time convenience**. The bytecode is identical.

Use it for scripts, single-file programs and teaching examples. **Do not use it** in a real codebase: explicit imports document what a file depends on, so a reviewer can see when a file starts reaching somewhere new. It also makes ambiguity possible when two modules export the same simple name.

Jars and modules

From `JarsAndModules.java`:

```
jar --create --file tool.jar --main-class com.example.tool.Tool -C classes .
```

```
META-INF/MANIFEST.MF
com/example/tool/Tool.class
```

`MANIFEST.MF` is what `--main-class` wrote, which is how `java -jar` knows where to start.

A **module** adds a declaration:

```
module com.example.tool {
    requires java.logging;    // what it needs
    exports com.example.tool; // what others may use
}
```

Three things a plain jar cannot do:

1. **Strong encapsulation.** A package that is not exported is inaccessible from outside, even by reflection. `public` stops meaning universally reachable.
2. **Dependencies checked at startup.** A missing `requires` fails when the JVM launches, not as a `NoClassDefFoundError` an hour into production.
3. **No split packages.** Two modules cannot contain the same package, which is exactly the silent shadowing described above.

Why most projects still use the classpath: modularising means every dependency must be modular too, or be treated as an automatic module, which gets you name checking without encapsulation.

Where most developers actually meet the module system: an `InaccessibleObjectException` from a library reflecting into `java.base`, fixed with `--add-opens`. The practical minimum is knowing why that flag exists: `java.base` does not export its internals to you, and `--add-opens` overrides that.

The tools

```
jar --list --file x.jar           # see inside
jar --describe-module --file x.jar # is it modular
jdeps --list-deps x.jar          # what it needs
javadoc -d docs src/**/*.java   # generate documentation
```

Run It

```
java modules/20-packages-and-modules/examples/Packages.java
java modules/20-packages-and-modules/examples/TheClasspath.java
java modules/20-packages-and-modules/examples/ModuleImports.java
java modules/20-packages-and-modules/examples/JarsAndModules.java

./scripts/verify-examples.sh modules/20-packages-and-modules
```

Two of these build a real packaged project in a temporary directory, compile it with `javac`, and delete it afterwards. Nothing is left behind.

Common Mistakes

Package statement not matching the directory. The compiler says so; believe it rather than fighting it.

Using the default package. Nothing with a package can import you.

Making everything `public` by reflex. Package-private exists and is usually right for collaborating classes.

Diagnosing `NoClassDefFoundError` as a missing jar when a static initialiser failed earlier.

Two versions of a library on the classpath. First wins, silently.

`import module` in production code. It hides what a file depends on.

Adding `--add-opens` without understanding it. It is a hole punched in another module's encapsulation. Sometimes necessary, never free.

Key Takeaways

- **The package statement and the directory must match**, and the compiler enforces it.
- **Package-private is the default access level**, not an absence of one.
- **The classpath is searched in order**, so a duplicate class is shadowed silently.
- `ClassNotFoundException` means asked for by name; `NoClassDefFoundError` means it was there at compile time. A failed static initialiser causes the second and hides the first.
- `import module` is a compile-time convenience for scripts, not for codebases.
- **Modules add encapsulation, startup-time dependency checking and no split packages**, which is why `--add-opens` exists.

Homework

[homework/README.md](#)

Build a two-package project from scratch, jar it, then diagnose three startup failures. Reference solution in [solutions/20-packages-and-modules/](#).

Homework 20: Build It Yourself

Build a real two-package project from scratch, package it, then reproduce three startup failures so you have seen each one deliberately.

Everything must happen in a temporary directory that is deleted before the program exits.

Part One: Two Packages

Create `ProjectBuilder.java` in the classic form.

Have it **write, compile and run** a small project with this layout:

```
src/com/example/model/Item.java
src/com/example/app/Inventory.java
```

`Item` is a record with a name and a quantity. It must have:

- A **public** method other packages call
- A **package-private** helper the public method uses

`Inventory` imports `Item`, builds a few, and prints them.

Compile with `javac` through `ToolProvider`, list the class files produced, then load and run

`Inventory` through a `URLClassLoader` pointed at the output directory.

Add a comment inside `Inventory` showing the line that would **not** compile if it tried to call the package-private helper, with the real error text. Check it.

Explain It

A comment on why the directory path had to mirror the package statement. Say what `javac` uses it for and what the JVM uses it for. They are not the same reason.

Part Two: Package It

Build a jar with `--main-class` set, then list its contents.

Also run `jar --describe-module` against it and print what comes back.

Explain It

Two comments:

1. **What `--main-class` actually did.** Name the file it wrote into and say what would still work without it.
2. **What `--describe-module` reported.** You did not write a `module-info.java`, so what did the tool derive, from where, and why is that not real modularity?

Part Three: Three Startup Failures

Reproduce each, print what you caught, and explain it:

1. **`ClassNotFoundException` for a class that never existed.** Say why the compiler could not have warned you, and name two real situations that cause this.
2. **`ClassNotFoundException` for a class that exists but is not on the loader's path.** Build an empty `URLClassLoader` and ask it for a class you just compiled.

Then write the interesting comment: the symptom is identical to failure 1 and the cause is completely different. Say what this one corresponds to in a real deployment.

3. **`NoClassDefFoundError` from a failed static initialiser.** Touch the class twice and print both results.

This is the one worth getting right. Write a comment covering:

- What the first attempt reports, and where the real cause is hiding
- Why the second attempt reports something less useful
- **The diagnostic rule.** What should you do when you see `NoClassDefFoundError` for a class you are certain is present?

Part Four: Modules In Practice

Print, with labels:

- `String`'s class loader, and what `null` means there
- Your own class's loader
- `String`'s module name
- Your own module, and whether it is named

Explain It

A comment on:

- What the **unnamed module** is, and what it reads and exports
- **Why `--add-opens` exists.** Be specific: what does `java.base` not do, what exception does a library get as a result, and what does the flag change?

Acceptance Criteria

- ☐ Runs with `java ProjectBuilder.java` and deletes its temp directory
- ☐ Two packages are written, compiled and run
- ☐ `Item` has both a public and a package-private member
- ☐ The non-compiling call is documented with its real error text
- ☐ A comment explains the directory rule from both `javac`'s and the JVM's side
- ☐ A jar is built with `--main-class` and its contents listed
- ☐ `--describe-module` output is printed and explained
- ☐ All three failures are reproduced with their real messages
- ☐ Failures 1 and 2 are distinguished by cause, not symptom
- ☐ The static-initialiser case shows both attempts
- ☐ A diagnostic rule for `NoClassDefFoundError` is stated
- ☐ The unnamed module and `--add-opens` are both explained

Stretch

Add a real `module-info.java` that exports only `com.example.model` and **not** `com.example.app`. Compile it as a module and run it with `--module-path`.

Then try to reflect into the unexported package from outside and record what you get.

Write a comment on what strong encapsulation gave you that the jar in part two did not, and what it cost in exchange.

Hint, if failure 2 refuses to fail

An ordinary `new URLClassLoader(urls)` delegates to its parent first, and the parent can probably find your class. Pass `null` as the parent to stop the delegation, and then the loader really does have nowhere to look.

Module 21: Testing Your Code

This curriculum has a zero-dependency rule, and JUnit is a dependency. So this module teaches testing without one, which turns out to be the more useful lesson anyway: a test framework is not magic, and seeing the whole of a working one makes the real ones easier to reason about.

It also covers `assert`, mainly so you know why it is **not** a testing tool.

What You'll Learn

- Why every `assert` in your program does nothing by default
- The two ways `assert` gets misused, one of which is a real production bug
- A complete test harness in 39 lines
- What a framework adds that you cannot reasonably hand-roll

Coming From Another Language

	Python	Java
Built-in test runner	<code>unittest</code> , <code>pytest</code>	none in the JDK
<code>assert</code> in tests	the normal way	disabled by default
Discovery	by naming convention	by annotation, via a framework
Standard choice	<code>pytest</code>	JUnit 5, effectively universal

The row that matters: Python's `assert` is on unless you ask otherwise. Java's is off unless you ask for it, and almost nobody asks.

The Lesson

Assertions are off

From `AssertionsAreOff.java`:

```
without -ea:  assertions enabled: false
with -ea:    assertions enabled: true
```

The detection uses a trick worth recognising:

```
boolean enabled = false;
assert enabled = true;
```

The assignment only runs if assertions are enabled, because the whole statement is skipped otherwise. It is one of very few places where a side effect inside an `assert` is correct.

Two rules follow.

Never put logic inside an assert. This is a real bug:

```
assert list.remove(item);
```

It works in tests with `-ea` and silently stops removing anything in production.

Never use `assert` to validate arguments from callers. Use an exception, which is always on:

```
if (n < 0) throw new IllegalArgumentException("n must not be negative");
```

`assert` is for **internal invariants you believe cannot be false**: an unreachable switch default, a state your own code has just established. [UsingAssertions.java](#) shows both, and uses the message form, which costs one string and turns a null-message `AssertionError` into one that explains itself.

A note on this repository: the verification script runs examples with plain `java <file>`, so assertions are off. [UsingAssertions.java](#) carries an `// EXPECT: assertions` marker so the script runs it with `-ea`. Without that, an example teaching `assert` would pass by doing nothing at all.

A test harness in 39 lines

[MicroHarness.java](#) contains a complete `Tests` class. Its real output:

```
PASS    addition works
PASS    string concatenation
PASS    list size
FAIL    this one is wrong on purpose
        returned false
FAIL    this one throws
        IllegalStateException: boom
PASS    equality with a useful message
FAIL    and one that fails
        expected <5> but was <4>

7 run, 4 passed, 3 failed
```

Three design points worth noticing, because every real framework makes the same ones:

1. **A test that throws is a failure, not a crash of the run.** The harness catches `Throwable`, which is one of the few places that is correct: it must survive anything a test does to it.
2. **Failures carry a detail**, so `expected <5> but was <4>` appears rather than a bare `FAIL`.

3. **A real harness exits non-zero** when anything failed, so CI notices. This one does not, because it has to run clean under this repository's own verification.

What a framework adds

From `WhatJUnitAdds.java`:

1. Discovery. The micro harness needs every test registered by hand. Forget a line and the test silently does not exist, and nothing reports a test that was never registered. JUnit scans for `@Test` and that failure mode disappears.

2. Isolation. Every `@Test` gets a fresh instance, so state cannot leak between tests. Without it:

```
test one added an item; list size is now 1
test two starts with size 1, not 0
```

A test that passes only because a previous one ran is worse than no test, because it fails when someone reorders them.

3. Assertion messages. `assertEquals` on a ten-field object pointing at the differing field is the whole debugging session.

4. The things there is no room to hand-roll: parameterised tests, `assertThrows`, timeouts, tagging, IDE integration, and a standard report format CI can read.

```
class CalculatorTest {
    @Test
    void addsTwoNumbers() {
        assertEquals(4, Calculator.add(2, 2));
    }

    @Test
    void rejectsNegativeInput() {
        assertThrows(IllegalArgumentException.class, () -> Calculator.sqrt(-1));
    }
}
```

On a real project, add JUnit 5 on day one. It is the default everywhere and nothing about it is controversial. What carries over from this module is not the harness: it is knowing that a test is a method that checks something and reports, that isolation is a deliberate feature, and that `assert` is not a testing tool.

Run It

```
java modules/21-testing-your-code/examples/AssertionsAreOff.java
java -ea modules/21-testing-your-code/examples/AssertionsAreOff.java

java -ea modules/21-testing-your-code/examples/UsingAssertions.java
java modules/21-testing-your-code/examples/MicroHarness.java
java modules/21-testing-your-code/examples/WhatJUnitAdds.java

./scripts/verify-examples.sh modules/21-testing-your-code
```

Run the first one both ways. The difference is the lesson.

Common Mistakes

Assuming `assert` runs. It does not, unless someone passed `-ea`.

Side effects inside an `assert`. `assert list.remove(x)` stops working in production.

Validating a caller's arguments with `assert`. Use an exception.

Tests that depend on each other. The second passes because the first ran, until someone reorders them.

A test with no detail on failure. "FAIL" tells you nothing; the expected and actual values tell you everything.

A harness that dies on the first exception. One bad test should not take down the run.

Writing your own framework for a real project. Everything above is the argument for using JUnit, not against it.

Key Takeaways

- **Assertions are disabled by default**, so `assert` is not a testing mechanism.
- **Never put logic or argument validation in an `assert`.** Internal invariants only.
- **A test harness is a method that checks something and reports.** Thirty-nine lines is enough to prove it.
- **Catch `Throwable` in a harness** so one bad test does not end the run.
- **Failures must carry expected and actual**, or debugging starts from nothing.
- **Use JUnit on real projects.** This module is why it exists, not a substitute.

Homework

[homework/README.md](#)

Extend the harness with the features a framework would give you, then find the bug `assert` hides.

Reference solution in [solutions/21-testing-your-code/](#).

Homework 21: Build The Framework You Would Otherwise Import

Extend the micro harness until it does the things a framework does, then find the bug that `assert` hides in production.

Zero third-party dependencies. Everything here is JDK only.

Part One: A Better Harness

Create `TestKit.java` in the classic form.

Write a `Suite` class supporting:

- `test(String name, Runnable body)` — records a pass or a failure
- `assertEquals(expected, actual)` — failure detail must show **both** values
- `assertThrows(Class<? extends Throwable>, Runnable)` — passes only if that exact type is thrown

`assertThrows` has **two** failure cases, not one. Handle both, with different messages:

1. Nothing was thrown
2. Something was thrown, but the wrong type

Then run a suite containing at least: two passes, a failing `assertEquals`, both `assertThrows` failure modes, and a test that throws something unexpected.

Add **parameterised** tests: one behaviour checked against four rows of input, generated in a loop. Name each case with its inputs, and say in a comment why naming it `case 3` would be worse.

Print a summary line, and print the exit code a real harness would return.

Part Two: Isolation

Write a small mutable class, then run two checks against it **twice**:

1. Sharing one instance between them
2. Giving each its own

Show the second check failing in the shared version and passing in the fresh one.

Explain It

A comment on why an order-dependent test is **worse than no test at all**. Your answer should say what happens later, to whom, and why the absence of a test is more honest.

Part Three: The Bug `assert` Hides

Write a class with a method containing this:

```
assert items.remove(item) : "expected " + item + " to be present";
```

Run your program **without** `-ea` and print the collection size before and after. Then write the correct version and show it working.

Explain It

A comment covering:

- Why this passes every test you will write for it
- What happens in production, and why the failure does not look like a crash
- **The rule that prevents it**, stated in one sentence

Also print whether assertions were enabled, using the detection trick, so the output explains itself either way.

Part Four: Which Check, Where

Write two methods:

1. One validating a **public caller's argument**
2. One asserting an **internal invariant** your own code just established

Use an exception for the first and `assert` for the second.

Explain It

A comment giving the test you would apply to decide between them. One sentence per branch, and it should be a question you can actually answer at a keyboard.

Acceptance Criteria

- ☐ Runs with `java TestKit.java`, no third-party imports
- ☐ `Suite` supports `test`, `assertEquals` and `assertThrows`

- ☐ `assertThrows` distinguishes "nothing thrown" from "wrong type thrown"
- ☐ A test throwing unexpectedly is a failure, not a crash of the run
- ☐ Parameterised cases are named with their inputs
- ☐ A comment says why `case 3` would be a worse name
- ☐ The summary shows counts, plus the exit code a real harness would use
- ☐ Shared state makes the second check fail; fresh state makes it pass
- ☐ A comment explains why order-dependent tests are worse than none
- ☐ The assert-hidden removal is shown **not** happening without `-ea`
- ☐ The corrected version is shown working
- ☐ The one-sentence rule is stated
- ☐ Exception and assert are each used where they belong, with the deciding test written down

Stretch

Add a timeout to `test`, so a body that hangs is reported as a failure rather than hanging the run.

Then write a comment on what you had to introduce to make that work, and why a timeout is harder to implement than it looks. Consider what happens to the thread running the hung test after you report it.

Hint, if part three shows the removal happening

Check how you ran it. With `-ea` the assert executes and the removal works, which is exactly why this bug survives testing. Run it without the flag, which is how production runs.

Module 22: What's New, Honestly

This module is where the curriculum is honest about its own title.

Module 01 claimed that JDK 27 is the runtime and toolchain, while the language you learn is the stable Java accumulated through JDK 25. Twenty-one modules later, here is the evidence.

Every version claim below links to its source. That habit is the real subject of this module, because everything else in it will go out of date.

What You'll Learn

- What JDK 26 and 27 actually delivered, from the release pages
- The six-month cadence, LTS, and which number matters for what
- How to read a JEP, and what the preview round number tells you
- Structured concurrency and primitive patterns, run behind `--enable-preview`

Coming From Another Language

	Python	Node.js	Java
Release cadence	~annual	~biannual major	every six months, fixed date
Long-term support	~5 years per minor	even-numbered majors, 30 months	LTS every two years
Trying unfinished features	<code>from __future__ import</code>	flags, or a release channel	<code>--enable-preview</code>
Where the spec lives	PEPs	TC39 proposals	JEPs

The closest analogue is TC39's staged proposals. A Java preview is roughly a stage 3 proposal: specified, implemented, shipped behind a flag, and still able to change before it lands.

The difference worth knowing: a Python `__future__` import is forward compatible, while a Java **preview class file is rejected outright** by a different release, even with the flag. That single fact decides most of what this module recommends.

The Lesson

What actually shipped

JDK 26, GA 17 March 2026:

JEP	Title	Status
500	Prepare to Make Final Mean Final	Final
504	Remove the Applet API	Final
516	Ahead-of-Time Object Caching with Any GC	Final
517	HTTP/3 for the HTTP Client API	Final
522	G1 GC: Improve Throughput by Reducing Synchronization	Final
524	PEM Encodings of Cryptographic Objects	Second Preview
525	Structured Concurrency	Sixth Preview
526	Lazy Constants	Second Preview
529	Vector API	Eleventh Incubator
530	Primitive Types in Patterns	Fourth Preview

JDK 27, GA 15 September 2026:

JEP	Title	Status
523	Make G1 the Default Garbage Collector in All Environments	Final
527	Post-Quantum Hybrid Key Exchange for TLS 1.3	Final
531	Lazy Constants	Third Preview
532	Primitive Types in Patterns	Fifth Preview
533	Structured Concurrency	Seventh Preview
534	Compact Object Headers by Default	Final
536	JFR In-Process Data Redaction	Final
537	Vector API	Twelfth Incubator
538	PEM Encodings of Cryptographic Objects	Third Preview

Nothing in JDK 27 changes how you write Java. Four of its five finalised JEPs are runtime and security work: a collector default, smaller object headers, TLS key exchange, flight recorder redaction. You benefit from all four without editing a line. Everything language-shaped is still in preview, some of it for the seventh consecutive release.

A course promising "the new features of Java 27" would have had to pad the list or teach preview features as settled. Both are worse than saying so.

The two in JDK 26 worth knowing about: [JEP 517](#) gives the built-in `HttpClient` HTTP/3, a real capability you get by changing a version request rather than adding a dependency. [JEP 500](#) warns when

code mutates a `final` field by reflection, ahead of forbidding it, which is aimed squarely at some serialisation and mocking libraries.

Releases and LTS

A feature release every six months, in March and September, on a fixed date whether or not anything is ready. Before 2017, releases were feature-driven and arrived every two to three years.

Release	Designation	Date
Java 17	LTS	September 2021
Java 21	LTS	September 2023
Java 25	LTS	September 2025
Java 26		March 2026
Java 27		September 2026

A non-LTS gets security patches until the next release supersedes it. Six months, then nothing. An LTS gets years, which is why nearly every production system runs one.

"What is new in Java 27" is the wrong question for most teams. The right one is "what is new since Java 21", which is where they actually are.

Situation	Use
Learning	the newest release. Features finalise here first.
Production	the current LTS, today Java 25
A library	the oldest LTS you must support

The accumulation nobody notices. Any single release looks thin. Between Java 21 and Java 25, all of these arrived, and every one appears in this curriculum:

JEP	Feature	Module
512	Compact source files and instance main methods	01
511	Module import declarations	20
513	Flexible constructor bodies	05
506	Scoped values	19
485	Stream gatherers	14
491	Synchronize virtual threads without pinning	19

Use `Runtime.version()` to check what you are on, not the `java.version` string. Parsing that string broke for everyone when Java 9 changed its format from `1.8.0_301` to `9.0.1`.

How to read a JEP

Anything you read about Java goes out of date, including this file. JEPs do not, because they **are** the specification.

- openjdk.org/jeps/0 is the index
- `openjdk.org/jeps/<number>` is one JEP
- `openjdk.org/projects/jdk/<version>` is what shipped in a release

The fields, in order of usefulness:

Field	Why
Status	only <i>Closed/Delivered</i> means it shipped
Release	no Release field means it is not scheduled for anything
Summary	one paragraph, often all you need
Motivation	<i>why</i> it exists. The section people skip and the one worth reading.
Alternatives	what was rejected. Answers most "why didn't they just..." questions.

The vocabulary:

Term	Meaning
Preview	complete and specified, not yet permanent. Needs <code>--enable-preview</code> . May change or be withdrawn.
Incubator	an API in a <code>jdk.incubator</code> module. Needs <code>--add-modules</code> . Less settled than preview.
Experimental	a JVM feature behind <code>-XX:+UnlockExperimentalVMOptions</code>
Final	permanent. It will not be removed.

The preview round number is a practical signal, not trivia. Structured concurrency is on its seventh. Pattern matching for switch took four. Records took two. A high number means the design is still being argued about and the API has usually changed between rounds, which is exactly what happened here: code written against the earlier structured concurrency previews does not compile against this one.

Three questions to ask about any feature:

1. Is it Final? If not, what round, and how many came before?
2. Which release delivered it, and is that an LTS?
3. Can the minimum version you must support actually use it?

The preview features, run

Both examples below carry an `// EXPECT: preview` marker, so the verification script runs them with `-enable-preview --source 27`.

Structured concurrency, seventh preview. Module 18's `ExecutorService` lets you submit two tasks and forget one. Nothing connects them, and a `Future` nobody inspects swallows its exception.

```
try (var scope = StructuredTaskScope.open()) {
    var user = scope.fork(() -> loadUser());
    var orders = scope.fork(() -> loadOrders());
    scope.join();
    return combine(user.get(), orders.get());
}
```

```
user:ada and orders:3
```

```
scope failed: ExecutionException
cause: the fast one failed
```

The second case returns immediately. A two-second sibling was **cancelled** the moment the fast task threw, rather than being waited on pointlessly.

And it closes a loop from Module 19:

```
child sees: req-7
```

Module 19 said a plain child thread does not inherit a `ScopedValue`, and that inheritance needs structured concurrency. That is the line above. It works because the scope guarantees the parent outlives its children, so the binding cannot expire while a child is reading it.

Primitive patterns, fifth preview.

```
42 instanceof byte: true
300 instanceof byte: false
```

For a reference type, `instanceof` asks "is this object of that type". For a primitive it asks something different: **can this value convert without loss**.

Today that narrowing is a cast, and `(byte) 300` gives `44` with no warning at all. That is Module 02's overflow. The pattern form refuses rather than wrapping.

Should you use either in production? No. Preview class files are refused by a different release even with the flag, so every JDK upgrade may break your build. Read them, try them, and watch the JEP for the round where they finalise.

Run It

```
java modules/22-whats-new/examples/WhatChanged.java
java modules/22-whats-new/examples/ReleasesAndLts.java
java modules/22-whats-new/examples/HowToReadAJep.java

java --enable-preview --source 27 modules/22-whats-
    new/examples/PreviewStructuredConcurrency.java
java --enable-preview --source 27 modules/22-whats-
    new/examples/PreviewPrimitivePatterns.java

./scripts/verify-examples.sh modules/22-whats-new
```

Common Mistakes

Enabling preview features in production. A preview class file is rejected by a different JDK release even with the flag.

Chasing the newest release for a production system. Six months of patches, then nothing.

Reading a blog post instead of the JEP. The post was written against a different preview round.

Parsing `java.version`. Use `Runtime.version()`.

Assuming a preview API is stable because it works. Seven rounds means seven chances it changed.

Judging a release by its JEP count. The interesting question is what accumulated since the last LTS.

Key Takeaways

- **JDK 27 delivered nothing that changes how you write Java.** Its finalised work is runtime and security.
- **LTS is the number that matters for production;** the newest release is the one that matters for learning.
- **The value accumulates between LTS releases,** not within any single one.
- **A JEP is the specification and outlives every article about it.** Read the Motivation section.
- **The preview round number tells you how settled a design is.**
- **Never ship preview features.** The class files are rejected by other releases.

Homework

[homework/README.md](#)

Research a feature from primary sources and decide whether you could use it. Reference solution in [solutions/22-whats-new/](#).

Homework 22: Research It Yourself

The last homework in this curriculum is not about writing Java. It is about answering "can I use this?" from primary sources, because every other skill here has a shelf life and this one does not.

Part One: Three Features, Researched

Create `FeatureResearch.java` in the classic form.

Pick three features covered in this curriculum, at least one **Final** and at least one **Preview**. For each, find and record from openjdk.org:

Field	Where to find it
JEP number	the JEP index, or the release page
Status	the JEP's own header
Delivering release	the Release field
Is that release an LTS?	the release schedule

Print them as a table, and **print the URLs you used** underneath. A claim with no source does not count here.

Part Two: The Three Questions

Write a method applying the module's checklist to a feature:

1. Is it Final? If not, which round?
2. Which release delivered it, and is that an LTS?
3. What is the minimum version you must support?

It must print a verdict. Run it against a Final feature and a Preview one, assuming you must support **JDK 21**.

The Interesting Result

One of your two will be Final, shipped in an LTS, and **still unusable**.

Write a comment explaining why. Your answer must distinguish a feature being **safe** from a feature being **available**, because conflating those is how somebody ships a library nobody can depend on.

One implementation note. Take the versions as numbers, not strings. Comparing `"JDK 21"` with `"JDK 25"` as text happens to work; comparing `"JDK 9"` with `"JDK 25"` does not. Say in a comment why that is the same mistake as parsing `java.version`.

Part Three: Read A History Section

Pick a feature that has previewed more than twice. Open its JEP and find the **History** section.

List every round: JEP number, release, and whether it was an incubator or a preview.

Then Answer

- How many rounds in total, counting incubator ones?
- Did the JEP **number** change between rounds, and what does that tell you that a repeated number would not?
- Did anything surprise you about the history compared with what you assumed before reading it?

That last question is the point of the exercise. Write down what you expected and what you found.

Then compare with a feature that finalised quickly, and state the heuristic you would use in future.

Part Four: Check The Version Properly

Print `Runtime.version()`, its `feature()`, and the `java.version` property.

Then write a check for a minimum required version.

Explain It

A comment on why parsing `java.version` is a trap. Be specific: what was the format before Java 9, what is it now, which component meant the version number in each, and what happened to code that split on dots and took index 1.

Acceptance Criteria

- ☐ Runs with `java FeatureResearch.java`
- ☐ Three features with JEP number, status, release and LTS status
- ☐ The source URLs are printed
- ☐ The three-question method prints a verdict for a Final and a Preview feature
- ☐ One feature is Final, in an LTS, and still unusable, with the reason explained
- ☐ The safe-versus-available distinction is stated explicitly
- ☐ Versions are compared as numbers, with a comment on why
- ☐ A full preview history is listed, incubator rounds included
- ☐ The JEP-number-changing signal is explained
- ☐ What you expected versus what you found is written down
- ☐ A heuristic for future features is stated

☐ `Runtime.version()` is used, with the `java.version` trap explained

Stretch

Find a feature that was **withdrawn** after previewing, rather than finalised.

Write a comment on what its JEP says about why, and what that implies for anyone who had shipped code using it during the preview.

Hint, if you cannot find the History section

It is near the bottom of the JEP, below Alternatives and Risks. It is the most useful part of the document and the one nobody scrolls to.

Implementation Plan: Java 27 Learning Curriculum

Phase: 2 (Plan) of Specify → Plan → Tasks → Implement Spec: [SPEC.md](#) Date: 2026-09-23

Verified Preconditions

Check	Result
JDK 27 installed	<code>java version "27" 2026-09-15</code> , build 27+35-2325
JDK 27 is default	Yes, <code>/Library/Java/JavaVirtualMachines/jdk-27.jdk</code>
JEP 512 compact source files work without flags	Confirmed, <code>java HelloWorld.java</code> printed <code>Hello, World!</code> , exit 0
<code>gh</code> CLI authenticated	Yes, account <code>aliturgutbozkurt</code> , <code>repo</code> scope present

The Module 01 on-ramp is proven on the real toolchain, not assumed.

Decisions Taken on Open Questions

Open questions 2 and 3 from the spec were not answered explicitly, so the spec's own recommendations stand:

- **Testing (Module 21):** zero-dependency. Bare `assert` with `-ea` plus a hand-rolled micro harness, and a section explaining what JUnit adds and why real projects use it. The no-dependency rule holds.
- **Preview features:** covered only in Module 22, clearly labeled, run behind `--enable-preview --source 27`. Not removed, because a learner should know the difference between stable and preview, but quarantined so nothing earlier depends on them.

Say the word if either should flip.

Components

Component	Responsibility	Depends on
scaffold	Repo skeleton, verification script, gitignore, README stub	—
template	Module 01 written as the reference implementation of the module format	scaffold
core-language	Modules 02–10, the syntax and object model a Java newcomer needs first	template
type-system	Modules 11–12, generics and collections	core-language
functional	Modules 13–15, lambdas, streams, Optional	type-system
platform	Modules 16–21, IO, stdlib, concurrency, modules, testing	functional
release-literacy	Module 22, what 26 and 27 changed, JEPs, preview vs. stable	platform
integration	Root README table of contents, full-repo verification pass	all

Build order: scaffold → template → core-language → type-system → functional → platform → release-literacy → integration

Why This Order

The ordering is pedagogical, not technical. Module authoring could in principle happen in any order once the template exists, but each module's prose assumes the learner has read the previous ones. Writing them in sequence is what keeps cross-references honest, for example Module 14 on streams referring back to the functional interfaces introduced in Module 13.

Two exceptions worth noting:

- scaffold must land first and completely. The verification script is what proves every later example runs, so writing modules before it exists means accumulating unverified content.
- template is a gate, not just the first module. Module 01 fixes the seven-section README structure, the comment style, the homework format, and the solution layout. Getting it wrong means reworking 21 modules, so it gets reviewed before Module 02 starts.

Risks and Mitigations

Risk	Impact	Mitigation
Preview features drift before finalizing	Module 22 teaches something that changes	Quarantine to Module 22, label every preview explicitly, state the JEP number and preview round so the reader can check current status
Fabricated version claims	Curriculum teaches falsehoods with confidence	Spec boundary forbids it. Every JDK claim traces to openjdk.org. Already applied: the JDK 27 JEP table came from the release page, not memory
Scope fatigue across 22 modules	Project stalls half-finished	One module per issue, each independently shippable and verifiable. A half-done curriculum of 11 verified modules still teaches
Examples silently break on a future JDK	Rot	<code>verify-examples.sh</code> is runnable in CI and covers every example and solution
Template churn after modules are written	Expensive rework	Module 01 is an explicit review gate before Module 02
Learner's JDK differs from 27	Examples fail confusingly	Root README states the required JDK, verify script checks <code>java -version</code> and fails loudly on mismatch

Parallel vs. Sequential

- **Sequential:** scaffold → template → everything else. Also Module 13 before 14, and 08 before 09, because of direct content dependencies.
- **Parallelizable:** within `platform`, Modules 16, 17, 20 and 21 have no content dependency on each other and can be written in any order or by different people at once.

Verification Checkpoints

After	Check
scaffold	<code>./scripts/verify-examples.sh</code> exits 0 on an empty tree and fails loudly when given a deliberately broken file
template	Module 01 passes verification, and its README contains all seven sections
each module	<code>./scripts/verify-examples.sh modules/<id></code> exits 0, homework brief exists, reference solution runs clean
integration	Full-repo verify exits 0, and all eight spec success criteria are checked off one by one

Task List: Java 27 Learning Curriculum

Phase: 3 (Tasks) of Specify → Plan → Tasks → Implement **Spec:** [SPEC.md](#) · **Plan:** [plan.md](#)

Every task below touches at most 5 files and is independently verifiable. Each task maps to one GitHub issue.

scaffold

✓ T01 — Repository scaffolding and verification script · #1

- Acceptance: `scripts/verify-examples.sh` walks `modules/**/examples/*.java` and `solutions/**/*.*java`, runs each with JDK 27, honours the `// EXPECT: compile-error` and `// EXPECT: preview` headers, fails on any unexpected non-zero exit, and refuses to run if `java -version` is not 27. Root `README.md` stub states the JDK requirement and how to run an example. `.gitignore` excludes `*.class` and editor cruft.
- Verify: `./scripts/verify-examples.sh` exits 0 on the empty tree; planting a deliberately broken `.java` file makes it exit non-zero with a readable message; removing it restores exit 0.
- Files: `scripts/verify-examples.sh`, `README.md`, `.gitignore`

template

✓ T02 — Module 01 getting-started, the reference implementation · #2

- Acceptance: Covers JDK 27 setup, the JVM/JRE/JDK distinction, compact source files (JEP 512, final in 25), `IO.println`, and what compiling actually does. README has all seven template sections. At least two runnable examples. Homework brief with acceptance criteria. Reference solution under `solutions/01-getting-started/`. This module's structure is the template every later module copies.
- Verify: `./scripts/verify-examples.sh modules/01-getting-started` exits 0; solution runs clean; README section headings match the spec template exactly.
- Files: `modules/01-getting-started/README.md`, `modules/01-getting-started/examples/*.java`, `modules/01-getting-started/homework/README.md`, `solutions/01-getting-started/Solution.java`
- **Gate: human review of the format before T03 begins.**

core-language

✓ T03 — Module 02 variables-and-types · #3

- Acceptance: Primitives vs. references, `var` and where it helps or hurts, autoboxing traps including the Integer cache, String immutability, text blocks. Coming-from-another-language section contrasts Python/JS dynamic typing.
- Verify: `./scripts/verify-examples.sh modules/02-variables-and-types` exits 0.
- Files: `modules/02-variables-and-types/**`, `solutions/02-variables-and-types/**`

✓ T04 — Module 03 control-flow · #4

- Acceptance: Operators, `if / else`, switch expressions with arrow form and `yield`, all loop forms, enhanced `for`, labelled break. Contrasts switch expressions against the old fall-through statement form.
- Verify: `./scripts/verify-examples.sh modules/03-control-flow` exits 0.
- Files: `modules/03-control-flow/**`, `solutions/03-control-flow/**`

✓ T05 — Module 04 methods · #5

- Acceptance: Declaration, overloading and its resolution rules, varargs, static vs. instance, and an explicit treatment of Java being pass-by-value including for references, which is the single most misunderstood point for newcomers.
- Verify: `./scripts/verify-examples.sh modules/04-methods` exits 0.
- Files: `modules/04-methods/**`, `solutions/04-methods/**`

✓ T06 — Module 05 classes-and-objects · #6

- Acceptance: Classes, fields, constructors, `this`, encapsulation, static members, flexible constructor bodies (JEP 513, final in 25). Graduates the learner from compact source files to the classic `public class` plus `public static void main(String[] args)` and explains why every real codebase uses it.
- Verify: `./scripts/verify-examples.sh modules/05-classes-and-objects` exits 0.
- Files: `modules/05-classes-and-objects/**`, `solutions/05-classes-and-objects/**`

✓ T07 — Module 06 inheritance-and-polymorphism · #7

- Acceptance: `extends`, `super`, `@Override`, `abstract`, `final`, dynamic dispatch, and the `equals / hashCode / toString` contract with a worked example of getting it wrong.
- Verify: `./scripts/verify-examples.sh modules/06-inheritance-and-polymorphism` exits 0.
- Files: `modules/06-inheritance-and-polymorphism/**`, `solutions/06-inheritance-and-polymorphism/**`

✓ T08 — Module 07 interfaces · #8

- Acceptance: Interfaces vs. abstract classes, default/static/private methods, functional interfaces introduced as a concept ahead of Module 13, and why Java favours composition here.
- Verify: `./scripts/verify-examples.sh modules/07-interfaces` exits 0.
- Files: `modules/07-interfaces/**`, `solutions/07-interfaces/**`

✓ T09 — Module 08 records-and-enums · #9

- Acceptance: Records, compact constructors, what records generate for you, enums carrying behaviour and state, and a decision guide for class vs. record vs. enum.
- Verify: `./scripts/verify-examples.sh modules/08-records-and-enums` exits 0.
- Files: `modules/08-records-and-enums/**`, `solutions/08-records-and-enums/**`

✓ T10 — Module 09 sealed-and-pattern-matching · #10

- Acceptance: Sealed types, `instanceof` patterns, switch pattern matching, record patterns, and exhaustiveness checking. Depends on records from Module 08.
- Verify: `./scripts/verify-examples.sh modules/09-sealed-and-pattern-matching` exits 0.
- Files: `modules/09-sealed-and-pattern-matching/**`, `solutions/09-sealed-and-pattern-matching/**`

✓ T11 — Module 10 exceptions · #11

- Acceptance: Checked vs. unchecked and why Java is alone in this, try/catch/finally, try-with-resources, custom exceptions, and an explicit list of anti-patterns such as swallowing exceptions.
- Verify: `./scripts/verify-examples.sh modules/10-exceptions` exits 0.
- Files: `modules/10-exceptions/**`, `solutions/10-exceptions/**`

type-system

✓ T12 — Module 11 generics · #12

- Acceptance: Generic classes and methods, bounded types, wildcards with PECS, type erasure and the concrete limitations it imposes.
- Verify: `./scripts/verify-examples.sh modules/11-generics` exits 0.
- Files: `modules/11-generics/**`, `solutions/11-generics/**`

✓ T13 — Module 12 collections · #13

- Acceptance: `List`, `Set`, `Map`, `Deque`, sequenced collections, choosing the right implementation, the equals/hashCode contract applied to hash-based collections, and immutable factory methods.
- Verify: `./scripts/verify-examples.sh modules/12-collections` exits 0.
- Files: `modules/12-collections/**`, `solutions/12-collections/**`

functional

✓ T14 — Module 13 lambdas · #14

- Acceptance: Lambda syntax, method references in all four forms, the standard functional interfaces, and variable capture including the effectively-final rule.
- Verify: `./scripts/verify-examples.sh modules/13-lambdas` exits 0.
- Files: `modules/13-lambdas/**`, `solutions/13-lambdas/**`

✓ T15 — Module 14 streams · #15

- Acceptance: Pipeline anatomy, intermediate vs. terminal operations, laziness, collectors, gatherers, and an honest treatment of when parallel streams help and when they hurt. Depends on Module 13.
- Verify: `./scripts/verify-examples.sh modules/14-streams` exits 0.
- Files: `modules/14-streams/**`, `solutions/14-streams/**`

✓ T16 — Module 15 optional · #16

- Acceptance: `Optional` as a return type, the idiomatic operations, and the anti-patterns that make it worse than plain null such as calling `get` unguarded or using it as a field.
- Verify: `./scripts/verify-examples.sh modules/15-optional` exits 0.
- Files: `modules/15-optional/**`, `solutions/15-optional/**`

platform

✓ T17 — Module 16 files-and-io · #17

- Acceptance: `Path` and `Files`, reading and writing text, streaming large files, try-with-resources applied for real, and why the legacy `File` API is avoided.
- Verify: `./scripts/verify-examples.sh modules/16-files-and-io` exits 0; examples clean up any files they create.
- Files: `modules/16-files-and-io/**`, `solutions/16-files-and-io/**`

✓ T18 — Module 17 standard-library-tour · #18

- Acceptance: String methods and formatting, `java.time` in depth enough to avoid the classic date bugs, `Math`, random number generation, and comparators.
- Verify: `./scripts/verify-examples.sh modules/17-standard-library-tour` exits 0; no example depends on the current wall-clock time for its assertions.
- Files: `modules/17-standard-library-tour/**`, `solutions/17-standard-library-tour/**`

✓ T19 — Module 18 concurrency-basics · #19

- Acceptance: Threads, a demonstrable race condition, `synchronized`, atomics, `ExecutorService`, and the memory-visibility problem stated plainly.
- Verify: `./scripts/verify-examples.sh modules/18-concurrency-basics` exits 0; examples are deterministic enough not to flake, or state their nondeterminism and still exit 0.
- Files: `modules/18-concurrency-basics/**`, `solutions/18-concurrency-basics/**`

✓ T20 — Module 19 virtual-threads · #20

- Acceptance: Virtual threads (final in 21), why blocking became cheap again, scoped values (JEP 506, final in 25), pinning pitfalls, and a benchmark example contrasting platform and virtual threads.
- Verify: `./scripts/verify-examples.sh modules/19-virtual-threads` exits 0.
- Files: `modules/19-virtual-threads/**`, `solutions/19-virtual-threads/**`

✓ T21 — Module 20 packages-and-modules · #21

- Acceptance: Packages, classpath, the module system at a practical level, module import declarations (JEP 511, final in 25), plus `jar` and `javadoc` usage.
- Verify: `./scripts/verify-examples.sh modules/20-packages-and-modules` exits 0; any multi-file example documents its exact compile command.
- Files: `modules/20-packages-and-modules/**`, `solutions/20-packages-and-modules/**`

✓ T22 — Module 21 testing-your-code · #22

- Acceptance: `assert` with `-ea`, a hand-rolled micro test harness kept under 50 lines, what a real JUnit setup adds and why projects adopt it. Zero third-party dependencies, per the spec boundary.
- Verify: `./scripts/verify-examples.sh modules/21-testing-your-code` exits 0; harness demonstrates both a passing and a failing assertion without failing the overall run.
- Files: `modules/21-testing-your-code/**`, `solutions/21-testing-your-code/**`

release-literacy

✓ T23 — Module 22 whats-new · #23

- Acceptance: What JDK 26 and 27 actually changed, sourced from `openjdk.org` rather than memory. LTS vs. non-LTS and why 25 matters more than 27 for production. How to read a JEP. Preview and incubator features explained, with structured concurrency (JEP 533, seventh preview) and primitive patterns (JEP 532, fifth preview) demonstrated behind `--enable-preview`, each labelled with its JEP number and preview round.
- Verify: `./scripts/verify-examples.sh modules/22-whats-new` exits 0 including the preview-flagged examples; every version claim in the README carries a link to its `openjdk.org` source.
- Files: `modules/22-whats-new/**`, `solutions/22-whats-new/**`

integration

✓ T24 — Root README and final verification pass · #24

- Acceptance: Root `README.md` links all 22 modules in order with a one-line description each, documents JDK 27 setup per platform, and explains how to use the solutions tree. All eight spec success criteria checked off explicitly.

- Verify: `./scripts/verify-examples.sh` exits 0 across the whole repository; every module folder confirmed to contain README, examples, and homework; no non-English content found; no third-party dependency anywhere.
- Files: `README.md`, `SPEC.md` (success criteria checklist)

amendment-1 — post-delivery scope

✓ A01 — Amend the spec for PDF output and the Turkish translation

- Acceptance: SPEC.md records both capabilities, supersedes criterion 6 rather than leaving it falsely checked, states the build-tooling distinction for criterion 7, and adds criteria 9 through 14 with how each was verified.
- Verify: 13 criteria pass, 1 superseded; every new criterion measured by script rather than asserted.
- Files: `SPEC.md`, `tr/SPEC.md`, `tasks/todo.md`

✓ A02 — Restore English code blocks in the Turkish translation

- Acceptance: no `java` code block under `tr/` contains non-English text, and block counts align one-to-one with the English source.
- Verify: 35 violations found across 18 files and restored; re-scan reports zero. Turkish PDFs rebuilt.
- Files: 18 files under `tr/modules/`